

**BỘ GIÁO DỤC VÀ ĐÀO TẠO  
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI**

**Đỗ Quang Hợp**

**NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG  
EDGE AI TÍCH HỢP AI AGENT VÀ TỐI ƯU  
MÔ HÌNH YOLO**

**OPTIMIZING YOLO FOR EDGE AI AND DEVELOPING AN  
EDGE-AGENT MANAGEMENT SYSTEM WITH DASHBOARD  
AND AI AGENT**

**ĐỒ ÁN TỐT NGHIỆP**

Ngành: Khoa Học Máy Tính  
Khoa: Khoa Công Nghệ Thông Tin

**HÀ NỘI – 2026**

**BỘ GIÁO DỤC VÀ ĐÀO TẠO  
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI**

**Đỗ Quang Hợp**

**NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG  
EDGE  
AI TÍCH HỢP AI AGENT VÀ TỐI ƯU MÔ  
HÌNH  
YOLO**

**OPTIMIZING YOLO FOR EDGE AI AND DEVELOPING AN  
EDGE-AGENT MANAGEMENT SYSTEM WITH DASHBOARD  
AND AI AGENT**

**ĐỒ ÁN TỐT NGHIỆP**

**Ngành: Khoa Học Máy Tính  
Khoa: Khoa Công Nghệ Thông Tin**

**Giảng viên hướng dẫn: TS. Hoàng Nam Thắng  
Sinh viên thực hiện: Đỗ Quang Hợp  
Mã số sinh viên: 68CNCS  
Lớp: 68CS2**

**HÀ NỘI – 2026**

Trang này dùng để dán bản Nhận xét của người hướng dẫn, hoặc thay bằng bản nhận xét đã ký.

Trang này dùng để dán bản Nhận xét của người phản biện, hoặc thay bằng bản nhận xét đã ký.

# TÓM TẮT

**Tên đề tài:** NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG EDGE AI TÍCH HỢP AI AGENT VÀ TỐI ƯU MÔ HÌNH YOLO

**Sinh viên thực hiện:** Đỗ Quang Hợp

**Mã SV:** 68CNCS

**Lớp:** 68CS2

Đề án nghiên cứu đồng thời hai vấn đề: tối ưu mô hình YOLO cho thiết bị Edge AI và xây dựng nền tảng quản lý vòng đời AI trên một đội thiết bị phân tán. Ở phần mô hình, đề án xây dựng pipeline tối ưu YOLO26n cho bài toán phát hiện trạng bị bảo hộ lao động gồm: tìm siêu tham số bằng thuật toán Sea-Horse Optimizer, thừa hóa BatchNorm, cắt tỉa kênh có cấu trúc, phục hồi độ chính xác bằng chưng cất tri thức Channel-Wise Distillation, Quantization-Aware Training và chuyển đổi sang TensorRT INT8. Cắt tỉa 30% giảm 15,7% số tham số và 21,2% FLOPs của mô hình; các thực nghiệm sơ bộ kiểm chứng từng giai đoạn và cho thấy ảnh hưởng của lịch thừa hóa, ngân sách phục hồi và trọng số chưng cất.

Ở phần nền tảng, đề án xây dựng kiến trúc Cloud Control Plane – Edge Data Plane. Cloud Server (FastAPI, PostgreSQL, Redis) quản lý cluster, node, runtime và deployment theo mô hình desired state, giao tiếp với thiết bị qua MQTT. Edge Agent điều khiển container AI Runtime bằng các thao tác idempotent, lưu desired state, chống lệnh trùng và tự phục hồi runtime lỗi; AI Runtime chạy YOLO trên luồng camera và báo cáo telemetry theo một đặc tả thống nhất. Hệ thống có Web Dashboard theo dõi realtime qua WebSocket, giám sát hạ tầng bằng Prometheus và AI Assistant xây dựng trên LangGraph truy cập hệ thống qua 17 công cụ Model Context Protocol với cơ chế kiểm soát thao tác ghi. Toàn bộ được đóng gói container và triển khai tự động lên AWS EC2 bằng GitHub Actions; 17/18 kiểm thử tự động của luồng triển khai đầu cuối đạt yêu cầu.

**Từ khóa:** Edge AI, YOLO26, pruning, knowledge distillation, QAT, TensorRT, Jetson, MQTT, desired state, AI Agent, MCP.

Trang này dùng để dán bản Nhiệm vụ đồ án tốt nghiệp, hoặc thay bằng bản nhiệm vụ đã được phê duyệt.

# LỜI NÓI ĐẦU

Sự phát triển nhanh của trí tuệ nhân tạo tại biên đang mở ra khả năng xử lý dữ liệu hình ảnh trực tiếp tại nơi dữ liệu được sinh ra. Cách tiếp cận này giúp giảm độ trễ, hạn chế lưu lượng truyền về máy chủ và tăng khả năng hoạt động khi kết nối mạng không ổn định. Tuy nhiên, việc đưa mô hình phát hiện đối tượng YOLO lên thiết bị có tài nguyên hạn chế không chỉ là bài toán tối ưu mô hình. Một hệ thống thực tế còn cần cơ chế triển khai phiên bản, giám sát trạng thái, thu thập chỉ số, điều khiển từ xa và hỗ trợ người vận hành ra quyết định.

Từ bối cảnh đó, đồ án lựa chọn đề tài “Tối ưu mô hình YOLO cho Edge AI và xây dựng hệ thống quản lý Edge Agent tích hợp Dashboard và AI Agent”. Nội dung báo cáo tập trung vào hai phần có quan hệ chặt chẽ: tối ưu chuỗi suy luận để mô hình đáp ứng ràng buộc của thiết bị biên; đồng thời thiết kế nền tảng quản lý tập trung cho phép theo dõi, cấu hình và điều phối nhiều Edge Agent. AI Agent truy cập hệ thống qua các công cụ có kiểm soát, giúp người vận hành tra cứu trạng thái và thực hiện thao tác bằng ngôn ngữ tự nhiên sau khi xác nhận.

Trong quá trình thực hiện đồ án, em đã nhận được sự hướng dẫn và hỗ trợ của nhiều cá nhân, tập thể. Em xin trân trọng cảm ơn TS. Hoàng Nam Thắng đã định hướng chuyên môn, góp ý phương pháp thực hiện và dành thời gian nhận xét các phiên bản của đồ án. Em cảm ơn các thầy cô của Trường Đại học Xây dựng Hà Nội đã trang bị kiến thức nền tảng trong suốt quá trình học tập. Em cũng xin cảm ơn gia đình, bạn bè và các thành viên đã hỗ trợ thử nghiệm hệ thống. Mặc dù đã cố gắng hoàn thiện, báo cáo khó tránh khỏi thiếu sót; em mong nhận được ý kiến đóng góp để tiếp tục cải tiến kết quả nghiên cứu.

*Hà Nội, ngày 29 tháng 9 năm 2026*

Sinh viên thực hiện

Đỗ Quang Hợp

## CAM ĐOAN

Tôi xin cam đoan đồ án tốt nghiệp này là kết quả học tập, nghiên cứu và triển khai của bản thân dưới sự hướng dẫn của TS. Hoàng Nam Thắng. Các nội dung tham khảo, số liệu và kết quả của công trình khác được sử dụng trong báo cáo đều được trích dẫn nguồn rõ ràng. Các hình vẽ, bảng biểu và kết quả thực nghiệm do tôi tạo ra được trình bày trung thực; những dữ liệu chưa được đo kiểm được đánh dấu để bổ sung, không được sử dụng như kết quả đã xác nhận.

Tôi chịu trách nhiệm về tính trung thực, tính nguyên bản và việc tuân thủ quy định liên chính học thuật của toàn bộ nội dung đồ án. Nếu phát hiện có hành vi sao chép, giả mạo số liệu hoặc sử dụng kết quả của người khác mà không trích dẫn phù hợp, tôi xin hoàn toàn chịu trách nhiệm theo quy định của Nhà trường.

*Hà Nội, ngày 29 tháng 9 năm 2026*

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Đỗ Quang Hợp



# Mục lục

|                                                                            |            |
|----------------------------------------------------------------------------|------------|
| <b>LỜI NÓI ĐẦU</b>                                                         | <b>i</b>   |
| <b>LỜI CAM ĐOAN</b>                                                        | <b>ii</b>  |
| <b>MỤC LỤC</b>                                                             | <b>ii</b>  |
| <b>DANH SÁCH HÌNH VẼ</b>                                                   | <b>xi</b>  |
| <b>DANH SÁCH BẢNG</b>                                                      | <b>xii</b> |
| <b>DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT</b>                                 | <b>xv</b>  |
| <b>1 TỔNG QUAN ĐỀ TÀI</b>                                                  | <b>1</b>   |
| 1.1 Đặt vấn đề .....                                                       | 1          |
| 1.1.1 Sự phát triển của AI tại Edge .....                                  | 1          |
| 1.1.2 Hạn chế của triển khai AI trên Edge Device .....                     | 1          |
| 1.1.3 Bài toán quản lý và triển khai AI Model trên nhiều Edge Device ..... | 2          |
| 1.1.4 Sự cần thiết của tối ưu mô hình AI .....                             | 2          |
| 1.2 Mục tiêu đề tài .....                                                  | 3          |
| 1.2.1 Mục tiêu tổng quát .....                                             | 3          |
| 1.2.2 Mục tiêu cụ thể .....                                                | 3          |
| 1.3 Phạm vi đề tài .....                                                   | 3          |
| 1.4 Đối tượng nghiên cứu .....                                             | 4          |
| 1.5 Phương pháp nghiên cứu .....                                           | 4          |
| 1.6 Đóng góp của đề tài .....                                              | 5          |
| 1.7 Cấu trúc đồ án .....                                                   | 5          |
| 1.8 Kết chương .....                                                       | 6          |

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>2</b> | <b>CƠ SỞ LÝ THUYẾT</b>                      | <b>7</b> |
| 2.1      | Edge AI.....                                | 7        |
| 2.1.1    | Khái niệm Edge AI.....                      | 7        |
| 2.1.2    | Kiến trúc Cloud–Edge .....                  | 7        |
| 2.1.3    | Đặc điểm của Edge Device .....              | 8        |
| 2.1.4    | Các vấn đề khi triển khai AI trên Edge..... | 8        |
| 2.2      | Object Detection và YOLO.....               | 8        |
| 2.2.1    | Bài toán Object Detection.....              | 8        |
| 2.2.2    | Kiến trúc YOLO .....                        | 9        |
| 2.2.3    | Quy trình huấn luyện YOLO .....             | 9        |
| 2.2.4    | Các chỉ số đánh giá.....                    | 9        |
| 2.3      | Model Compression.....                      | 10       |
| 2.3.1    | Tổng quan Model Compression .....           | 10       |
| 2.3.2    | Model Pruning .....                         | 10       |
| 2.3.3    | Structured Pruning .....                    | 10       |
| 2.3.4    | Unstructured Pruning .....                  | 10       |
| 2.3.5    | Pruning Ratio.....                          | 11       |
| 2.3.6    | Fine-tuning sau Pruning.....                | 11       |
| 2.4      | Model Quantization .....                    | 11       |
| 2.4.1    | Khái niệm Quantization.....                 | 11       |
| 2.4.2    | FP32, FP16 và INT8 .....                    | 11       |
| 2.4.3    | Post-Training Quantization .....            | 12       |
| 2.4.4    | Quantization-Aware Training .....           | 12       |
| 2.4.5    | Fake Quantization.....                      | 12       |
| 2.4.6    | So sánh PTQ và QAT .....                    | 12       |
| 2.5      | TensorRT .....                              | 13       |
| 2.5.1    | Kiến trúc TensorRT .....                    | 13       |
| 2.5.2    | ONNX.....                                   | 13       |

|        |                                     |    |
|--------|-------------------------------------|----|
| 2.5.3  | TensorRT Engine.....                | 13 |
| 2.5.4  | FP16 Inference .....                | 13 |
| 2.5.5  | INT8 Inference .....                | 13 |
| 2.5.6  | TensorRT trên NVIDIA Jetson.....    | 14 |
| 2.6    | NVIDIA Jetson.....                  | 14 |
| 2.6.1  | Jetson Nano .....                   | 14 |
| 2.6.2  | Jetson Orin Nano.....               | 14 |
| 2.6.3  | So sánh tài nguyên phần cứng .....  | 15 |
| 2.6.4  | CUDA và GPU Acceleration .....      | 15 |
| 2.7    | AI Model Deployment.....            | 16 |
| 2.7.1  | Model Registry.....                 | 16 |
| 2.7.2  | Model Versioning .....              | 16 |
| 2.7.3  | Model Artifact.....                 | 16 |
| 2.7.4  | Deployment Strategy.....            | 16 |
| 2.7.5  | Rollback.....                       | 16 |
| 2.8    | Edge Runtime .....                  | 17 |
| 2.8.1  | AI Inference Pipeline .....         | 17 |
| 2.8.2  | Runtime Lifecycle.....              | 17 |
| 2.8.3  | GPU Resource Management.....        | 17 |
| 2.8.4  | Runtime Monitoring .....            | 18 |
| 2.9    | Distributed Edge Platform .....     | 18 |
| 2.9.1  | Cloud Control Plane.....            | 18 |
| 2.9.2  | Edge Agent.....                     | 18 |
| 2.9.3  | Telemetry .....                     | 18 |
| 2.9.4  | Message Broker .....                | 18 |
| 2.9.5  | Monitoring .....                    | 19 |
| 2.10   | RAG và AI Agent.....                | 19 |
| 2.10.1 | Retrieval-Augmented Generation..... | 19 |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 2.10.2   | Embedding Search .....                          | 19        |
| 2.10.3   | Hybrid Search .....                             | 19        |
| 2.10.4   | AI Agent.....                                   | 19        |
| 2.10.5   | Tool Calling .....                              | 20        |
| 2.10.6   | MCP .....                                       | 20        |
| 2.11     | Domain-Driven Design .....                      | 20        |
| 2.11.1   | Entity .....                                    | 20        |
| 2.11.2   | Value Object.....                               | 20        |
| 2.11.3   | Aggregate.....                                  | 21        |
| 2.11.4   | Repository .....                                | 21        |
| 2.11.5   | Domain Service .....                            | 21        |
| 2.11.6   | Bounded Context.....                            | 21        |
| 2.11.7   | Kiến trúc port–adapter .....                    | 21        |
| 2.12     | Kết chương .....                                | 22        |
| <b>3</b> | <b>NGHIÊN CỨU VÀ TỐI ƯU MÔ HÌNH YOLO</b>        | <b>23</b> |
| 3.1      | Bài toán tối ưu.....                            | 23        |
| 3.1.1    | Bài toán ứng dụng.....                          | 23        |
| 3.1.2    | Mô hình baseline .....                          | 23        |
| 3.1.3    | Hạn chế của mô hình baseline .....              | 23        |
| 3.1.4    | Mục tiêu tối ưu .....                           | 24        |
| 3.2      | Xây dựng mô hình YOLO baseline.....             | 24        |
| 3.2.1    | Dataset.....                                    | 24        |
| 3.2.2    | Tiền xử lý và tăng cường dữ liệu .....          | 25        |
| 3.2.3    | Tìm siêu tham số bằng Sea-Horse Optimizer ..... | 25        |
| 3.2.4    | Huấn luyện .....                                | 26        |
| 3.2.5    | Đánh giá baseline .....                         | 26        |
| 3.3      | Tối ưu bằng Pruning .....                       | 27        |
| 3.3.1    | Lựa chọn phương pháp pruning .....              | 27        |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 3.3.2    | Huấn luyện thừa hóa .....                     | 27        |
| 3.3.3    | Cắt tỉa theo ngưỡng toàn cục.....             | 27        |
| 3.3.4    | Phục hồi độ chính xác.....                    | 28        |
| 3.4      | Tối ưu bằng QAT.....                          | 29        |
| 3.4.1    | Thiết lập quantization .....                  | 29        |
| 3.4.2    | Hiệu chỉnh.....                               | 29        |
| 3.4.3    | Fake quantization và huấn luyện.....          | 29        |
| 3.4.4    | Export INT8.....                              | 29        |
| 3.4.5    | Đánh giá mô hình QAT .....                    | 30        |
| 3.5      | Pipeline tối ưu kết hợp .....                 | 30        |
| 3.5.1    | Quy trình .....                               | 30        |
| 3.5.2    | Ma trận cấu hình đánh giá .....               | 31        |
| 3.6      | Chuyển đổi sang TensorRT .....                | 31        |
| 3.6.1    | Export ONNX.....                              | 31        |
| 3.6.2    | TensorRT FP16 và INT8 .....                   | 32        |
| 3.6.3    | Kiểm tra độ chính xác sau chuyển đổi.....     | 32        |
| 3.7      | Lựa chọn mô hình triển khai.....              | 32        |
| 3.8      | Kết chương .....                              | 33        |
| <b>4</b> | <b>PHÂN TÍCH VÀ THIẾT KẾ NỀN TẢNG EDGE AI</b> | <b>34</b> |
| 4.1      | Yêu cầu hệ thống.....                         | 34        |
| 4.1.1    | Tác nhân và phạm vi .....                     | 34        |
| 4.1.2    | Yêu cầu chức năng .....                       | 34        |
| 4.1.3    | Yêu cầu phi chức năng.....                    | 35        |
| 4.2      | Kiến trúc tổng thể .....                      | 36        |
| 4.2.1    | Control Plane và Data Plane .....             | 37        |
| 4.2.2    | Giao thức MQTT và cấu trúc topic .....        | 37        |
| 4.2.3    | Tổ chức mã nguồn theo lớp.....                | 37        |

|       |                                          |    |
|-------|------------------------------------------|----|
| 4.3   | Thiết kế Domain .....                    | 38 |
| 4.3.1 | Mô hình phân cấp tài nguyên .....        | 38 |
| 4.3.2 | Cloud domain .....                       | 39 |
| 4.3.3 | Edge domain .....                        | 40 |
| 4.3.4 | Hợp đồng dùng chung Cloud–Edge .....     | 40 |
| 4.4   | Thiết kế Cloud Control Plane.....        | 41 |
| 4.4.1 | Fleet Service.....                       | 41 |
| 4.4.2 | Deployment Service.....                  | 41 |
| 4.4.3 | Các handler bất đồng bộ .....            | 42 |
| 4.4.4 | Monitoring và Realtime Service .....     | 42 |
| 4.4.5 | Triển khai tiến trình .....              | 42 |
| 4.5   | Thiết kế Edge Agent .....                | 43 |
| 4.5.1 | Vòng lặp điều khiển.....                 | 43 |
| 4.5.2 | Xử lý lệnh .....                         | 44 |
| 4.5.3 | Desired state và reconcile.....          | 44 |
| 4.5.4 | Khởi chạy runtime .....                  | 44 |
| 4.5.5 | API cục bộ.....                          | 45 |
| 4.6   | Thiết kế Edge Runtime.....               | 45 |
| 4.7   | Thiết kế Model Deployment.....           | 46 |
| 4.7.1 | Lưu trữ và phân phối model .....         | 46 |
| 4.7.2 | Luồng triển khai.....                    | 46 |
| 4.7.3 | Cập nhật và khôi phục .....              | 47 |
| 4.8   | Thiết kế Monitoring và cảnh báo .....    | 47 |
| 4.8.1 | Đặc tả telemetry .....                   | 48 |
| 4.8.2 | Metric hạ tầng với Prometheus .....      | 48 |
| 4.8.3 | Thiết kế mô-đun cảnh báo sớm .....       | 49 |
| 4.9   | Thiết kế AI Assistant.....               | 50 |
| 4.9.1 | Công cụ qua Model Context Protocol ..... | 50 |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| 4.9.2    | Kiểm soát thao tác thay đổi .....      | 51        |
| 4.9.3    | Đồ thị tác tử .....                    | 51        |
| 4.9.4    | So sánh với hướng tiếp cận RAG .....   | 52        |
| 4.10     | Kết chương .....                       | 52        |
| <b>5</b> | <b>XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG</b> | <b>53</b> |
| 5.1      | Môi trường và công nghệ .....          | 53        |
| 5.1.1    | Công nghệ sử dụng .....                | 53        |
| 5.1.2    | Môi trường Cloud .....                 | 54        |
| 5.1.3    | Môi trường Edge .....                  | 54        |
| 5.1.4    | Mạng .....                             | 54        |
| 5.1.5    | CI/CD .....                            | 55        |
| 5.2      | Xây dựng Cloud Server .....            | 55        |
| 5.2.1    | Điểm khởi chạy và vòng đời .....       | 55        |
| 5.2.2    | REST API .....                         | 56        |
| 5.2.3    | Tầng lưu trữ .....                     | 56        |
| 5.2.4    | Tiêu thụ MQTT .....                    | 57        |
| 5.2.5    | Kho model .....                        | 57        |
| 5.3      | Xây dựng Edge Agent .....              | 57        |
| 5.3.1    | Lắp ráp và cấu hình .....              | 57        |
| 5.3.2    | Giao tiếp MQTT .....                   | 57        |
| 5.3.3    | Điều khiển container .....             | 58        |
| 5.3.4    | Tải model .....                        | 58        |
| 5.3.5    | Thu thập telemetry .....               | 58        |
| 5.4      | Xây dựng AI Runtime .....              | 59        |
| 5.4.1    | Điểm khởi chạy .....                   | 59        |
| 5.4.2    | Suy luận YOLO .....                    | 59        |
| 5.4.3    | Đo lường .....                         | 59        |
| 5.4.4    | Đóng gói .....                         | 60        |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 5.5      | Xây dựng AI Assistant.....                    | 60        |
| 5.5.1    | Cấu trúc dịch vụ.....                         | 60        |
| 5.5.2    | Kết nối MCP .....                             | 60        |
| 5.5.3    | Prompt.....                                   | 60        |
| 5.5.4    | Biểu đồ và gợi ý.....                         | 61        |
| 5.5.5    | API và phát trực tuyến .....                  | 61        |
| 5.6      | Xây dựng hệ thống giám sát .....              | 61        |
| 5.6.1    | Telemetry ứng dụng.....                       | 61        |
| 5.6.2    | Metric hạ tầng .....                          | 61        |
| 5.6.3    | Dữ liệu cho cảnh báo sớm .....                | 62        |
| 5.7      | Giao diện quản lý .....                       | 62        |
| 5.8      | Kết chương.....                               | 63        |
| <b>6</b> | <b>THỰC NGHIỆM VÀ ĐÁNH GIÁ</b>                | <b>65</b> |
| 6.1      | Môi trường thực nghiệm .....                  | 65        |
| 6.1.1    | Môi trường huấn luyện.....                    | 65        |
| 6.1.2    | Quy trình đo trên thiết bị.....               | 65        |
| 6.2      | Kết quả tối ưu mô hình.....                   | 66        |
| 6.2.1    | Thực nghiệm sơ bộ trên COCO128.....           | 66        |
| 6.2.2    | Kết quả trên tập Construction-PPE .....       | 67        |
| 6.3      | Benchmark trên Jetson Nano .....              | 68        |
| 6.4      | Benchmark trên Jetson Orin Nano.....          | 68        |
| 6.5      | So sánh Jetson Nano và Jetson Orin Nano ..... | 68        |
| 6.6      | Benchmark Edge Runtime .....                  | 69        |
| 6.7      | Kiểm chứng nền tảng .....                     | 69        |
| 6.7.1    | Kiểm thử tự động.....                         | 69        |
| 6.7.2    | Hiệu năng vận hành .....                      | 71        |
| 6.8      | Kịch bản vận hành đầu cuối .....              | 72        |



|          |                                     |           |
|----------|-------------------------------------|-----------|
| 6.9      | Kết chương .....                    | 73        |
| <b>7</b> | <b>KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN</b> | <b>74</b> |
| 7.1      | Kết quả đạt được .....              | 74        |
| 7.1.1    | Tối ưu mô hình YOLO.....            | 74        |
| 7.1.2    | Nền tảng Edge AI .....              | 74        |
| 7.1.3    | AI Assistant .....                  | 75        |
| 7.2      | Hạn chế.....                        | 75        |
| 7.3      | Hướng phát triển .....              | 76        |
|          | <b>TÀI LIỆU THAM KHẢO</b>           | <b>78</b> |
| <b>A</b> | <b>CẤU TRÚC SOURCE CODE</b>         | <b>80</b> |
| <b>B</b> | <b>DATABASE SCHEMA</b>              | <b>82</b> |
| <b>C</b> | <b>API SPECIFICATION</b>            | <b>84</b> |
| <b>D</b> | <b>MCP TOOL SPECIFICATION</b>       | <b>86</b> |
| <b>E</b> | <b>MODEL TRAINING CONFIGURATION</b> | <b>87</b> |
| <b>F</b> | <b>BENCHMARK CONFIGURATION</b>      | <b>89</b> |
| <b>G</b> | <b>DEPLOYMENT CONFIGURATION</b>     | <b>90</b> |

## Danh sách hình vẽ

|     |                                                                 |    |
|-----|-----------------------------------------------------------------|----|
| 1.1 | Vòng đời AI tại Edge được nghiên cứu trong đồ án . . . . .      | 3  |
| 2.1 | Pipeline suy luận tại Edge . . . . .                            | 17 |
| 3.1 | Pipeline tối ưu YOLO26n từ baseline đến TensorRT INT8 . . . . . | 30 |
| 4.1 | Kiến trúc tổng thể nền tảng Edge AI . . . . .                   | 36 |
| 4.2 | Phân cấp tài nguyên của nền tảng . . . . .                      | 38 |
| 4.3 | Máy trạng thái của Deployment . . . . .                         | 39 |
| 4.4 | Các thành phần của Cloud Control Plane . . . . .                | 41 |
| 4.5 | Các thành phần chính của Edge Agent . . . . .                   | 43 |
| 4.6 | Thiết kế pipeline Edge Runtime . . . . .                        | 45 |
| 4.7 | Trình tự triển khai model từ Dashboard đến AI Runtime . . . . . | 47 |
| 4.8 | Luồng xử lý của mô-đun cảnh báo sớm suy giảm . . . . .          | 49 |
| 4.9 | Đồ thị xử lý một lượt hội thoại của AI Assistant . . . . .      | 51 |
| 7.1 | Các hướng phát triển từ kết quả hiện tại . . . . .              | 76 |

## Danh sách bảng

|     |                                                                                   |    |
|-----|-----------------------------------------------------------------------------------|----|
| 2.1 | So sánh PTQ và QAT . . . . .                                                      | 12 |
| 2.2 | Thông số hai thiết bị Jetson (theo đặc tả của NVIDIA [24]) . . . . .              | 15 |
| 3.1 | Tập dữ liệu Construction-PPE . . . . .                                            | 25 |
| 3.2 | Không gian tìm kiếm siêu tham số của SHO . . . . .                                | 26 |
| 3.3 | Kết quả chất lượng của mô hình baseline trên tập test Construction-PPE .          | 26 |
| 3.4 | Độ phức tạp của mô hình trước và sau cắt tỉa 30% (đo tại $640 \times 640$ ) . . . | 28 |
| 3.5 | Cấu hình các giai đoạn trong pipeline tối ưu . . . . .                            | 31 |
| 3.6 | Ma trận cấu hình tối ưu mô hình . . . . .                                         | 31 |
| 3.7 | Cấu hình model đề xuất triển khai . . . . .                                       | 33 |
| 4.1 | Yêu cầu chức năng của nền tảng . . . . .                                          | 34 |
| 4.2 | Cấu trúc topic MQTT giữa Cloud và Edge . . . . .                                  | 37 |
| 4.3 | Các entity chính của Cloud domain . . . . .                                       | 39 |
| 4.4 | Trạng thái của AI Runtime . . . . .                                               | 40 |
| 4.5 | Nhóm đặc trưng telemetry của Edge Node . . . . .                                  | 48 |
| 4.6 | Luật cảnh báo metric hạ tầng Edge . . . . .                                       | 49 |
| 5.1 | Các thành phần phần mềm của hệ thống . . . . .                                    | 53 |
| 5.2 | Các workflow CI/CD . . . . .                                                      | 55 |
| 5.3 | REST API của Cloud . . . . .                                                      | 56 |
| 5.4 | Các màn hình chính của Web Dashboard . . . . .                                    | 63 |
| 6.1 | Môi trường thực nghiệm . . . . .                                                  | 65 |
| 6.2 | Thực nghiệm thừa hóa BatchNorm trên COCO128 (CPU) . . . . .                       | 66 |
| 6.3 | Phục hồi độ chính xác sau cắt tỉa 30% trên COCO128 (CPU, ảnh 320) . .             | 66 |
| 6.4 | Ảnh hưởng của $\lambda_{cwd}$ (YOLO26n, 5 epoch, COCO128, ảnh 320) . . . . .      | 67 |
| 6.5 | Độ biến thiên theo seed (YOLO26n, 5 epoch, COCO128) . . . . .                     | 67 |

|      |                                                                                      |    |
|------|--------------------------------------------------------------------------------------|----|
| 6.6  | Kết quả các cấu hình tối ưu trên tập test Construction-PPE . . . . .                 | 68 |
| 6.7  | Suy luận YOLO26n trên Jetson Nano (ảnh 640, batch 1) . . . . .                       | 68 |
| 6.8  | Suy luận YOLO26n trên Jetson Orin Nano (ảnh 640, batch 1) . . . . .                  | 68 |
| 6.9  | Chỉ số pipeline AI Runtime lấy từ telemetry (cấu hình Pruned-30 + distill) . . . . . | 69 |
| 6.10 | Kết quả kiểm thử tự động của nền tảng . . . . .                                      | 70 |
| 6.11 | Hiệu năng vận hành của nền tảng . . . . .                                            | 71 |
| 6.12 | Kết quả kịch bản đầu cuối . . . . .                                                  | 72 |
| B.1  | Các bảng nghiệp vụ chính . . . . .                                                   | 82 |
| C.1  | Cloud REST API . . . . .                                                             | 84 |
| C.2  | Edge Agent API (cổng 8081) và AI Assistant API (cổng 8100) . . . . .                 | 85 |
| D.1  | Danh sách công cụ MCP . . . . .                                                      | 86 |
| F.1  | Cấu hình tối thiểu của một lần benchmark . . . . .                                   | 89 |

# DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT

| Ký hiệu | Ý nghĩa                                                                        |
|---------|--------------------------------------------------------------------------------|
| AI      | Artificial Intelligence – Trí tuệ nhân tạo                                     |
| API     | Application Programming Interface – Giao diện lập trình ứng dụng               |
| BN      | Batch Normalization – Chuẩn hóa theo lô                                        |
| CI/CD   | Continuous Integration/Continuous Deployment – Tích hợp và triển khai liên tục |
| CWD     | Channel-Wise Distillation – Chung cất tri thức theo kênh                       |
| EC2     | Amazon Elastic Compute Cloud – Dịch vụ máy ảo của AWS                          |
| GHCR    | GitHub Container Registry – Kho image container của GitHub                     |
| KD      | Knowledge Distillation – Chung cất tri thức                                    |
| PPE     | Personal Protective Equipment – Trang bị bảo hộ lao động                       |
| Q/DQ    | Quantize/Dequantize – Cặp nút lượng tử hóa/giải lượng tử trong ONNX            |
| SHO     | Sea-Horse Optimizer – Thuật toán tối ưu cá ngựa                                |
| SSE     | Server-Sent Events – Cơ chế đẩy sự kiện từ máy chủ qua HTTP                    |
| WS      | WebSocket – Giao thức truyền hai chiều trên một kết nối TCP                    |
| CUDA    | Compute Unified Device Architecture – Nền tảng tính toán GPU của NVIDIA        |
| CPU     | Central Processing Unit – Bộ xử lý trung tâm                                   |
| DDD     | Domain-Driven Design – Thiết kế hướng miền                                     |
| GPU     | Graphics Processing Unit – Bộ xử lý đồ họa                                     |
| NPU     | Neural Processing Unit – Bộ xử lý mạng nơ-ron                                  |
| Edge AI | Thực thi tác vụ trí tuệ nhân tạo trên hoặc gần nguồn sinh dữ liệu              |
| FP16    | Floating Point 16-bit – Số thực dấu phẩy động 16 bit                           |
| FP32    | Floating Point 32-bit – Số thực dấu phẩy động 32 bit                           |
| FPS     | Frames Per Second – Số khung hình xử lý trong một giây                         |
| INT8    | Integer 8-bit – Số nguyên 8 bit                                                |
| IoU     | Intersection over Union – Tỷ lệ giao trên hợp                                  |
| LLM     | Large Language Model – Mô hình ngôn ngữ lớn                                    |
| mAP     | mean Average Precision – Độ chính xác trung bình trên các lớp                  |

| Ký hiệu | Ý nghĩa                                                                   |
|---------|---------------------------------------------------------------------------|
| MCP     | Model Context Protocol – Giao thức kết nối mô hình với công cụ và dữ liệu |
| MQTT    | Message Queuing Telemetry Transport – Giao thức truyền thông điệp nhẹ     |
| ONNX    | Open Neural Network Exchange – Định dạng trao đổi mô hình học máy         |
| PTQ     | Post-Training Quantization – Lượng tử hóa sau huấn luyện                  |
| QAT     | Quantization-Aware Training – Huấn luyện có nhận biết lượng tử hóa        |
| RAG     | Retrieval-Augmented Generation – Sinh nội dung có tăng cường truy xuất    |
| RAM     | Random Access Memory – Bộ nhớ truy cập ngẫu nhiên                         |
| RBAC    | Role-Based Access Control – Kiểm soát truy cập theo vai trò               |
| REST    | Representational State Transfer – Phong cách kiến trúc dịch vụ web        |
| TLS     | Transport Layer Security – Giao thức bảo mật tầng truyền tải              |
| YOLO    | You Only Look Once – Họ mô hình phát hiện đối tượng một giai đoạn         |

# CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

Chương này trình bày bối cảnh hình thành đề tài, những khó khăn khi đưa mô hình trí tuệ nhân tạo xuống thiết bị biên và nhu cầu quản lý tập trung một đội thiết bị phân tán. Từ đó, chương xác định mục tiêu, phạm vi, đối tượng, phương pháp nghiên cứu và các đóng góp dự kiến của đồ án.

## 1.1 Đặt vấn đề

### 1.1.1 Sự phát triển của AI tại Edge

Các hệ thống thị giác máy tính trước đây thường truyền dữ liệu từ camera về máy chủ hoặc trung tâm dữ liệu để xử lý. Cách tiếp cận này thuận tiện cho việc tập trung tài nguyên nhưng tạo ra độ trễ truyền thông, tiêu thụ băng thông và làm tăng rủi ro lộ dữ liệu. Edge AI đưa mô hình suy luận đến gần nguồn sinh dữ liệu, cho phép thiết bị phản hồi ngay cả khi kết nối mạng không ổn định và chỉ gửi về đám mây các kết quả đã được chọn lọc.

Sự phổ biến của GPU nhúng, NPU và các bộ thực thi tối ưu đã mở rộng phạm vi ứng dụng Edge AI trong giám sát an toàn, giao thông, nông nghiệp, bán lẻ và tự động hóa công nghiệp. Trong các ứng dụng này, mô hình phát hiện đối tượng thuộc họ YOLO phù hợp nhờ khả năng xử lý một giai đoạn và cân bằng tương đối tốt giữa độ chính xác với tốc độ [1, 3]. Tuy nhiên, một mô hình chạy được trên máy trạm chưa đồng nghĩa với việc có thể vận hành ổn định trên thiết bị biên.

### 1.1.2 Hạn chế của triển khai AI trên Edge Device

Thiết bị biên bị giới hạn đồng thời bởi nhiều loại tài nguyên. Các giới hạn này có quan hệ với nhau và phải được xem xét trên toàn bộ chuỗi suy luận thay vì chỉ đo thời gian thực thi của riêng mạng nơ-ron.

- **Tài nguyên GPU:** số nhân xử lý và thông lượng tính toán thấp hơn máy chủ, trong khi GPU còn có thể được chia sẻ cho giải mã video, hiển thị và các tác vụ khác.
- **Bộ nhớ:** dung lượng RAM và bộ nhớ GPU giới hạn kích thước mô hình, batch size, số luồng camera và lượng bộ đệm có thể duy trì.
- **Điện năng:** thiết bị thường hoạt động theo power mode cố định; tăng xung nhịp có thể cải thiện FPS nhưng làm tăng công suất tiêu thụ.

- **Nhiệt độ:** tải suy luận kéo dài có thể làm thiết bị giảm xung để bảo vệ phần cứng, khiến hiệu năng thực tế thấp hơn phép đo ngắn hạn.
- **Yêu cầu thời gian thực:** hệ thống phải kiểm soát cả độ trễ trung bình, độ trễ phần vi cao và độ trễ đầu cuối từ camera đến kết quả.

Vì vậy, bài toán triển khai cần tối ưu đồng thời độ chính xác, tốc độ, kích thước mô hình, bộ nhớ và điện năng. Mọi so sánh phải được thực hiện trên cùng dữ liệu, cùng kích thước đầu vào, cùng thiết bị và cùng quy trình đo.

### ***1.1.3 Bài toán quản lý và triển khai AI Model trên nhiều Edge Device***

Khi số lượng thiết bị tăng, việc sao chép thủ công tệp mô hình và khởi động tiến trình qua từng phiên SSH không còn phù hợp. Người vận hành cần biết thiết bị nào đang trực tuyến, phiên bản mô hình đang chạy, cấu hình mong muốn, trạng thái thực tế và lịch sử thay đổi. Một lỗi triển khai phải được phát hiện, khoanh vùng và khôi phục mà không làm gián đoạn toàn bộ đội thiết bị.

Bài toán vì thế bao gồm cả mặt phẳng dữ liệu tại Edge và mặt phẳng điều khiển trên Cloud. Cloud Control Plane quản lý thiết bị, runtime, deployment, lệnh điều khiển và dữ liệu giám sát. Edge Agent tiếp nhận desired state, tải model, điều khiển AI Runtime và báo cáo trạng thái, telemetry. Mỗi thao tác phải có định danh, trạng thái và lịch sử để có thể truy nguyên khi xảy ra sự cố.

### ***1.1.4 Sự cần thiết của tối ưu mô hình AI***

Mua phần cứng mạnh hơn chỉ giải quyết một phần vấn đề và làm tăng chi phí khi quy mô đội thiết bị mở rộng. Tối ưu mô hình giúp khai thác hiệu quả tài nguyên hiện có, tăng số luồng camera có thể phục vụ và giảm thời gian phản hồi. Trong đề án này, pruning được dùng để loại bỏ phần tham số hoặc cấu trúc ít quan trọng; Quantization-Aware Training (QAT) giúp mô hình thích nghi với sai số lượng tử; TensorRT chuyển đổi đồ thị sang engine tối ưu cho GPU NVIDIA.

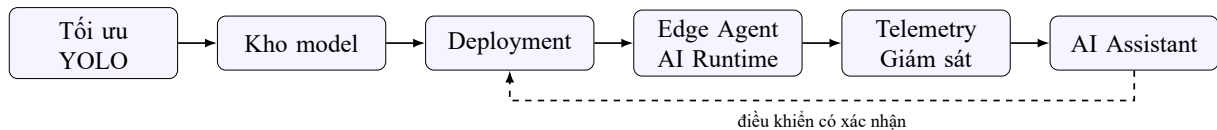
Tối ưu chỉ có ý nghĩa khi mức giảm chi phí suy luận không làm chất lượng nhận diện suy giảm vượt ngưỡng chấp nhận. Do đó, đề án không tìm mô hình nhỏ nhất hoặc nhanh nhất một cách độc lập mà tìm cấu hình có trade-off phù hợp giữa accuracy, latency, FPS, model size, GPU memory và power trên Jetson Nano và Jetson Orin Nano.



## 1.2 Mục tiêu đề tài

### 1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề án là tối ưu mô hình phát hiện đối tượng YOLO để chạy hiệu quả trên thiết bị Jetson, đồng thời xây dựng nền tảng quản lý tập trung cho phép triển khai, điều khiển, giám sát mô hình trên nhiều thiết bị biên và hỗ trợ vận hành bằng AI Assistant. Luồng tổng thể được thể hiện trong Hình 1.1.



Hình 1.1: Vòng đời AI tại Edge được nghiên cứu trong đề án

### 1.2.2 Mục tiêu cụ thể

1. Xây dựng mô hình YOLO26n baseline cho bài toán phát hiện trang bị bảo hộ lao động, với siêu tham số tìm bằng thuật toán Sea-Horse Optimizer.
2. Tối ưu mô hình bằng thừa hóa BatchNorm và cắt tỉa kênh có cấu trúc.
3. Phục hồi độ chính xác sau cắt tỉa bằng fine-tune và chưng cất tri thức.
4. Tối ưu bằng Quantization-Aware Training và chuyển đổi sang TensorRT INT8.
5. Đánh giá mô hình trên Jetson Nano và Jetson Orin Nano.
6. Xây dựng Cloud Control Plane quản lý cluster, node, runtime, deployment theo mô hình desired state, giao tiếp với Edge qua MQTT.
7. Xây dựng Edge Agent điều khiển vòng đời container AI Runtime với khả năng tự phục hồi, và AI Runtime chạy pipeline camera–suy luận có đo lường.
8. Xây dựng hệ thống telemetry, giám sát và Web Dashboard theo dõi realtime.
9. Xây dựng AI Assistant truy vấn và điều khiển hệ thống bằng ngôn ngữ tự nhiên qua công cụ MCP có kiểm soát.
10. Đóng gói container và tự động hóa triển khai bằng CI/CD.

## 1.3 Phạm vi đề tài

Đề án tập trung vào bài toán phát hiện trang bị bảo hộ lao động bằng YOLO26n trên hai thiết bị đại diện là Jetson Nano và Jetson Orin Nano. Phạm vi tối ưu gồm tìm siêu tham số, thừa hóa và cắt tỉa kênh, chưng cất tri thức, QAT, xuất ONNX và tạo TensorRT engine. Các phép đo tập trung vào độ chính xác, latency, FPS, độ phức tạp mô hình, bộ

nhớ GPU và công suất.

Ở tầng nền tảng, đồ án xây dựng các chức năng cốt lõi: quản lý cluster, node, runtime và deployment; điều khiển container tại biên; phân phối model; telemetry và giám sát; Web Dashboard; AI Assistant. Hệ thống được triển khai thực tế với Cloud trên một máy chủ AWS EC2 và các Edge Node thử nghiệm ở quy mô phòng thí nghiệm (máy trạm x86 có GPU NVIDIA và thiết bị Jetson), chưa hướng đến thay thế một nền tảng MLOps thương mại ở quy mô hàng nghìn node.

Đồ án không đặt mục tiêu nghiên cứu kiến trúc YOLO mới, phần cứng tăng tốc mới hay mô hình ngôn ngữ nền tảng. Triển khai theo nhóm/canary, rollback tự động, mô hình cảnh báo sớm bằng học máy và học liên tục được xác định là hướng phát triển.

## 1.4 Đối tượng nghiên cứu

Đối tượng nghiên cứu thứ nhất là mô hình phát hiện đối tượng YOLO26 và ảnh hưởng của cắt tia, chung cất, QAT, FP16, INT8 đến chất lượng nhận diện cũng như hiệu năng suy luận. Đối tượng thứ hai là pipeline AI Runtime tại biên, gồm thu nhận khung hình, suy luận và đo lường chất lượng vận hành.

Đối tượng thứ ba là kiến trúc nền tảng phân tán Cloud–Edge: Control Plane, Edge Agent, giao thức lệnh qua MQTT, desired state và actual state, telemetry, vòng đời container và model. Đối tượng thứ tư là AI Agent dùng tool calling qua Model Context Protocol để truy cập dữ liệu vận hành và điều khiển hệ thống trong phạm vi được cho phép.

## 1.5 Phương pháp nghiên cứu

Đồ án kết hợp nghiên cứu tài liệu, thực nghiệm mô hình và xây dựng hệ thống. Các tài liệu về Edge AI, YOLO, cắt tia, chung cất, lượng tử hóa, TensorRT, hệ thống phân tán, AI Agent và Domain-Driven Design được tổng hợp để hình thành cơ sở lý thuyết.

Đối với mô hình, phương pháp thực nghiệm có kiểm soát được sử dụng: giữ cố định tập dữ liệu, cách chia tập, kích thước ảnh và tiêu chí đo; sau đó thay đổi từng yếu tố như phương án phục hồi, trọng số chung cất hay precision. Thực nghiệm sơ bộ trên tập nhỏ được chạy trước để kiểm chứng pipeline, lặp với nhiều seed để đánh giá độ biến thiên, trước khi chạy tốn kém trên tập mục tiêu.

Đối với nền tảng, đồ án áp dụng Domain-Driven Design và kiến trúc port–adapter, hiện thực theo mô hình Cloud Control Plane – Edge Agent. Tính đúng đắn được kiểm

chứng bằng kiểm thử tự động chạy mã sản phẩm của Cloud và Agent với hạ tầng giả lập trong bộ nhớ, bao gồm các kịch bản triển khai thành công, xung đột, lỗi pull image, container bị crash, telemetry và xác thực; sau đó được kiểm chứng trên hệ thống triển khai thật.

### 1.6 Đóng góp của đề tài

Các đóng góp chính của đề án gồm:

1. Một pipeline tối ưu YOLO26 tái lập được gồm tìm siêu tham số bằng Sea-Horse Optimizer, thừa hóa BatchNorm, cắt tỉa kênh có cấu trúc, chưng cất CWD và QAT cho TensorRT INT8; kèm các module cắt tỉa và lượng tử hóa riêng cho kiến trúc YOLO26 được bổ sung vào Ultralytics.
2. Một nền tảng Cloud–Edge quản lý cluster, node, runtime và deployment theo mô hình desired state, với hợp đồng thông điệp dùng chung, cơ chế chống lệnh trùng và chống ghi đè kết quả cũ.
3. Một Edge Agent chạy đa kiến trúc, điều khiển container idempotent, lưu desired state, tự phục hồi runtime lỗi và phân phối model đã ghim phiên bản.
4. Một AI Runtime có đo lường chi tiết cùng đặc tả telemetry thống nhất, làm nền cho giám sát và thiết kế mô-đun cảnh báo sớm suy giảm.
5. Một AI Assistant truy cập hệ thống duy nhất qua công cụ MCP do Cloud công bố, có kiểm soát thao tác ghi.
6. Hệ thống được đóng gói và triển khai thực tế trên AWS EC2 bằng CI/CD, có bộ kiểm thử tự động cho luồng triển khai đầu cuối.

### 1.7 Cấu trúc đề án

Ngoài phần đầu báo cáo, tài liệu tham khảo và phụ lục, đề án gồm bảy chương. Chương 1 trình bày bối cảnh, mục tiêu, phạm vi và đóng góp. Chương 2 hệ thống hóa cơ sở lý thuyết theo pipeline từ Edge AI, YOLO, nén và lượng tử mô hình đến TensorRT, triển khai phân tán, RAG, AI Agent và Domain-Driven Design.

Chương 3 trình bày pipeline tối ưu YOLO26n: baseline với siêu tham số tìm bằng SHO, thừa hóa và cắt tỉa kênh, chưng cất tri thức, QAT, chuyển đổi TensorRT và lựa chọn model triển khai. Chương 4 phân tích yêu cầu, thiết kế domain và kiến trúc nền tảng Edge AI. Chương 5 mô tả quá trình xây dựng Cloud Server, Edge Agent, AI Runtime, AI Assistant, giám sát, Web Dashboard và CI/CD.

Chương 6 trình bày môi trường thực nghiệm, kết quả tối ưu mô hình, benchmark trên Jetson Nano và Jetson Orin Nano, và kiểm chứng nền tảng. Chương 7 tổng kết kết quả, phân tích hạn chế và đề xuất hướng phát triển.

### 1.8 Kết chương

Chương 1 đã xác định đề án là bài toán kết hợp giữa nghiên cứu tối ưu YOLO và xây dựng nền tảng vận hành AI phân tán. Các mục tiêu không dừng ở việc tăng FPS mà bao phủ toàn bộ vòng đời model từ huấn luyện, đóng gói, triển khai đến giám sát và xử lý sự cố. Chương tiếp theo trình bày các kiến thức nền tảng cần thiết để thiết kế và đánh giá giải pháp.

## CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này tổ chức cơ sở lý thuyết theo đúng pipeline của đề tài: từ đặc trưng Edge AI, mô hình YOLO, các kỹ thuật nén và lượng tử hóa, TensorRT và Jetson đến quản lý model deployment, Edge Runtime, nền tảng phân tán, RAG, AI Agent và Domain-Driven Design. Cách sắp xếp này tạo cầu nối trực tiếp từ kiến thức nền tảng sang nghiên cứu tối ưu ở Chương 3 và thiết kế nền tảng ở Chương 4.

### 2.1 Edge AI

#### 2.1.1 *Khái niệm Edge AI*

Edge AI là cách triển khai tác vụ trí tuệ nhân tạo trên thiết bị đặt gần nguồn sinh dữ liệu, thay vì gửi toàn bộ dữ liệu thô về đám mây. Thiết bị có thể là gateway, máy tính nhúng, camera thông minh hoặc máy công nghiệp. Lợi ích chính gồm giảm độ trễ, giảm băng thông, tăng khả năng hoạt động ngoại tuyến và hạn chế dữ liệu nhạy cảm rời khỏi hiện trường [10, 11].

Edge AI không loại bỏ vai trò của Cloud. Cloud vẫn phù hợp cho huấn luyện, lưu trữ artifact, quản lý đội thiết bị và phân tích dài hạn; Edge chịu trách nhiệm cho vòng lặp phản hồi nhanh. Do đó, giải pháp thực tế thường là một hệ thống lai trong đó Cloud và Edge có nhiệm vụ khác nhau nhưng chia sẻ cùng vòng đời model.

#### 2.1.2 *Kiến trúc Cloud–Edge*

Kiến trúc Cloud–Edge gồm ba lớp chính: Cloud Control Plane, kênh truyền thông và Edge Plane. Control Plane lưu desired state, model version, chính sách và lịch sử vận hành. Edge Plane chứa Edge Agent và AI Runtime, duy trì actual state và tiếp tục suy luận khi mất kết nối. Kênh truyền thông mang lệnh, sự kiện và telemetry; các tệp model dung lượng lớn có thể được phân phối qua object storage.

Kiến trúc cần chấp nhận kết nối không ổn định. Lệnh phải có định danh duy nhất, thời hạn hiệu lực và khả năng thực thi lặp an toàn. Edge Agent phải lưu trạng thái cục bộ, đồng bộ lại khi có mạng và không được dừng runtime đang khỏe chỉ vì Control Plane tạm thời không truy cập được.

### 2.1.3 Đặc điểm của Edge Device

Edge Device có tài nguyên nhỏ hơn máy chủ, thường dùng bộ nhớ chia sẻ giữa CPU và GPU, giới hạn điện năng và tản nhiệt. Phần mềm phụ thuộc chặt vào phiên bản hệ điều hành, CUDA, cuDNN, TensorRT và driver. Cùng một model artifact có thể không chạy được trên hai thiết bị nếu engine được tạo cho kiến trúc GPU hoặc phiên bản runtime khác nhau.

Thiết bị biên cũng nằm ngoài trung tâm dữ liệu nên dễ gặp mất nguồn, mất mạng, camera lỗi hoặc thay đổi môi trường. Vì vậy, khả năng quan sát, tự khởi động lại, rollback và xác thực artifact là một phần của yêu cầu hệ thống chứ không phải chức năng bổ sung.

### 2.1.4 Các vấn đề khi triển khai AI trên Edge

Năm nhóm vấn đề chính là năng lực tính toán, bộ nhớ, latency, power và thermal. Chúng tạo thành bài toán tối ưu đa mục tiêu: giảm số phép tính nhưng không làm accuracy giảm quá mức; giảm precision nhưng vẫn duy trì tính ổn định số; tăng FPS nhưng không gây quá nhiệt hoặc vượt ngân sách điện.

Ngoài model inference, hệ thống còn phải tính chi phí camera capture, chuyển đổi màu, resize, chuẩn hóa, non-maximum suppression, truyền kết quả và telemetry. Do đó, benchmark chỉ đo thời gian TensorRT engine là cần thiết nhưng chưa đủ để kết luận về khả năng đáp ứng thời gian thực của toàn hệ thống.

## 2.2 Object Detection và YOLO

### 2.2.1 Bài toán Object Detection

Object Detection xác định đồng thời lớp và vị trí của các đối tượng trong ảnh. Đầu ra thường là tập các bounding box  $b_i = (x_i, y_i, w_i, h_i)$ , nhãn lớp và độ tin cậy. Khác với phân loại ảnh, một ảnh có thể chứa nhiều đối tượng và mô hình phải giải quyết cả bài toán định vị lẫn phân loại.

Độ chồng lấp giữa hộp dự đoán  $A$  và hộp chuẩn  $B$  được đo bằng Intersection over Union:

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}. \quad (2.1)$$

Một dự đoán được xem là đúng khi lớp phù hợp và IoU vượt ngưỡng đánh giá.

### 2.2.2 Kiến trúc YOLO

YOLO là họ mô hình phát hiện đối tượng một giai đoạn, thực hiện trích xuất đặc trưng và dự đoán trong một đồ thị thống nhất [1, 2]. Kiến trúc tổng quát gồm backbone để trích xuất đặc trưng, neck để tổng hợp đặc trưng đa tỉ lệ và detection head để dự đoán vị trí, objectness và lớp.

Các biến thể YOLO khác nhau ở block tích chập, cơ chế kết hợp đặc trưng, hàm loss và cách biểu diễn detection head. Khi tối ưu, cần bảo toàn những tầng ảnh hưởng mạnh đến đầu ra, đồng thời ưu tiên cắt tỉa cấu trúc mà TensorRT có thể khai thác thành giảm thời gian thực thi thực tế.

### 2.2.3 Quy trình huấn luyện YOLO

Quy trình huấn luyện gồm chuẩn hóa nhãn, chia train–validation–test, tăng cường dữ liệu, khởi tạo trọng số, tối ưu loss và chọn checkpoint. Mọi thí nghiệm phải lưu seed, phiên bản mã nguồn, cấu hình optimizer, learning rate, batch size, image size và số epoch để có thể tái lập.

Tập test chỉ được dùng cho đánh giá cuối. Các quyết định về pruning ratio, lịch fine-tuning và ngưỡng dừng sớm phải dựa trên tập validation nhằm tránh điều chỉnh gián tiếp theo tập test.

### 2.2.4 Các chỉ số đánh giá

Với  $TP$ ,  $FP$  và  $FN$  lần lượt là số dự đoán đúng, dương tính giả và âm tính giả, Precision và Recall được xác định bởi:

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}. \quad (2.2)$$

Average Precision là diện tích dưới đường Precision–Recall của một lớp. mAP là trung bình AP trên các lớp.  $mAP@50$  dùng ngưỡng IoU 0,5;  $mAP@50:95$  lấy trung bình trên các ngưỡng từ 0,5 đến 0,95 với bước 0,05. Đồ án báo cáo đồng thời Precision, Recall,  $mAP@50$  và  $mAP@50:95$  để tránh kết luận dựa trên một chỉ số đơn lẻ.

## 2.3 Model Compression

### 2.3.1 Tổng quan Model Compression

Model Compression là nhóm kỹ thuật giảm chi phí lưu trữ hoặc suy luận của mạng nơ-ron. Các hướng phổ biến gồm pruning, quantization, knowledge distillation và thiết kế kiến trúc gọn. Đồ án tập trung vào pruning và quantization vì hai kỹ thuật này có thể áp dụng trực tiếp lên baseline và kết hợp với TensorRT.

Hiệu quả nén phải được đánh giá trên cả kích thước artifact, số phép tính lý thuyết và latency thực tế. Một mô hình có nhiều trọng số bằng không chưa chắc chạy nhanh hơn nếu backend vẫn thực hiện phép tính dense.

### 2.3.2 Model Pruning

Pruning loại bỏ trọng số, kênh hoặc cấu trúc được xem là ít quan trọng. Tiêu chí có thể dựa trên độ lớn trọng số, chuẩn của filter, độ nhảy loss hoặc thông tin gradient. Quy trình cơ bản gồm huấn luyện baseline, tính điểm quan trọng, cắt tỉa, fine-tuning và đánh giá lại [5, 6].

### 2.3.3 Structured Pruning

Structured pruning loại bỏ đơn vị có cấu trúc như channel, filter hoặc block. Kết quả là tensor có kích thước nhỏ hơn và có thể được thư viện dense hiện có khai thác mà không cần kernel thưa chuyên dụng. Đây là lựa chọn phù hợp khi mục tiêu cuối là TensorRT trên Jetson.

Nhược điểm là mỗi quyết định loại bỏ channel ảnh hưởng đến các tầng lân cận và có thể làm accuracy giảm nhanh. Việc cắt tỉa cần tôn trọng quan hệ residual, concatenation và detection head trong kiến trúc YOLO.

### 2.3.4 Unstructured Pruning

Unstructured pruning đặt các trọng số riêng lẻ về không và có thể đạt sparsity cao với mức suy giảm accuracy thấp hơn. Tuy nhiên, kích thước tensor không đổi; latency chỉ giảm rõ khi phần cứng và runtime hỗ trợ sparse kernel tương ứng.

Trong đồ án, unstructured pruning có thể được dùng làm đối chứng về sparsity, nhưng cấu hình triển khai ưu tiên structured pruning nếu mục tiêu là giảm latency thực



tế trên Jetson.

### 2.3.5 *Pruning Ratio*

Pruning ratio biểu diễn tỷ lệ phần tử hoặc cấu trúc bị loại bỏ:

$$r = \frac{N_{before} - N_{after}}{N_{before}} \times 100\%. \quad (2.3)$$

Các mức 20%, 30% và 40% được dùng làm điểm thực nghiệm ban đầu. Tỷ lệ này không mặc định áp dụng đồng đều cho mọi tầng; các tầng đầu, tầng đầu ra và nhánh nhảy cảm có thể được bảo vệ hoặc đặt ngưỡng riêng.

### 2.3.6 *Fine-tuning sau Pruning*

Sau pruning, phân bố đặc trưng bị thay đổi nên mô hình cần được fine-tune với learning rate nhỏ hơn baseline. Fine-tuning phục hồi accuracy trong khi duy trì cấu trúc đã cắt. Checkpoint tốt nhất được chọn theo validation mAP và phải được đánh giá lại trên tập test trước khi chuyển sang QAT hoặc TensorRT.

## 2.4 Model Quantization

### 2.4.1 *Khái niệm Quantization*

Quantization ánh xạ giá trị số thực có độ chính xác cao sang biểu diễn có số bit thấp hơn. Với lượng tử tuyến tính, một giá trị  $x$  được ánh xạ thành số nguyên  $q$  thông qua scale  $s$  và zero-point  $z$ :

$$q = \text{clip} \left( \text{round} \left( \frac{x}{s} \right) + z \right). \quad (2.4)$$

Việc giảm số bit làm giảm dung lượng và có thể tăng thông lượng, nhưng gây sai số lượng tử cần được kiểm soát.

### 2.4.2 *FP32, FP16 và INT8*

FP32 là biểu diễn phổ biến khi huấn luyện và được dùng làm chuẩn chất lượng. FP16 giảm một nửa dung lượng mỗi phần tử, phù hợp với Tensor Core và thường giữ

accuracy gần FP32. INT8 dùng số nguyên 8 bit, có tiềm năng giảm bộ nhớ và tăng tốc mạnh hơn nhưng nhạy với dải giá trị activation và yêu cầu calibration hoặc QAT.

### 2.4.3 *Post-Training Quantization*

Post-Training Quantization thực hiện lượng tử sau khi mô hình đã huấn luyện. PTQ không yêu cầu huấn luyện lại hoặc chỉ cần calibration dataset để ước lượng dải activation. Ưu điểm là quy trình nhanh; hạn chế là accuracy có thể giảm với tầng nhạy cảm, dữ liệu lệch hoặc mô hình nhỏ.

### 2.4.4 *Quantization-Aware Training*

QAT mô phỏng hiệu ứng lượng tử trong quá trình huấn luyện để trọng số thích nghi với sai số làm tròn và clipping [4]. Forward pass sử dụng các nút fake quantization, còn gradient được xấp xỉ để cập nhật trọng số thực. QAT tốn thêm thời gian huấn luyện nhưng thường duy trì accuracy INT8 tốt hơn PTQ.

### 2.4.5 *Fake Quantization*

Fake quantization lượng tử và giải lượng tử tensor trong forward pass nhưng vẫn lưu trọng số ở dạng số thực để tối ưu. Các observer thu thập thống kê min-max hoặc histogram nhằm xác định scale. Khi export, các thông tin này được chuyển thành toán tử lượng tử hoặc metadata phục vụ TensorRT.

### 2.4.6 *So sánh PTQ và QAT*

Bảng 2.1: So sánh PTQ và QAT

| Tiêu chí              | PTQ                        | QAT                                  |
|-----------------------|----------------------------|--------------------------------------|
| Dữ liệu cần thiết     | Tập calibration đại diện   | Dữ liệu huấn luyện/validation        |
| Chi phí thực hiện     | Thấp, không huấn luyện lại | Cao hơn do fine-tuning có fake quant |
| Khả năng giữ accuracy | Phụ thuộc độ nhạy mô hình  | Thường tốt hơn với INT8              |
| Độ phức tạp pipeline  | Đơn giản                   | Cần cấu hình và export tương thích   |
| Vai trò trong đồ án   | Baseline INT8 tham chiếu   | Phương pháp tối ưu chính             |

## 2.5 TensorRT

### 2.5.1 Kiến trúc TensorRT

TensorRT là bộ SDK tối ưu và thực thi suy luận cho GPU NVIDIA. Builder tiếp nhận đồ thị mạng, áp dụng tối ưu như fusion, lựa chọn kernel và precision, sau đó tạo serialized engine. Runtime nạp engine, cấp phát tensor và thực thi qua execution context [9].

Engine phụ thuộc vào kiến trúc GPU, phiên bản TensorRT, plugin và shape profile. Vì vậy, manifest của model artifact phải ghi rõ môi trường tạo engine và điều kiện tương thích.

### 2.5.2 ONNX

ONNX là định dạng trung gian giúp trao đổi mô hình giữa framework huấn luyện và runtime suy luận [8]. Pipeline của đồ án export checkpoint sang ONNX, kiểm tra graph, so sánh đầu ra với framework gốc rồi mới xây dựng TensorRT engine. Các toán tử không được hỗ trợ phải được thay thế, phân rã hoặc hiện thực bằng plugin.

### 2.5.3 TensorRT Engine

TensorRT engine là artifact đã được tối ưu cho một tập cấu hình cụ thể. Quá trình build có thể lựa chọn tactic khác nhau theo workspace và phần cứng. Engine cần đi kèm input shape, data type, tên binding, preprocessing, postprocessing và hàm băm để Edge Runtime tải đúng cách.

### 2.5.4 FP16 Inference

FP16 inference cho phép TensorRT dùng kernel bán chính xác ở các tầng được hỗ trợ và quay về precision khác khi cần. Đây thường là cấu hình triển khai đầu tiên vì quy trình chuyển đổi đơn giản hơn INT8 và mức suy giảm accuracy nhỏ. Kết quả vẫn phải được so sánh với FP32 trên cùng tập test.

### 2.5.5 INT8 Inference

INT8 inference yêu cầu scale activation phù hợp. Với PTQ, TensorRT dùng calibrator; với QAT, thông tin lượng tử được mang theo từ graph nếu pipeline export hỗ

trợ. Một số tầng có thể giữ FP16 hoặc FP32 để bảo toàn accuracy, tạo ra engine mixed precision.

### **2.5.6 *TensorRT trên NVIDIA Jetson***

Trên Jetson, TensorRT được phân phối cùng hệ sinh thái JetPack và sử dụng CUDA, cuDNN cùng driver tương ứng. Việc build engine trực tiếp trên thiết bị đích giúp giảm rủi ro không tương thích. Đồ án lưu riêng engine cho Jetson Nano và Jetson Orin Nano, thay vì giả định một engine có thể dùng chung cho mọi GPU.

## **2.6 NVIDIA Jetson**

### **2.6.1 *Jetson Nano***

Jetson Nano đại diện cho nhóm thiết bị Edge AI có tài nguyên hạn chế. Thiết bị phù hợp để đánh giá mức tối thiểu mà model có thể vận hành, đồng thời làm rõ ảnh hưởng của bộ nhớ, power mode và thermal throttling. Phiên bản JetPack, CUDA, TensorRT, dung lượng RAM và chế độ công suất phải được ghi theo đúng thiết bị thực nghiệm, vì các biến thể phần cứng có thể khác nhau.

### **2.6.2 *Jetson Orin Nano***

Jetson Orin Nano đại diện cho thế hệ GPU nhúng mới hơn, có kiến trúc và năng lực tăng tốc khác Jetson Nano. Thiết bị được dùng để đánh giá khả năng mở rộng hiệu năng của cùng pipeline model, đồng thời kiểm tra tính di động của cơ chế triển khai và Edge Runtime giữa hai lớp node.

### 2.6.3 So sánh tài nguyên phần cứng

Bảng 2.2: Thông số hai thiết bị Jetson (theo đặc tả của NVIDIA [24])

| Thành phần       | Jetson Nano 4GB                     | Jetson Orin Nano 8GB                    |
|------------------|-------------------------------------|-----------------------------------------|
| Kiến trúc GPU    | NVIDIA Maxwell                      | NVIDIA Ampere                           |
| Nhân GPU         | 128 CUDA core, không có Tensor Core | 1024 CUDA core, 32 Tensor Core thế hệ 3 |
| CPU              | 4 nhân ARM Cortex-A57, 1,43 GHz     | 6 nhân ARM Cortex-A78AE                 |
| RAM              | 4 GB LPDDR4 64-bit, 25,6 GB/s       | 8 GB LPDDR5 128-bit, 68 GB/s            |
| Năng lực AI      | 472 GFLOPS (FP16)                   | 40 TOPS INT8 (67 TOPS ở chế độ Super)   |
| JetPack mới nhất | 4.6.x (L4T R32.7)                   | 6.x (L4T R36)                           |
| CUDA / TensorRT  | 10.2 / 8.2                          | 12.x / 8.6–10.x                         |
| Power mode       | 5 W, 10 W (MAXN)                    | 7 W, 15 W, 25 W (Super)                 |
| Hỗ trợ INT8      | Không có đơn vị INT8 chuyên dụng    | Tensor Core hỗ trợ INT8                 |

Khác biệt quan trọng nhất đối với tối ưu mô hình là Jetson Nano không có Tensor Core và phần cứng không hỗ trợ tăng tốc INT8, nên trên thiết bị này lợi ích chủ yếu đến từ FP16 và từ việc giảm FLOPs bằng cắt tỉa; còn trên Jetson Orin Nano, INT8 qua Tensor Core là nguồn tăng tốc chính. Jetson Nano cũng dùng ở JetPack 4.6 (Ubuntu 18.04, Python 3.6 mặc định), trong khi Orin Nano dùng JetPack 6 (Ubuntu 22.04); điều này buộc image runtime phải build riêng cho từng loại thiết bị. Phiên bản phần mềm và power mode cụ thể khi đo được ghi lại trong Bảng 6.1.

### 2.6.4 CUDA và GPU Acceleration

CUDA cung cấp mô hình lập trình và runtime để thực thi kernel trên GPU NVIDIA. Trong pipeline, TensorRT sử dụng CUDA để lập lịch kernel và truyền dữ liệu. Tối ưu end-to-end cần hạn chế sao chép CPU–GPU, tái sử dụng bộ đệm, dùng stream phù hợp và đồng bộ chỉ tại những điểm cần thiết.

GPU acceleration không tự động bảo đảm latency thấp. Kích thước input, batch, memory allocation, camera decode và hậu xử lý trên CPU có thể trở thành nút thắt. Vì vậy, đồ án đo từng giai đoạn của pipeline và tổng thời gian đầu cuối.

## 2.7 AI Model Deployment

### 2.7.1 *Model Registry*

Model Registry là kho quản lý metadata và artifact của các phiên bản mô hình. Mỗi bản ghi liên kết model name, version, framework, precision, input shape, chỉ số đánh giá, vị trí artifact, checksum và ma trận tương thích thiết bị. Registry không chỉ là thư mục lưu tệp mà là nguồn sự thật cho quyết định triển khai.

### 2.7.2 *Model Versioning*

Model versioning giúp phân biệt các checkpoint, cấu hình tối ưu và engine. Một phiên bản đã phát hành phải bất biến; thay đổi nội dung yêu cầu tạo phiên bản mới. Ngoài version dễ đọc, hệ thống lưu checksum để nhận diện chính xác bytes của artifact và ngăn trường hợp hai tệp khác nhau dùng chung tên.

### 2.7.3 *Model Artifact*

Artifact triển khai có thể gồm TensorRT engine, ONNX dự phòng, label map, manifest, cấu hình preprocessing–postprocessing và chữ ký hoặc checksum. Gói phải tự mô tả đủ để Edge Agent kiểm tra tính tương thích trước khi dùng runtime hiện tại.

### 2.7.4 *Deployment Strategy*

Chiến lược triển khai xác định thứ tự và phạm vi node nhận model. Các lựa chọn gồm triển khai đồng loạt, rolling deployment và canary. Với đội thiết bị phân tán, canary giúp kiểm tra một nhóm nhỏ trước khi mở rộng, đồng thời đặt ngưỡng dừng dựa trên health check, FPS, error rate hoặc nhiệt độ.

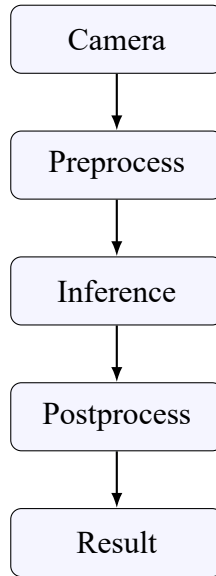
### 2.7.5 *Rollback*

Rollback đưa node về phiên bản đã biết là ổn định khi download, validation, activation hoặc health check thất bại. Edge Agent cần giữ ít nhất một artifact trước đó, cập nhật con trỏ active theo cách nguyên tử và báo cáo nguyên nhân. Cơ chế rollback phải được kiểm thử như một luồng bình thường, không chỉ dùng khi sự cố đã xảy ra.

## 2.8 Edge Runtime

### 2.8.1 *AI Inference Pipeline*

Edge Runtime biến frame từ camera thành detection theo chuỗi trong Hình 2.1.



Hình 2.1: Pipeline suy luận tại Edge

Preprocess phải khớp với huấn luyện về resize, letterbox, thứ tự kênh và chuẩn hóa. Postprocess giải mã tensor, áp dụng confidence threshold và non-maximum suppression. Sai khác ở hai bước này có thể làm giảm accuracy dù engine cho đầu ra số học chính xác.

### 2.8.2 *Runtime Lifecycle*

Runtime lifecycle gồm các trạng thái created, loading, ready, running, degraded, stopping, stopped và failed. Mọi chuyển trạng thái phải có điều kiện hợp lệ và reason code. Edge Agent không coi tiến trình đang tồn tại là đủ; health check cần xác nhận camera, engine và vòng xử lý frame đều hoạt động.

### 2.8.3 *GPU Resource Management*

Quản lý GPU bao gồm cấp phát bộ nhớ, số execution context, stream, batch size và số runtime đồng thời. Hệ thống cần từ chối hoặc trì hoãn cấu hình vượt khả năng node thay vì để thiết bị rơi vào out-of-memory. Model manifest và node capability được dùng để kiểm tra trước khi activation.

### **2.8.4 Runtime Monitoring**

Runtime phát sinh các chỉ số FPS, latency từng stage, queue depth, frame drop, GPU usage, GPU memory, nhiệt độ, power và error count. Metrics cần gắn node, runtime, model version và timestamp. Cửa sổ tổng hợp giúp phát hiện suy giảm kéo dài thay vì tạo incident từ một mẫu nhiễu đơn lẻ.

## **2.9 Distributed Edge Platform**

### **2.9.1 Cloud Control Plane**

Cloud Control Plane duy trì trạng thái mong muốn và điều phối các domain nghiệp vụ. Thành phần này cung cấp API cho Dashboard và AI Assistant, lưu thông tin thiết bị, runtime, lịch sử deployment và sự kiện. Control Plane không tham gia trực tiếp vào vòng lặp inference nên Edge vẫn hoạt động khi Cloud tạm thời gián đoạn.

### **2.9.2 Edge Agent**

Edge Agent là đại diện quản lý trên mỗi node. Agent đăng ký danh tính, nhận lệnh, đồng bộ desired state, tải artifact, điều khiển runtime và gửi actual state. Agent phải giới hạn quyền hệ thống, xác thực nguồn lệnh và bảo đảm một command không bị thực thi hai lần khi message broker gửi lại.

### **2.9.3 Telemetry**

Telemetry gồm metric, log, event và state report. Metric phục vụ theo dõi xu hướng; event ghi nhận thay đổi rời rạc; log hỗ trợ chẩn đoán chi tiết; state report đối chiếu desired state với actual state. Khi mất mạng, dữ liệu quan trọng được đệm cục bộ theo dung lượng giới hạn và gửi lại theo thứ tự phù hợp.

### **2.9.4 Message Broker**

Message broker tách Control Plane khỏi kết nối trực tiếp đến từng node. MQTT phù hợp với thiết bị biên nhờ mô hình publish–subscribe và mức QoS linh hoạt [12]. Topic phải được phân vùng theo tenant/node, áp dụng TLS và quyền publish/subscribe tối thiểu.



### **2.9.5 Monitoring**

Monitoring thu thập, lưu trữ và trực quan hóa telemetry; đồng thời đánh giá rule để phát hiện bất thường vận hành. Một rule cần metric, cửa sổ thời gian, ngưỡng, mức nghiêm trọng và điều kiện đóng. Kết quả phát hiện được chuyển thành cảnh báo gửi tới người vận hành thay vì chỉ tồn tại dưới dạng biểu đồ.

## **2.10 RAG và AI Agent**

### **2.10.1 Retrieval-Augmented Generation**

Retrieval-Augmented Generation (RAG) kết hợp truy xuất tài liệu với mô hình ngôn ngữ. Câu hỏi được dùng để tìm các đoạn liên quan; nội dung truy xuất được đưa vào ngữ cảnh để tạo câu trả lời có căn cứ [14]. RAG phù hợp với tri thức tương đối ổn định như runbook, hướng dẫn triển khai và mô tả lỗi.

### **2.10.2 Embedding Search**

Embedding search ánh xạ truy vấn và tài liệu thành vector rồi xếp hạng theo độ tương đồng. Phương pháp này tìm được nội dung gần nghĩa dù từ khóa không trùng khớp. Chất lượng phụ thuộc cách chia đoạn, mô hình embedding, metadata filter và ngưỡng lấy kết quả.

### **2.10.3 Hybrid Search**

Hybrid search kết hợp tìm kiếm từ khóa với vector search. Từ khóa phù hợp với mã lỗi, model version và tên node; vector search phù hợp với câu hỏi diễn đạt tự nhiên. Hai danh sách có thể được chuẩn hóa điểm, hợp nhất và rerank trước khi đưa cho Agent.

### **2.10.4 AI Agent**

AI Agent mở rộng mô hình ngôn ngữ bằng khả năng lập kế hoạch và gọi công cụ. Theo mẫu ReAct [22], agent xen kẽ suy luận và hành động: gọi công cụ, đọc kết quả, rồi quyết định bước tiếp theo. Nhờ đó agent truy vấn được trạng thái node, runtime, deployment hay telemetry mới nhất thay vì chỉ trả lời từ tài liệu. Mô hình ngôn ngữ không được truy cập trực tiếp cơ sở dữ liệu hay shell; mọi hành động đi qua tool schema và cơ chế kiểm soát quyền.

### **2.10.5 Tool Calling**

Tool calling là cơ chế mô hình chọn hàm và tạo tham số có cấu trúc. Hệ thống phải kiểm tra schema, quyền người dùng, phạm vi node và policy trước khi thực thi. Các tool thay đổi trạng thái cần thêm bước xác nhận của con người trước khi thực thi, và có thể bị ẩn hoàn toàn khỏi mô hình khi hệ thống chỉ cho phép tra cứu.

### **2.10.6 MCP**

Model Context Protocol (MCP) chuẩn hóa cách AI application khám phá và gọi tools/resources do server cung cấp [20]. Trong phạm vi đề án, MCP được dùng làm biên tích hợp duy nhất giữa AI Assistant và các công cụ vận hành của Cloud. MCP không thay thế cơ chế xác thực hoặc phân quyền của domain; server vẫn phải xác minh danh tính và chính sách cho từng lần gọi.

## **2.11 Domain-Driven Design**

Domain-Driven Design (DDD) tổ chức phần mềm quanh ngôn ngữ và quy tắc nghiệp vụ thay vì quanh bảng dữ liệu hoặc framework [15]. Cách tiếp cận này phù hợp với nền tảng có nhiều miền như quản lý thiết bị, triển khai, điều khiển runtime, giám sát và trợ lý AI.

### **2.11.1 Entity**

Entity có định danh ổn định và vòng đời thay đổi theo thời gian. Ví dụ gồm Cluster, Node, Runtime và Deployment. Hai entity có cùng thuộc tính nhưng khác định danh vẫn là hai đối tượng khác nhau.

### **2.11.2 Value Object**

Value Object được xác định bởi giá trị và thường bất biến, chẳng hạn tham chiếu model (kho, tên tệp, phiên bản), vị trí model đã ghim commit hoặc bản ghi heartbeat của node. Value Object giúp đóng gói validation và tránh truyền các chuỗi rời rạc không có ngữ nghĩa.

### 2.11.3 *Aggregate*

Aggregate là biên nhất quán cho một nhóm entity và value object. Aggregate root kiểm soát mọi thay đổi bên trong. Deployment có thể là aggregate quản lý target, batch, trạng thái và policy; các thay đổi phải đi qua hành vi của Deployment thay vì cập nhật bảng trực tiếp.

### 2.11.4 *Repository*

Repository cung cấp giao diện tải và lưu aggregate theo ngôn ngữ domain. Domain không phụ thuộc chi tiết PostgreSQL, object storage hoặc message broker. Điều này giúp kiểm thử quy tắc nghiệp vụ độc lập với hạ tầng.

### 2.11.5 *Domain Service*

Domain Service chứa hành vi nghiệp vụ không thuộc tự nhiên về một entity, ví dụ chọn nhóm canary dựa trên capability hoặc đánh giá chính sách triển khai. Service không nên trở thành nơi chứa toàn bộ logic; invariant vẫn được giữ trong aggregate khi có thể.

### 2.11.6 *Bounded Context*

Bounded Context xác định biên mô hình và ngôn ngữ. Trong đồ án có ba context: Cloud quản lý cluster, node, runtime và deployment; Edge quản lý desired state và vòng đời container trên một node; Assistant điều phối hội thoại và tool call. Các context chỉ trao đổi qua API, thông điệp MQTT hoặc công cụ MCP với hợp đồng rõ ràng.

### 2.11.7 *Kiến trúc port–adapter*

Kiến trúc port–adapter (hexagonal) hỗ trợ DDD: tầng application định nghĩa *port* là interface tới thế giới bên ngoài (container engine, camera, detector, kho lưu trữ), tầng infrastructure cung cấp *adapter* hiện thực port. Nhờ đó cùng một service nghiệp vụ chạy được với Docker thật hoặc bản giả lập, với camera thật hoặc nguồn khung tổng hợp, giúp kiểm thử luồng phức tạp mà không cần phần cứng.

## 2.12 Kết chương

Chương 2 đã trình bày chuỗi kiến thức từ model đến platform. YOLO, pruning, QAT và TensorRT cung cấp nền tảng cho nghiên cứu tối ưu; quản lý model, triển khai, Edge Runtime và nền tảng phân tán giải thích cách đưa model vào vận hành; AI Agent, tool calling, MCP và DDD cung cấp cơ sở cho AI Assistant và cách chia domain. Chương 3 sử dụng các cơ sở này để xây dựng và đánh giá pipeline tối ưu YOLO.

## CHƯƠNG 3. NGHIÊN CỨU VÀ TỐI ƯU MÔ HÌNH YOLO

Chương này trình bày phần nghiên cứu mô hình trước khi đưa model vào nền tảng. Đồ án sử dụng YOLO26n — biến thể nhỏ nhất của thể hệ YOLO26 — làm điểm xuất phát và xây dựng một pipeline tối ưu nối tiếp gồm sáu giai đoạn: tìm siêu tham số huấn luyện baseline bằng thuật toán Sea-Horse Optimizer (SHO), huấn luyện thừa hóa BatchNorm, cắt tỉa kênh có cấu trúc, phục hồi độ chính xác bằng chưng cất tri thức hoặc fine-tune, huấn luyện nhận biết lượng tử hóa (QAT) và chuyển đổi sang TensorRT INT8. Mỗi giai đoạn nhận checkpoint của giai đoạn trước và sinh checkpoint cho giai đoạn sau.

### 3.1 Bài toán tối ưu

#### 3.1.1 Bài toán ứng dụng

Bài toán ứng dụng là giám sát tuân thủ trang bị bảo hộ lao động (PPE – Personal Protective Equipment) trên công trường xây dựng: phát hiện người lao động và các trang bị mũ bảo hộ, áo phản quang, găng tay, ủng, kính bảo hộ, đồng thời phát hiện trường hợp *thiếu* trang bị. Camera công trường thường ở xa hạ tầng mạng tốt, nên suy luận tại biên trên thiết bị Jetson là lựa chọn phù hợp.

#### 3.1.2 Mô hình baseline

Mô hình baseline là YOLO26n do Ultralytics phát hành [19], khởi tạo từ trọng số tiền huấn luyện trên COCO rồi huấn luyện trên tập PPE ở độ phân giải  $640 \times 640$ , suy luận FP32. YOLO26n có 2,57 triệu tham số và 6,12 GFLOPs tại  $640 \times 640$  (đo trên checkpoint gốc). Baseline cung cấp mốc chất lượng, kích thước và hiệu năng để đánh giá mọi biến thể sau đó.

#### 3.1.3 Hạn chế của mô hình baseline

Dù đã là biến thể nhỏ, baseline chưa được tối ưu riêng cho Jetson: mọi kênh tích chập đều được giữ lại kể cả kênh đóng góp rất ít, và phép tính chạy ở FP32 trong khi GPU Jetson có đơn vị tính FP16/INT8 nhanh hơn nhiều. Trên Jetson Nano (GPU Maxwell 128 nhân, không có Tensor Core), chi phí FP32 là nút thắt chính; trên Jetson Orin Nano,

Tensor Core thể hệ Ampere chỉ phát huy khi mô hình chạy INT8 hoặc FP16. Mức hạn chế cụ thể được xác nhận bằng số đo trên thiết bị thay vì chỉ suy luận từ FLOPs.

### 3.1.4 Mục tiêu tối ưu

Đề án tối ưu đồng thời sáu tiêu chí: độ chính xác, latency, FPS, kích thước mô hình, bộ nhớ GPU và công suất. Bài toán được biểu diễn dưới dạng:

$$\min_{m \in \mathcal{M}} (-A(m), L(m), -F(m), S(m), G(m), P(m)), \quad (3.1)$$

trong đó  $m$  là một cấu hình model,  $A$  là  $mAP@50:95$ ,  $L$  là latency,  $F$  là FPS,  $S$  là kích thước,  $G$  là bộ nhớ GPU và  $P$  là công suất. Thay vì gộp tùy ý thành một điểm duy nhất, đề án loại trước các cấu hình vi phạm ràng buộc rồi so sánh các cấu hình còn lại theo trade-off.

Một cấu hình chỉ được xem là ứng viên triển khai khi thỏa đồng thời ba ràng buộc:

1.  $mAP@50:95$  trên tập test giảm không quá **2 điểm phần trăm** so với baseline FP32;
2. latency suy luận p95 không vượt **66 ms**, tương ứng FPS mục tiêu 15 — giá trị mặc định `target_fps` của AI Runtime;
3. thiết bị chạy ổn định trong bài đo kéo dài 30 phút, không giảm xung do nhiệt làm FPS rơi dưới ngưỡng.

## 3.2 Xây dựng mô hình YOLO baseline

### 3.2.1 Dataset

Tập dữ liệu sử dụng là **Construction-PPE** do Ultralytics công bố (khoảng 178 MB), gồm ảnh công trường với nhãn hộp bao cho 11 lớp. Tập được chia sẵn thành train/validation/test như Bảng 3.1, tỷ lệ xấp xỉ 80/10/10.

Bảng 3.1: Tập dữ liệu Construction-PPE

| Thuộc tính         | Giá trị                                                |
|--------------------|--------------------------------------------------------|
| Số ảnh             | 1.416 (train 1.132, validation 143, test 141)          |
| Số lớp             | 11                                                     |
| Lớp trang bị       | helmet, gloves, vest, boots, goggles                   |
| Lớp thiếu trang bị | no_helmet, no_goggle, no_gloves, no_boots              |
| Lớp khác           | Person, none                                           |
| Định dạng nhãn     | YOLO (lớp, tâm $x$ , tâm $y$ , rộng, cao đã chuẩn hóa) |

Ngoài tập công khai, đồ án thu thập thêm dữ liệu thực tế gồm các đoạn video và ảnh công trường (thư mục `dataset_raw`) và một tập PPE tùy chỉnh 9 lớp xuất từ Roboflow (tập validation 229 ảnh) để đánh giá khả năng tổng quát trên dữ liệu ngoài phân phối huấn luyện. Nhãn được kiểm tra về chỉ số lớp, hộp vượt biên và ảnh lỗi trước khi huấn luyện; các khung trích từ cùng một video được giữ trong cùng một tập để tránh rò rỉ ngữ cảnh giữa train và test.

### 3.2.2 Tiền xử lý và tăng cường dữ liệu

Ảnh được đưa về  $640 \times 640$  bằng letterbox (giữ tỷ lệ, đệm viền), chuyển BGR  $\rightarrow$  RGB và chuẩn hóa về  $[0, 1]$ . Các phép tăng cường dùng bộ tăng cường của Ultralytics: biến đổi HSV, xoay, tịnh tiến, co giãn, cắt xiên, phối cảnh, lật dọc/ngang, mosaic, mixup và copy-paste. Thay vì dùng hệ số mặc định, cường độ của các phép này được tìm tự động bằng SHO cùng các siêu tham số tối ưu hóa (mục tiếp theo). Mosaic được tắt ở 10 epoch cuối (`close_mosaic=10`) để mô hình thích nghi với phân bố ảnh thật. Cấu hình tiền xử lý suy luận (letterbox, thứ tự kênh, chuẩn hóa) được giữ nguyên khi export và trong AI Runtime.

### 3.2.3 Tìm siêu tham số bằng Sea-Horse Optimizer

Sea-Horse Optimizer [18] là thuật toán tối ưu bầy đàn mô phỏng hành vi di chuyển, săn mồi và sinh sản của cá ngựa. Mỗi cá thể là một vector siêu tham số; ở mỗi thế hệ, cá thể di chuyển theo quỹ đạo xoắn ốc hoặc bước Lévy quanh cá thể tốt nhất (khám phá), săn mồi với xác suất thành công ngẫu nhiên (khai thác), rồi nửa quần thể tốt nhất sinh thế hệ con. Đồ án hiện thực SHO độc lập với Ultralytics (`optimization/sho.py`) để kiểm thử quy tắc cập nhật bằng hàm mục tiêu toán học trước khi dùng cho YOLO.

Không gian tìm kiếm gồm 18 chiều (Bảng 3.2). Hàm mục tiêu cần cực tiểu là

$f(\mathbf{x}) = 1 - mAP@50:95(\mathbf{x})$ , trong đó  $mAP@50:95(\mathbf{x})$  đo trên tập validation sau một lần huấn luyện *proxy* ngắn 3 epoch từ cùng checkpoint khởi tạo. Ngân sách tìm kiếm: quần thể 6 cá thể, 3 thế hệ. Bộ siêu tham số tốt nhất được ghi ra `optimized_baseline.yaml` và dùng để huấn luyện baseline đầy đủ.

Bảng 3.2: Không gian tìm kiếm siêu tham số của SHO

| Tham số                   | Khoảng               | Ý nghĩa                          |
|---------------------------|----------------------|----------------------------------|
| lr0                       | $[10^{-5}, 10^{-1}]$ | Learning rate ban đầu            |
| lrf                       | $[0,01; 1]$          | Tỷ lệ learning rate cuối/ban đầu |
| momentum                  | $[0,6; 0,98]$        | Momentum/ $\beta_1$              |
| weight_decay              | $[0; 0,001]$         | Hệ số suy giảm trọng số          |
| warmup_epochs             | $[0; 5]$             | Số epoch khởi động               |
| warmup_momentum           | $[0; 0,95]$          | Momentum giai đoạn khởi động     |
| hsv_h, hsv_s, hsv_v       | $[0; 0,9]$           | Biến đổi màu, bão hòa, độ sáng   |
| degrees                   | $[0; 45]$            | Góc xoay (độ)                    |
| translate, scale          | $[0; 0,9]$           | Tịnh tiến, co giãn               |
| shear                     | $[0; 10]$            | Cắt xiên (độ)                    |
| perspective               | $[0; 0,001]$         | Phối cảnh                        |
| flipud                    | $[0; 1]$             | Xác suất lật dọc                 |
| mosaic, mixup, copy_paste | $[0; 1]$             | Xác suất các phép ghép ảnh       |

### 3.2.4 Huấn luyện

Baseline được huấn luyện 100 epoch, batch 16, ảnh  $640 \times 640$ , trên GPU Google Colab; toàn bộ đầu ra được ghi trực tiếp lên Google Drive để không mất khi phiên Colab bị ngắt. Optimizer và learning rate lấy từ kết quả SHO. Seed được cố định và `deterministic=True`. Checkpoint tốt nhất được chọn theo  $mAP@50:95$  trên tập validation, không chọn theo tập test.

### 3.2.5 Đánh giá baseline

Bảng 3.3: Kết quả chất lượng của mô hình baseline trên tập test Construction-PPE

| Model                 | Params | GFLOPs | Precision       | Recall          | mAP@50          | mAP@50:95       |
|-----------------------|--------|--------|-----------------|-----------------|-----------------|-----------------|
| YOLO26n baseline FP32 | 2,57 M | 6,12   | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |



Ngoài chỉ số tổng hợp, AP theo lớp và confusion matrix được dùng để nhận diện lớp yếu. Các lớp “thiếu trang bị” (no\_gloves, no\_goggles) có đối tượng nhỏ và ít mẫu, dự kiến là nhóm nhạy cảm nhất với cắt tỉa và lượng tử hóa, nên được theo dõi riêng ở các giai đoạn sau.

### 3.3 Tối ưu bằng Pruning

#### 3.3.1 Lựa chọn phương pháp pruning

Đồ án chọn **cắt tỉa kênh có cấu trúc** theo phương pháp Network Slimming [16], vì kết quả là một mạng nhỏ hơn thật sự (ít kênh hơn) mà TensorRT khai thác trực tiếp, không cần phần cứng hỗ trợ ma trận thưa. Ý tưởng là dùng hệ số co giãn  $\gamma$  của lớp BatchNorm sau mỗi tích chập làm thước đo tầm quan trọng của kênh: kênh có  $|\gamma|$  nhỏ đóng góp ít vào đầu ra và có thể loại bỏ. Unstructured pruning chỉ làm thưa trọng số mà không giảm kích thước tensor nên không đem lại tăng tốc trên Jetson, và không được chọn.

#### 3.3.2 Huấn luyện thưa hóa

Để phân bố  $\gamma$  tách rõ giữa kênh quan trọng và kênh thừa, mô hình được huấn luyện với phạt L1 trên  $\gamma$ :

$$\mathcal{L} = \mathcal{L}_{det} + s \sum_{\gamma \in \Gamma} |\gamma|, \quad (3.2)$$

hiện thực bằng cách cộng  $s \cdot \text{sign}(\gamma)$  vào gradient của  $\gamma$  sau mỗi lần lan truyền ngược. Hệ số  $s$  giảm dần theo epoch,  $s_e = s(1 - 0,9e/E)$ , để phạt mạnh ở đầu và để mô hình phục hồi độ chính xác ở cuối. Cấu hình:  $s = 10^{-2}$ , 200 epoch, ảnh 640. Một callback in thống kê  $\gamma$  mỗi epoch để theo dõi mức thưa. Thử nghiệm với  $s$  hằng số cho thấy độ chính xác validation sụp gần về 0, nên lịch giảm dần là cần thiết.

#### 3.3.3 Cắt tỉa theo ngưỡng toàn cục

Script `prune.py` thực hiện:

1. Thu thập mọi lớp BN, trừ các lớp trong danh sách bảo vệ (nhánh có ràng buộc cộng dư, lớp đầu vào của head và các tầng nhạy cảm);
2. Sắp xếp toàn bộ  $|\gamma|$  và lấy ngưỡng  $\tau$  tại vị trí `prune_ratio`. Ngưỡng bị chặn trên bởi giá trị nhỏ nhất trong các  $\max |\gamma|$  của từng lớp để không lớp nào bị cắt hết kênh; nếu tỷ lệ yêu cầu vượt giới hạn này, tỷ lệ tự động được hạ xuống;

3. Tạo mặt nạ kênh  $|\gamma| \geq \tau$  cho từng lớp và dựng kiến trúc mới với các module chuyên dụng (C3k2Pruned, C2PSAPruned, SPPFPruned, DetectPruned) theo cấu hình `yolo26-pruned.yaml`;
4. Sao chép trọng số của các kênh được giữ sang mô hình mới, lưu mô hình (lớp `DetectionModelPruned`) cùng từ điển mặt nạ kênh.

Tỷ lệ cắt tỉa chính được chọn là **30%**. Với YOLO26n, cắt 30% kênh BN cho kết quả ở Bảng 3.4: số tham số giảm 15,7% và GFLOPs giảm 21,2%. Mức giảm FLOPs lớn hơn mức giảm tham số vì các kênh bị cắt tập trung ở tầng có độ phân giải không gian lớn.

Bảng 3.4: Độ phức tạp của mô hình trước và sau cắt tỉa 30% (đo tại  $640 \times 640$ )

| Mô hình            | Tham số   | GFLOPs | Thay đổi                     |
|--------------------|-----------|--------|------------------------------|
| YOLO26n baseline   | 2.572.280 | 6,12   | —                            |
| YOLO26n pruned 30% | 2.169.072 | 4,82   | −15,7% tham số, −21,2% FLOPs |

### 3.3.4 Phục hồi độ chính xác

Ngay sau khi cắt, độ chính xác giảm mạnh và cần được phục hồi. Đồ án so sánh hai phương án thay thế nhau (không chạy nối tiếp):

**Fine-tune thường:** huấn luyện tiếp mô hình đã cắt 100 epoch, AdamW,  $lr_0 = 10^{-3}$ . Cấu trúc đã cắt được khóa bởi kiến trúc mới nên kênh bị loại không xuất hiện trở lại.

**Chung cất tri thức:** mô hình đã cắt là *student*, YOLO26l (hoặc YOLO26x) là *teacher*. Hàm mất mát

$$\mathcal{L} = \mathcal{L}_{det} + \lambda_{cwd} \mathcal{L}_{CWD} + \alpha (\mathcal{L}_{cls-KD} + \mathcal{L}_{reg-KD}), \quad (3.3)$$

trong đó  $\mathcal{L}_{CWD}$  là Channel-Wise Distillation [17]: với mỗi kênh của bản đồ đặc trưng, phân bố không gian được chuẩn hóa bằng softmax với nhiệt độ  $T$  và student được ép khớp phân bố của teacher bằng KL divergence; một lớp tích chập  $1 \times 1$  (adapter) đưa số kênh của student về bằng teacher.  $\mathcal{L}_{cls-KD}$  chung cất điểm phân lớp ở head. Cấu hình mặc định:  $\lambda_{cwd} = 50$ ,  $T = 4$ ; trọng số  $\alpha$  tăng dần theo epoch. Vì hàm mất mát đã bao gồm  $\mathcal{L}_{det}$  trên nhãn thật, chung cất thay thế hoàn toàn bước fine-tune.

Kết quả thực nghiệm của các phương án được trình bày ở Mục 6.2.1.

## 3.4 Tối ưu bằng QAT

### 3.4.1 Thiết lập quantization

QAT được hiện thực bằng thư viện `pytorch-quantization` của NVIDIA, là đường chính thức để tạo mô hình INT8 có thông tin lượng tử tường minh cho TensorRT [4]. Hàm `quant_modules.initialize()` thay mọi `nn.Conv2d` bằng `QuantConv2d` trước khi dựng mô hình. Theo mặc định của thư viện, lượng tử hóa là **đối xứng**: trọng số lượng tử **theo từng kênh đầu ra** (per-channel), activation lượng tử **theo tensor** (per-tensor). Với các phép cộng dư và ghép kênh trong các khối C3k2, C2PSA đã cắt tĩa, đồ án viết các lớp lượng tử riêng (`quant_ops_pruned_yolo26.py`) chèn `TensorQuantizer` cho từng đầu vào, để TensorRT có thể hợp nhất phép toán ở INT8 thay vì chuyển đổi qua lại giữa INT8 và FP16. Các lượng tử của những nhánh không cần thiết được vô hiệu hóa để giữ độ chính xác.

### 3.4.2 Hiệu chỉnh

Trước khi huấn luyện, thang lượng tử (amax) của activation được hiệu chỉnh trên 256 batch dữ liệu huấn luyện bằng phương pháp **entropy**: với mỗi tensor, thu histogram giá trị và chọn ngưỡng cắt cực tiểu hóa KL divergence giữa phân bố gốc và phân bố sau lượng tử. Phương pháp này bỏ qua các giá trị ngoại lai hiếm gặp, cho độ phân giải lượng tử tốt hơn so với lấy giá trị lớn nhất.

### 3.4.3 Fake quantization và huấn luyện

Sau hiệu chỉnh, các nút fake-quantization mô phỏng phép làm tròn và cắt về INT8 trong lan truyền thuận, còn gradient đi qua bằng Straight-Through Estimator. Mô hình được fine-tune nhẹ 20 epoch, batch 8, optimizer SGD momentum 0,9; learning rate đầu vào  $10^{-3}$  được chia 100 bên trong trainer (thành  $10^{-5}$ ) vì QAT chỉ cần điều chỉnh nhỏ quanh điểm đã hội tụ. Tùy chọn hiệu chỉnh lại định kỳ (`recalib_every`) mặc định tắt. QAT chỉ được chạy trên checkpoint *đã phục hồi* sau cắt tĩa, không chạy trên checkpoint vừa cắt còn yếu.

### 3.4.4 Export INT8

Checkpoint QAT được export sang ONNX, trong đó mỗi tensor lượng tử được biểu diễn bằng một cặp nút Q/DQ (`QuantizeLinear` và `DequantizeLinear`) mang thang

lượng tử đã học. TensorRT đọc các nút Q/DQ này để build engine INT8 trực tiếp, không cần bước hiệu chỉnh PTQ riêng. Thư viện pytorch-quantization yêu cầu CUDA nên giai đoạn này chạy trên môi trường Linux có GPU (Google Colab), không chạy được trên máy Windows chỉ có CPU.

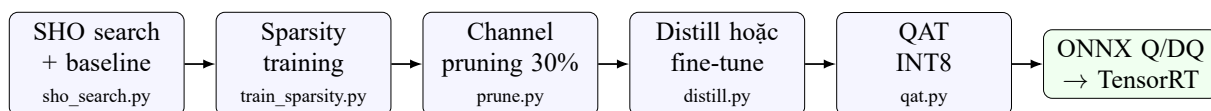
### 3.4.5 Đánh giá mô hình QAT

Mô hình QAT được đánh giá ở ba điểm: checkpoint fake-quant trong PyTorch, ONNX sau export và TensorRT engine trên thiết bị. Chênh lệch giữa các điểm cho biết sai số đến từ lượng tử, từ export hay từ backend.

## 3.5 Pipeline tối ưu kết hợp

### 3.5.1 Quy trình

Quy trình kết hợp được thể hiện trong Hình 3.1 và được đóng gói thành notebook yolo26\_ppe\_train\_pipeline.ipynb chạy trên Google Colab: mỗi ô thực hiện một giai đoạn bằng một script độc lập trong scripts/, checkpoint của giai đoạn trước là đầu vào của giai đoạn sau.



Hình 3.1: Pipeline tối ưu YOLO26n từ baseline đến TensorRT INT8

Thứ tự các giai đoạn có chủ đích: cắt tỉa thay đổi cấu trúc nên phải làm trước; phục hồi độ chính xác trước khi lượng tử để QAT bắt đầu từ một mô hình tốt; QAT là bước cuối vì mọi thay đổi trọng số sau QAT sẽ làm thang lượng tử mất hiệu lực. Bảng 3.5 tóm tắt cấu hình từng giai đoạn.

Bảng 3.5: Cấu hình các giai đoạn trong pipeline tối ưu

| Giai đoạn      | Cấu hình                                                                                         |
|----------------|--------------------------------------------------------------------------------------------------|
| SHO + baseline | Quản thẻ 6, 3 thể hệ, proxy 3 epoch; huấn luyện đầy đủ 100 epoch, batch 16, ảnh 640              |
| Sparsity       | 200 epoch, $s = 10^{-2}$ giảm dần tuyến tính về 0,1s                                             |
| Pruning        | Tỷ lệ 0,30, ngưỡng toàn cục trên $ \gamma $ , mô hình n                                          |
| Phục hồi       | Chung cất: teacher YOLO26l, $\lambda_{cud} = 50$ , $T = 4$ , 100 epoch; hoặc fine-tune 100 epoch |
| QAT            | 20 epoch, batch 8, SGD, $lr_0 = 10^{-3}/100$ , hiệu chỉnh entropy 256 batch                      |
| Export         | ONNX Q/DQ, ảnh $640 \times 640$ , batch 1; TensorRT FP16 và INT8                                 |

### 3.5.2 Ma trận cấu hình đánh giá

Bảng 3.6: Ma trận cấu hình tối ưu mô hình

| Cấu hình                  | Pruning | Phục hồi / QAT  | Precision đích |
|---------------------------|---------|-----------------|----------------|
| Baseline                  | 0%      | —               | FP32, FP16     |
| Pruned-30 + fine-tune     | 30%     | Fine-tune       | FP16           |
| Pruned-30 + distill       | 30%     | Chung cất       | FP16           |
| Baseline + QAT            | 0%      | QAT             | INT8           |
| Pruned-30 + distill + QAT | 30%     | Chung cất + QAT | INT8           |

Mỗi cấu hình dùng cùng dataset split, cùng kích thước ảnh và cùng quy trình đánh giá. Tên run, seed, checkpoint nguồn và cấu hình được lưu (tệp `args.yaml`, `results.csv` của mỗi run) để truy ngược artifact về thí nghiệm tạo ra nó.

## 3.6 Chuyển đổi sang TensorRT

### 3.6.1 Export ONNX

Mỗi checkpoint ứng viên được export sang ONNX với đầu vào tĩnh  $1 \times 3 \times 640 \times 640$  (batch 1, phù hợp xử lý luồng camera từng khung) bằng chức năng export của Ultralytics. ONNX checker xác nhận đồ thị hợp lệ; suy luận đối chiếu giữa PyTorch và ONNX Runtime trên một tập ảnh cố định đo sai khác tensor đầu ra và sai khác detection cuối. YOLO26 có head end-to-end không cần NMS nên đồ thị ONNX không chứa bước NMS phụ thuộc dữ liệu, giúp chuyển đổi TensorRT thuận lợi.

### 3.6.2 *TensorRT FP16 và INT8*

Engine được build *trên chính thiết bị đích* vì engine TensorRT gắn với kiến trúc GPU và phiên bản TensorRT: engine tạo trên Jetson Nano (TensorRT 8.2) không chạy trên Jetson Orin Nano và ngược lại. Engine FP16 được tạo từ baseline và các mô hình đã cắt tỉa. Engine INT8 được tạo từ ONNX Q/DQ của mô hình QAT; vì thang lượng tử đã có trong đồ thị nên không cần tập hiệu chỉnh bổ sung. Log build ghi lại phiên bản TensorRT, workspace và precision thực tế của từng lớp để xác định tỷ lệ phép toán thực sự chạy INT8.

### 3.6.3 *Kiểm tra độ chính xác sau chuyển đổi*

Kiểm tra gồm ba mức: sai khác tensor trên mẫu cố định, sai khác detection theo IoU và confidence, và chênh lệch  $mAP$  trên toàn tập test. Artifact vi phạm ràng buộc độ chính xác ở Mục 3.1 bị loại dù latency tốt. Engine đạt yêu cầu được đẩy lên kho model và triển khai qua nền tảng; AI Runtime nạp trực tiếp tệp .engine bằng Ultralytics.

## 3.7 Lựa chọn mô hình triển khai

Model triển khai được chọn theo hai bước: loại các cấu hình vi phạm ràng buộc độ chính xác, latency hoặc ổn định nhiệt; trong nhóm còn lại, ưu tiên cấu hình có latency thấp nhất trên thiết bị yếu hơn (Jetson Nano) vì đó là giới hạn của cả đội thiết bị.

Dựa trên phân tích độ phức tạp và kết quả thực nghiệm sơ bộ (Mục 6.2.1), cấu hình được đề xuất là **YOLO26n cắt tỉa 30% + phục hồi + QAT, chạy TensorRT INT8** (Bảng 3.7). Lý do: cắt tỉa giảm 21% FLOPs, INT8 giảm tiếp chi phí mỗi phép toán và bộ nhớ trọng số khoảng 4 lần so với FP32, còn QAT giữ độ chính xác tốt hơn PTQ. Với thiết bị hỗ trợ kém INT8, engine FP16 của mô hình đã cắt tỉa là phương án dự phòng. Kết luận cuối cùng phụ thuộc số đo trên thiết bị ở Chương 6.

Bảng 3.7: Cấu hình model đề xuất triển khai

| Thuộc tính              | Giá trị                                                                                                 |
|-------------------------|---------------------------------------------------------------------------------------------------------|
| Kiến trúc               | YOLO26n, đầu vào $640 \times 640$ , 11 lớp PPE                                                          |
| Phương pháp tối ưu      | SHO baseline $\rightarrow$ sparsity $\rightarrow$ pruning 30% $\rightarrow$ chưng cất $\rightarrow$ QAT |
| Độ phức tạp             | 2,17 M tham số, 4,82 GFLOPs                                                                             |
| Precision               | INT8 (TensorRT), dự phòng FP16                                                                          |
| Định dạng triển khai    | TensorRT .engine build riêng cho từng loại Jetson                                                       |
| Phân phối               | Kho model Hugging Face Hub, ghim theo commit SHA                                                        |
| Độ chính xác            | [CẦN BỔ SUNG: mAP@50, mAP@50:95 trên tập test]                                                          |
| Hiệu năng trên thiết bị | [CẦN BỔ SUNG: FPS, latency trên Jetson Nano và Orin Nano]                                               |

### 3.8 Kết chương

Chương 3 đã xây dựng pipeline tối ưu YOLO26n sáu giai đoạn: tìm siêu tham số bằng SHO, thừa hóa BatchNorm, cắt tỉa kênh 30% theo ngưỡng toàn cục, phục hồi bằng chưng cất CWD hoặc fine-tune, QAT với hiệu chỉnh entropy và chuyển đổi TensorRT INT8. Cắt tỉa 30% giảm 15,7% tham số và 21,2% FLOPs. Mỗi giai đoạn là một script độc lập có thể chạy lại, đầu ra được lưu cùng cấu hình để truy vết. Model sau tối ưu là một tệp engine được phân phối qua kho model và triển khai bởi nền tảng được thiết kế ở Chương 4.

## CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ NỀN TẢNG EDGE AI

Sau khi xác định pipeline tối ưu mô hình, chương này phân tích yêu cầu và thiết kế nền tảng đưa model vào vận hành trên nhiều thiết bị biên. Nền tảng được tổ chức theo mô hình Cloud Control Plane – Edge Data Plane: Cloud giữ trạng thái mong muốn (desired state) của toàn đội thiết bị, còn Edge Agent trên mỗi node hội tụ trạng thái thực tế (actual state) về trạng thái mong muốn và báo cáo ngược lại. Mã nguồn được chia thành các lớp domain, application và infrastructure theo tinh thần Domain-Driven Design; hợp đồng dữ liệu giữa Cloud và Edge được đặt trong một gói dùng chung để hai phía không bao giờ lệch schema.

### 4.1 Yêu cầu hệ thống

#### 4.1.1 Tác nhân và phạm vi

Hệ thống có bốn nhóm tác nhân. **Người vận hành** thao tác qua Dashboard hoặc trò chuyện với AI Assistant. **Cloud Control Plane** lưu trạng thái nền tảng và phát lệnh. **Edge Agent** chạy thường trực trên mỗi Edge Node, nhận lệnh và điều khiển container. **AI Runtime** là container thực thi mô hình YOLO trên luồng camera và báo cáo chỉ số cho Edge Agent. Tài nguyên được tổ chức phân cấp: một Cloud quản lý nhiều Edge Cluster; mỗi Cluster gồm nhiều Edge Node; mỗi Node có đúng một Edge Agent; mỗi Agent quản lý nhiều AI Runtime.

#### 4.1.2 Yêu cầu chức năng

Bảng 4.1: Yêu cầu chức năng của nền tảng

| Mã   | Nhóm            | Mô tả                                                                                                                                    |
|------|-----------------|------------------------------------------------------------------------------------------------------------------------------------------|
| FR01 | Quản lý Cluster | Tạo, liệt kê, xem chi tiết Edge Cluster; tên cluster là duy nhất.                                                                        |
| FR02 | Quản lý Node    | Đăng ký node vào cluster (tên duy nhất trong cluster), xem thông tin phần cứng, phiên bản Agent, trạng thái và thời điểm heartbeat cuối. |



| Mã   | Nhóm               | Mô tả                                                                                                                                                                                                |
|------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR03 | Theo dõi liveness  | Tự động chuyển node sang OFFLINE khi quá thời gian heartbeat (mặc định 30 s).                                                                                                                        |
| FR04 | Deployment         | Triển khai một AI Runtime (image, model, cấu hình, giới hạn tài nguyên) lên một node; cập nhật từng trường; gỡ bỏ; theo dõi trạng thái PENDING → IN_PROGRESS → SUCCEEDED/FAILED, REMOVING → REMOVED. |
| FR05 | Điều khiển Runtime | Start, stop, restart một runtime từ Cloud hoặc từ API cục bộ của Agent.                                                                                                                              |
| FR06 | Phân phối model    | Tham chiếu model theo kho model (Hugging Face Hub) được ghim theo commit; Agent tải, lưu cache và gắn file model vào container.                                                                      |
| FR07 | Telemetry          | Thu thập chỉ số hệ thống, GPU và runtime (FPS, latency p50/p95/p99, hàng đợi, camera, confidence) theo chu kỳ; lưu mẫu mới nhất và lịch sử ngắn hạn.                                                 |
| FR08 | Event              | Ghi nhận sự kiện runtime, lỗi và kết quả deployment theo từng node.                                                                                                                                  |
| FR09 | Realtime           | Đẩy trạng thái, telemetry và event tới Dashboard qua WebSocket theo bộ lọc topic.                                                                                                                    |
| FR10 | AI Assistant       | Hỏi đáp bằng ngôn ngữ tự nhiên về cluster, node, runtime, deployment, telemetry, event; sinh biểu đồ và gợi ý câu hỏi tiếp theo; thao tác thay đổi chỉ thực hiện sau xác nhận.                       |
| FR11 | Giám sát hạ tầng   | Thu metric node, container, GPU tại Edge và đẩy về Prometheus trên Cloud; cảnh báo theo luật.                                                                                                        |

#### 4.1.3 Yêu cầu phi chức năng

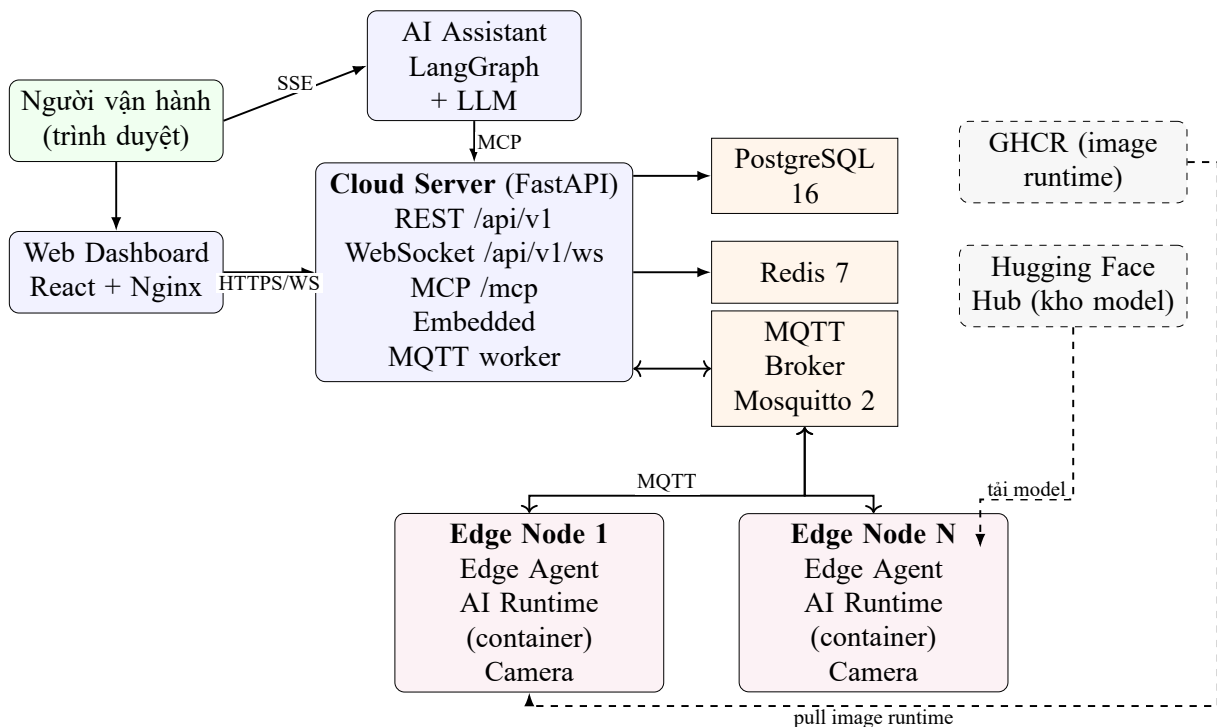
- **Tính tự trị tại biên:** AI Runtime tiếp tục suy luận khi mất kết nối Cloud; Agent lưu desired state xuống đĩa và khôi phục sau khi khởi động lại.
- **Idempotency:** gửi lại cùng một deployment không tạo runtime mới; Agent bỏ qua lệnh trùng command\_id; kết quả của lệnh đã bị thay thế không được ghi đè trạng thái mới.
- **Tự phục hồi:** vòng reconcile định kỳ tạo lại container bị dừng bất thường, có giới hạn số lần thất bại liên tiếp để tránh vòng lặp khởi động.
- **Bảo mật:** REST, WebSocket và MCP yêu cầu API key; Agent cũng yêu cầu API key

cho API cục bộ; bảng khóa chỉ lưu hàm băm; model gắn vào container ở chế độ chỉ đọc [13].

- **Khả năng quan sát:** mọi dịch vụ ghi log có cấu trúc (tùy chọn JSON), cung cấp /health kiểm tra database, Redis và broker.
- **Khả năng mở rộng:** Cloud API không giữ trạng thái phiên; phần tiêu thụ MQTT có thể chạy trong cùng tiến trình hoặc tách thành worker riêng.
- **Tính di động:** Agent build đa kiến trúc (amd64, arm64) để chạy trên máy x86 và Jetson; runtime chọn nguồn khung hình và detector qua cấu hình.

## 4.2 Kiến trúc tổng thể

Kiến trúc tổng thể được thể hiện trong Hình 4.1. Người dùng làm việc qua Web Dashboard; mọi yêu cầu đi qua reverse proxy Nginx tới Cloud API, WebSocket hoặc AI Assistant. Cloud Server là một ứng dụng FastAPI duy nhất phục vụ đồng thời REST /api/v1, WebSocket /api/v1/ws, MCP server /mcp và bộ tiêu thụ MQTT (embedded worker). Trạng thái bền vững nằm trong PostgreSQL, telemetry ngắn hạn nằm trong Redis, còn kênh Cloud-Edge là MQTT broker. Model được lưu trên Hugging Face Hub; image runtime nằm trên GitHub Container Registry (GHCR).



Hình 4.1: Kiến trúc tổng thể nền tảng Edge AI

### 4.2.1 Control Plane và Data Plane

**Control Plane** (Cloud) chịu trách nhiệm quản lý tài nguyên, lưu desired state của từng runtime dưới dạng Deployment, phát lệnh, tổng hợp trạng thái và cung cấp dữ liệu giám sát. **Data Plane** (Edge) chịu trách nhiệm xử lý dữ liệu tại chỗ: thu nhận khung hình, suy luận, quản lý vòng đời container và thu thập telemetry. Hai mặt phẳng chỉ trao đổi qua các thông điệp đã định nghĩa trong gói hợp đồng dùng chung src/shared/contracts.

Luồng điều khiển đi theo chiều **Client** → **Cloud** → **Edge Agent** → **AI Runtime**; luồng telemetry đi theo chiều ngược lại **AI Runtime** → **Edge Agent** → **Cloud**. AI Runtime không kết nối trực tiếp tới broker mà gửi chỉ số qua HTTP tới API cục bộ của Agent; nhờ đó chỉ Agent giữ credential MQTT và Cloud chỉ cần tin cậy một thực thể trên mỗi node.

### 4.2.2 Giao thức MQTT và cấu trúc topic

MQTT [12] được chọn làm kênh Cloud–Edge vì nhẹ, hỗ trợ kết nối từ sau NAT (Edge chủ động kết nối ra ngoài), có cơ chế retained message và phù hợp với đường truyền không ổn định. Topic được tổ chức theo phân cấp tài nguyên như Bảng 4.2.

Bảng 4.2: Cấu trúc topic MQTT giữa Cloud và Edge

| Topic                                  | Chiều      | Nội dung                                          |
|----------------------------------------|------------|---------------------------------------------------|
| edge/{cluster}/{node}/command/{type}   | Cloud→Edge | Lệnh deploy, update, start, stop, restart, remove |
| edge/{cluster}/{node}/state            | Edge→Cloud | NodeState (retained), đồng thời là heartbeat      |
| edge/{cluster}/{node}/telemetry/{kind} | Edge→Cloud | system, gpu, runtime                              |
| edge/{cluster}/{node}/event/{kind}     | Edge→Cloud | runtime, error, deployment                        |

Cloud đăng ký các wildcard edge/+/\*/state/#, edge/+/\*/telemetry/+ và edge/+/\*/event/+; mỗi Agent chỉ đăng ký edge/{cluster}/{node}/command/+ của chính nó. Lớp EdgeTopics vừa sinh vừa phân tích topic, được cả Cloud và Edge dùng chung.

### 4.2.3 Tổ chức mã nguồn theo lớp

Mã nguồn backend là một monorepo Python 3.12, tách phần ứng dụng khởi chạy (apps/) khỏi phần nghiệp vụ (src/). Trong mỗi bounded context (cloud, edge,

assistant), mã được chia thành bốn lớp:

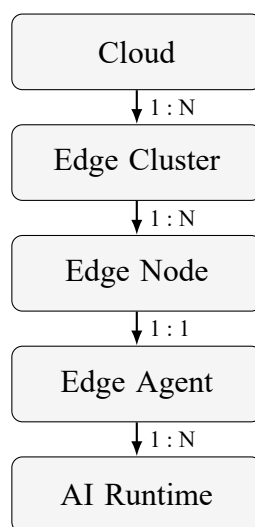
- **domain**: entity, value object và interface repository, không phụ thuộc framework;
- **application**: service thực hiện use case, handler tiêu thụ thông điệp và *port* (interface tới hạ tầng như container engine, camera, detector);
- **infrastructure**: *adapter* hiện thực port và repository (PostgreSQL, Redis, Docker CLI, OpenCV, Ultralytics, MQTT);
- **container**: lớp lắp ráp phụ thuộc, chọn adapter theo cấu hình (ví dụ `storage_backend=memory` để chạy không cần database).

Cách tổ chức port-adapter cho phép thay `DockerRuntimeDriver` bằng `FakeRuntimeDriver`, hay `OpenCvFrameSource` bằng `SyntheticFrameSource` khi kiểm thử, mà không sửa service nghiệp vụ. Ba gói nội bộ dùng lại giữa các dịch vụ: `hopquangdo-messaging` (trình tượng PubSub trên MQTT), `hopquangdo-http` (xác thực API key, health router, xử lý lỗi) và `hopquangdo-observability` (cấu hình log).

## 4.3 Thiết kế Domain

### 4.3.1 Mô hình phân cấp tài nguyên

Mô hình dữ liệu nghiệp vụ phản ánh phân cấp vật lý của hệ thống như Hình 4.2. Quan hệ Cluster-Node và Node-Runtime là một-nhiều; Node-Agent là một-một nên Agent không cần entity riêng mà được biểu diễn qua các thuộc tính của Node (`agent_version`, `last_heartbeat_at`).



Hình 4.2: Phân cấp tài nguyên của nền tảng

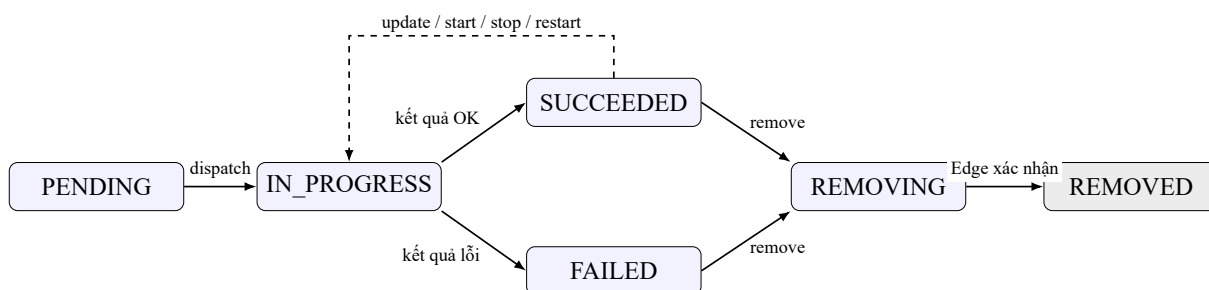
### 4.3.2 Cloud domain

Bảng 4.3 tóm tắt các entity của Cloud. Entity được hiện thực bằng dataclass, không phụ thuộc ORM; repository chuyển đổi giữa entity và bản ghi database.

Bảng 4.3: Các entity chính của Cloud domain

| Entity     | Thuộc tính chính                                                                                                   | Quy tắc nghiệp vụ                                                                                   |
|------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Cluster    | id, name, status, location, metadata                                                                               | Tên duy nhất trên toàn hệ thống.                                                                    |
| Node       | id, cluster_id, name, hostname, ip, status, agent_version, cpu_cores, memory_total_mb, gpu_name, last_heartbeat_at | Tên duy nhất trong cluster; thông tin phần cứng do Agent tự báo cáo; OFFLINE khi quá hạn heartbeat. |
| Runtime    | id, node_id, name, type, status, image, model_id, config, environment, resource_limits, container_id               | Trạng thái do Edge báo cáo; bị xóa khi Edge xác nhận gỡ bỏ.                                         |
| Deployment | id, cluster_id, node_id, runtime_id, spec, status, last_command_id, message                                        | Desired state của một runtime trên một node; chỉ chấp nhận kết quả của lệnh mới nhất.               |

**Deployment là aggregate trung tâm.** Mỗi lần phát lệnh, phương thức `dispatched(command_id)` ghi lại `last_command_id` và chuyển trạng thái. Khi Edge báo kết quả, `apply_result(command_id, status)` chỉ áp dụng nếu `command_id` trùng lệnh mới nhất; kết quả của lệnh cũ đến muộn bị bỏ qua. Đây là cơ chế chống ghi đè ngược trạng thái khi mạng giao lại thông điệp không theo thứ tự. Deployment ở trạng thái **REMOVED** là trạng thái kết thúc và không nhận lệnh mới.



Hình 4.3: Máy trạng thái của Deployment

Value object gồm `ModelRef` (`repo_id`, `filename`, `revision`), `ModelLocation` (URL tải đã ghim theo commit SHA) và `NodeHeartbeat` (các thông tin Node trích từ bản tin

NodeState).

#### 4.3.3 *Edge domain*

Phía Edge có entity RuntimeInstance gồm RuntimeSpec mong muốn, desired\_running, trạng thái hiện tại, container\_id, restart\_count, số lần khởi chạy thất bại liên tiếp và thông điệp lỗi gần nhất. DesiredStateRepository lưu danh sách RuntimeInstance xuống tệp JSON trong state\_dir để Agent khôi phục sau khi khởi động lại.

Vòng đời runtime dùng chung một enum giữa hai phía (Bảng 4.4).

Bảng 4.4: Trạng thái của AI Runtime

| Trạng thái         | Ý nghĩa                                                 |
|--------------------|---------------------------------------------------------|
| CREATING           | Agent đang tải model và pull image.                     |
| STARTING           | Container đang được khởi chạy.                          |
| RUNNING            | Container chạy bình thường.                             |
| DEGRADED           | Container chạy nhưng hiệu năng suy giảm.                |
| STOPPING / STOPPED | Đang dừng / đã dừng theo yêu cầu.                       |
| RESTARTING         | Đang khởi động lại.                                     |
| FAILED             | Khởi chạy thất bại hoặc vượt số lần thử tối đa.         |
| UNKNOWN            | Chưa xác định (ví dụ ngay sau khi Agent khởi động lại). |

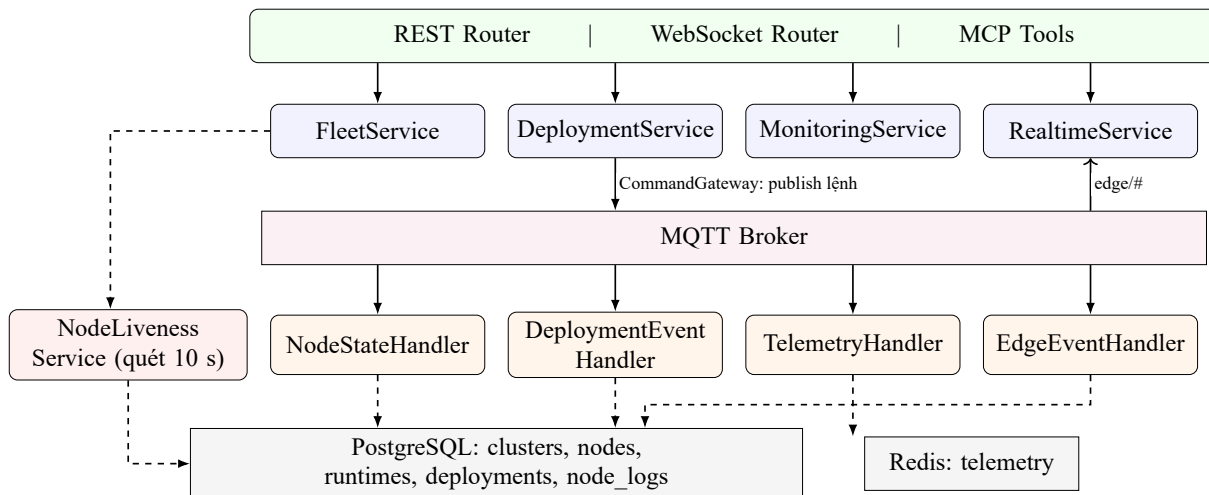
#### 4.3.4 *Hợp đồng dùng chung Cloud–Edge*

Gói src/shared/contracts định nghĩa bằng Pydantic toàn bộ thông điệp trao đổi: Command và RuntimeSpec (Cloud→Edge); NodeState, RuntimeState, SystemTelemetry, GpuTelemetry, RuntimeTelemetry, EdgeEvent, DeploymentEvent (Edge→Cloud) cùng các enum trạng thái. Giá trị enum trùng khớp với kiểu enum trong PostgreSQL. Vì hai phía import cùng một mã nguồn, thông điệp sai schema bị từ chối ngay tại nơi nhận thay vì làm hỏng dữ liệu.

RuntimeSpec là desired state của một runtime: runtime\_id, name, type (VISION, SAFETY, ANOMALY, OCR, CUSTOM), image\_uri, image\_tag, model\_id, model\_uri, config, environment và resource\_limits. Deploy và update là idempotent theo runtime\_id.

## 4.4 Thiết kế Cloud Control Plane

Cloud Control Plane gồm các application service xử lý yêu cầu đồng bộ từ người dùng và các handler xử lý thông điệp bất đồng bộ từ Edge (Hình 4.4).



Hình 4.4: Các thành phần của Cloud Control Plane

### 4.4.1 Fleet Service

FleetService hiện thực các use case trên phân cấp Cluster → Node → Runtime: tạo cluster (từ chối tên trùng), đăng ký node vào cluster (từ chối tên trùng trong cùng cluster), liệt kê và tra cứu. Node được đăng ký trước trên Cloud; khi Agent với node\_id tương ứng bắt đầu gửi NodeState, Cloud cập nhật thông tin phần cứng. Bản tin NodeState từ node chưa đăng ký bị ghi log và bỏ qua, tránh việc thiết bị lạ tự chen vào hệ thống.

### 4.4.2 Deployment Service

DeploymentService ghi desired state rồi phát lệnh; kết quả được Edge báo về bất đồng bộ. Do đó các API tạo, cập nhật, gỡ bỏ trả mã HTTP 202 Accepted. Các quy tắc chính:

- **Tạo mới:** kiểm tra node tồn tại. Nếu đã có runtime cùng tên trên node với deployment còn hiệu lực: spec giống hệt thì trả lại deployment cũ (gửi lại idempotent); spec khác thì báo xung đột và yêu cầu dùng PUT. Ngược lại, sinh runtime\_id, tạo bản ghi Runtime (CREATING) và Deployment, rồi phát lệnh deploy.
- **Cập nhật:** chỉ ghi đè những trường được gửi lên, đồng bộ bản ghi Runtime và phát lệnh update.
- **Gỡ bỏ:** chuyển REMOVING và phát lệnh remove; bản ghi Deployment được giữ làm

lịch sử, bản ghi Runtime bị xóa khi Edge xác nhận.

- **Điều khiển:** start/stop/restart chỉ áp dụng cho runtime có deployment còn hiệu lực.

Trước khi publish, service luôn gọi `dispatched()` và lưu deployment, nên trạng thái `IN_PROGRESS` và `last_command_id` đã bền vững trước khi lệnh rời khỏi Cloud. CommandGateway đóng gói việc publish lên topic `edge/{cluster}/{node}/command/{type}`.

#### 4.4.3 Các handler bất đồng bộ

- **NodeStateHandler** tiêu thụ `edge/+/+/state`: cập nhật heartbeat, trạng thái Agent, thông tin phân cứng và đồng bộ trạng thái, `container_id` của từng runtime do node báo cáo.
- **DeploymentEventHandler** tiêu thụ `event/deployment`: tìm deployment theo `deployment_id` hoặc `runtime_id`, áp dụng kết quả qua `apply_result`, cập nhật trạng thái runtime hoặc xóa runtime khi đã `REMOVED`.
- **TelemetryHandler** tiêu thụ `telemetry/{kind}`: kiểm tra schema theo loại, lưu mẫu vào Redis; mẫu runtime được khóa theo `runtime:{runtime_id}`.
- **EdgeEventHandler** tiêu thụ `event/+`: ghi mọi sự kiện vào nhật ký của node (`node_logs`).
- **NodeLivenessService** quét định kỳ (10 s) và chuyển các node có heartbeat cũ hơn `node_heartbeat_timeout_sec` (30 s) sang `OFFLINE`.

#### 4.4.4 Monitoring và Realtime Service

MonitoringService cung cấp mẫu telemetry mới nhất của một node (theo từng loại), lịch sử gần đây của một loại và danh sách event. RedisTelemetryRepository lưu `telemetry:{node}:latest` dạng hash và `telemetry:{node}:{kind}` dạng list giới hạn 720 mẫu (tương đương 1 giờ với chu kỳ 5 s), có TTL 1 giờ.

RealtimeService đăng ký `edge/#` trên broker và phân phối thông điệp tới các WebSocket client theo bộ lọc topic của từng client (hỗ trợ wildcard MQTT + và #). Client có thể đổi bộ lọc khi đang kết nối bằng thông điệp `subscribe/unsubscribe`; xác nhận ack đi qua cùng hàng đợi để giữ thứ tự với dữ liệu.

#### 4.4.5 Triển khai tiến trình

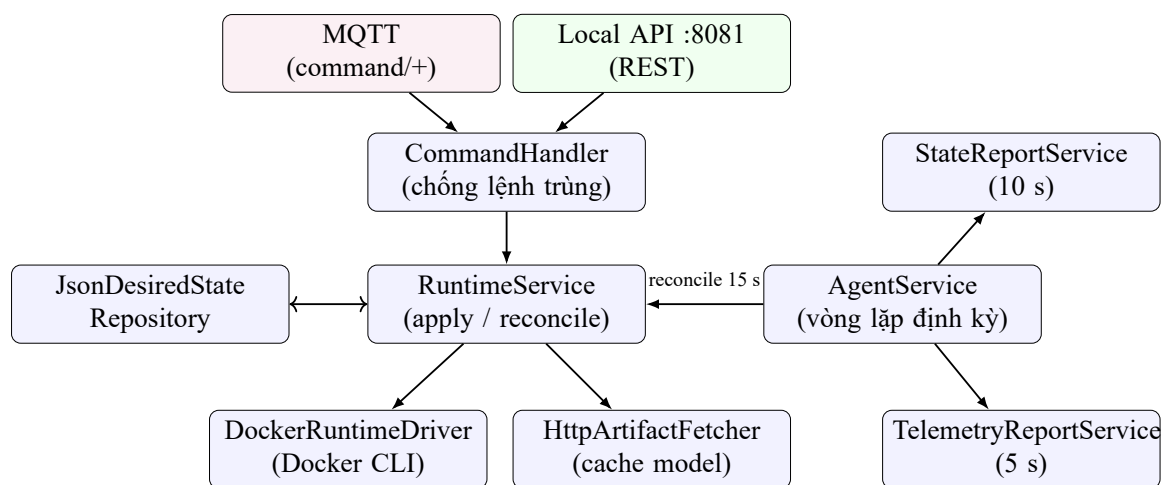
Cloud chạy như một tiến trình duy nhất trên cổng 8000, phục vụ REST, WebSocket, MCP và embedded worker (`CLOUD_EMBEDDED_WORKER=true`). Khi cần mở rộng, worker



có thể tách thành tiến trình riêng (apps/cloud/worker) trong khi API được nhân bản. Chế độ `CLOUD_STORAGE_BACKEND=memory` thay PostgreSQL và Redis bằng repository trong bộ nhớ, phục vụ phát triển và kiểm thử tự động.

## 4.5 Thiết kế Edge Agent

Edge Agent là tiến trình thường trực duy nhất được cài trên Edge Node; mọi AI Runtime do Agent tạo động từ image runtime. Các thành phần chính được thể hiện trong Hình 4.5.



Hình 4.5: Các thành phần chính của Edge Agent

### 4.5.1 Vòng lặp điều khiển

Khi khởi động, AgentService kết nối broker, khôi phục desired state từ đĩa, đăng ký topic lệnh và phát ngay một NodeState. Sau đó Agent chạy ba vòng lặp độc lập (chu kỳ cấu hình được):

- **State** (10 s): phát NodeState dạng retained, đóng vai trò heartbeat và báo cáo toàn bộ runtime. Agent ở trạng thái `DEGRADED` nếu có runtime mong muốn chạy nhưng đang `FAILED` hoặc `DEGRADED`.
- **Telemetry** (5 s): thu và phát telemetry system và gpu.
- **Reconcile** (15 s): so sánh desired state với trạng thái container thực tế và hội tụ.

Khi dừng, Agent hủy các vòng lặp và phát NodeState `OFFLINE` trước khi ngắt kết nối.

### 4.5.2 Xử lý lệnh

CommandHandler nhận lệnh từ MQTT hoặc từ API cục bộ; hai nguồn đi qua cùng một đường xử lý nên kết quả đều được báo về Cloud. Lệnh không đúng schema bị từ chối; lệnh có `command_id` đã xử lý (bộ nhớ đệm 256 id gần nhất) bị bỏ qua để chống giao lặp của broker. Sau khi thực thi, Agent phát `DeploymentEvent` chứa `command_id`, `deployment_id`, trạng thái `deployment` và trạng thái `runtime`, rồi phát ngay một `NodeState` mới. Lỗi nghiệp vụ (`AppException`) và lỗi bất ngờ đều được chuyển thành kết quả `FAILED` có thông điệp, không làm dừng Agent.

### 4.5.3 Desired state và reconcile

RuntimeService giữ desired state của mọi runtime trên node và là nơi duy nhất thay đổi container. Mọi thao tác đi qua một khóa bất đồng bộ nên các lệnh trên cùng node được tuần tự hóa. Các thao tác đều idempotent:

- `apply(spec)`: nếu spec giống hệt và runtime đang `RUNNING` thì không làm gì; nếu khác, hủy container cũ, lưu desired state và khởi chạy mới.
- `start/stop`: đặt `desired_running` rồi thao tác container; `remove` với runtime không tồn tại vẫn thành công.
- `reconcile()`: với runtime mong muốn dừng mà container đang chạy thì dừng; mong muốn chạy mà container mất hoặc đã thoát thì ghi lý do (mã thoát, lỗi), xóa container cũ và tạo lại; sau `max_failures` (5) lần thất bại liên tiếp thì đánh dấu `FAILED` và ngừng thử.
- `restore()`: sau khi Agent khởi động lại, đọc desired state từ đĩa và tìm container còn sống theo nhãn `edgeops.runtime_id` để tiếp quản thay vì tạo trùng.

Mỗi lần trạng thái runtime thay đổi, Agent phát một `EdgeEvent(runtime.status)`; chuyển sang `FAILED` được phát trên kênh `error` với mức `ERROR`.

### 4.5.4 Khởi chạy runtime

Quy trình `_launch` gồm: (1) chuyển `CREATING`; (2) nếu spec có `model_uri`, tải model qua `ArtifactFetcher` và gắn vào container tại `/models/<file>` ở chế độ chỉ đọc; (3) pull image theo chính sách `always/missing/never`; (4) chuyển `STARTING` và chạy container với biến môi trường `RUNTIME_ID`, `RUNTIME_NAME`, `RUNTIME_CONFIG`, `MODEL_PATH`, `NODE_ID`, `AGENT_URL`; (5) chuyển `RUNNING`. Giới hạn tài nguyên trong spec được ánh xạ sang tham số Docker: `--gpus`, `--runtime nvidia` (Jetson), `--cpus`, `--memory`, `--device` (camera). Container được gắn chính sách `--restart`

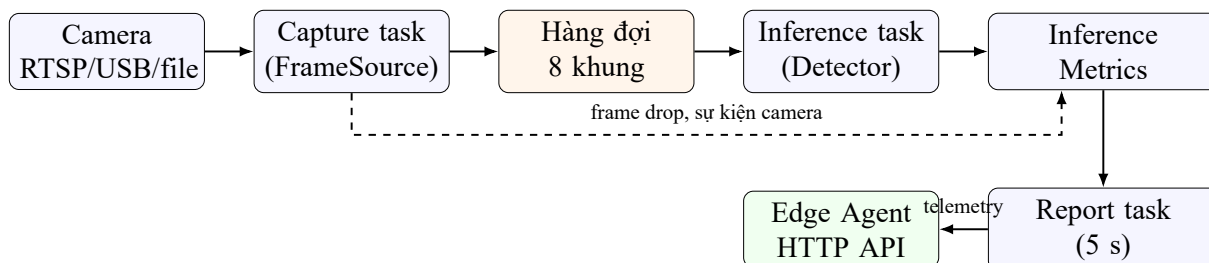
unless-stopped nên Docker tự khởi động lại khi thiết bị reboot.

#### 4.5.5 API cục bộ

Agent cung cấp REST API trên cổng 8081 (bảo vệ bằng API key) gồm hai nhóm: nhóm quản lý (GET /status, GET/POST/PUT/DELETE /runtimes, POST /runtimes/{id}/{start|stop|restart}) cho phép vận hành tại chỗ khi không có Cloud; và nhóm báo cáo (POST /runtimes/{id}/telemetry, POST /runtimes/{id}/events) để AI Runtime gửi chỉ số. Agent làm giàu mẫu telemetry runtime với node\_id, trạng thái, số lần restart, tính  $\text{effective\_fps\_ratio} = \text{FPS thực} / \text{FPS mục tiêu}$ , rồi chuyển tiếp lên Cloud.

## 4.6 Thiết kế Edge Runtime

AI Runtime là một container độc lập do Agent tạo ra. Toàn bộ cấu hình được truyền qua biến môi trường; khóa RUNTIME\_CONFIG (JSON) chọn nguồn khung hình (source), FPS mục tiêu (target\_fps, mặc định 15), detector (ultralalytics khi có MODEL\_PATH, ngược lại synthetic), ngưỡng tin cậy (conf) và chu kỳ báo cáo. Luồng xử lý được thể hiện trong Hình 4.6.



Hình 4.6: Thiết kế pipeline Edge Runtime

**Ba tác vụ song song.** InferenceService chạy ba tác vụ trong một TaskGroup: *capture* đọc khung từ FrameSource; *infer* lấy khung từ hàng đợi và gọi Detector với thời gian chờ tối đa 2 s; *report* định kỳ chụp ảnh chỉ số và gửi cho Agent. Khi một tác vụ lỗi nghiêm trọng, cả nhóm dừng và container thoát, để vòng reconcile của Agent tạo lại container.

**Hàng đợi giới hạn, ưu tiên khung mới.** Hàng đợi chỉ giữ 8 khung. Khi đầy, khung cũ nhất bị loại và được tính là *frame drop*. Chính sách này giữ độ trễ không tăng vô hạn khi detector chậm hơn camera: hệ thống chấp nhận bỏ khung để luôn suy luận trên dữ liệu mới nhất.

**Nguồn khung và detector là port.** `OpenCvFrameSource` đọc RTSP, USB (chỉ số thiết bị) hoặc tệp video, tự kết nối lại sau 3 s khi luồng mất và phát sự kiện `camera.disconnected`. `UltralyticsDetector` nạp mô hình qua API `YOLO(...)` của Ultralytics nên dùng được trực tiếp tệp `.pt`, `.onnx` hoặc `TensorRT .engine`; lời gọi suy luận chạy trong thread riêng để không chặn vòng sự kiện. Các adapter `SyntheticFrameSource` và `SyntheticDetector` cho phép chạy toàn bộ pipeline khi không có camera hoặc GPU.

**Chỉ số runtime.** `InferenceMetrics` tích lũy trong mỗi cửa sổ báo cáo: FPS suy luận và FPS camera, latency trung bình và p50/p95/p99, tỷ lệ lỗi và timeout, tỷ lệ frame drop, tuổi khung mới nhất, thời gian camera bị đứng hình, độ sáng trung bình, số detection, confidence trung bình/độ lệch/nhỏ nhất và entropy phân bố lớp. Các trường này khớp tên với đặc tả telemetry ở Mục 4.8.

## 4.7 Thiết kế Model Deployment

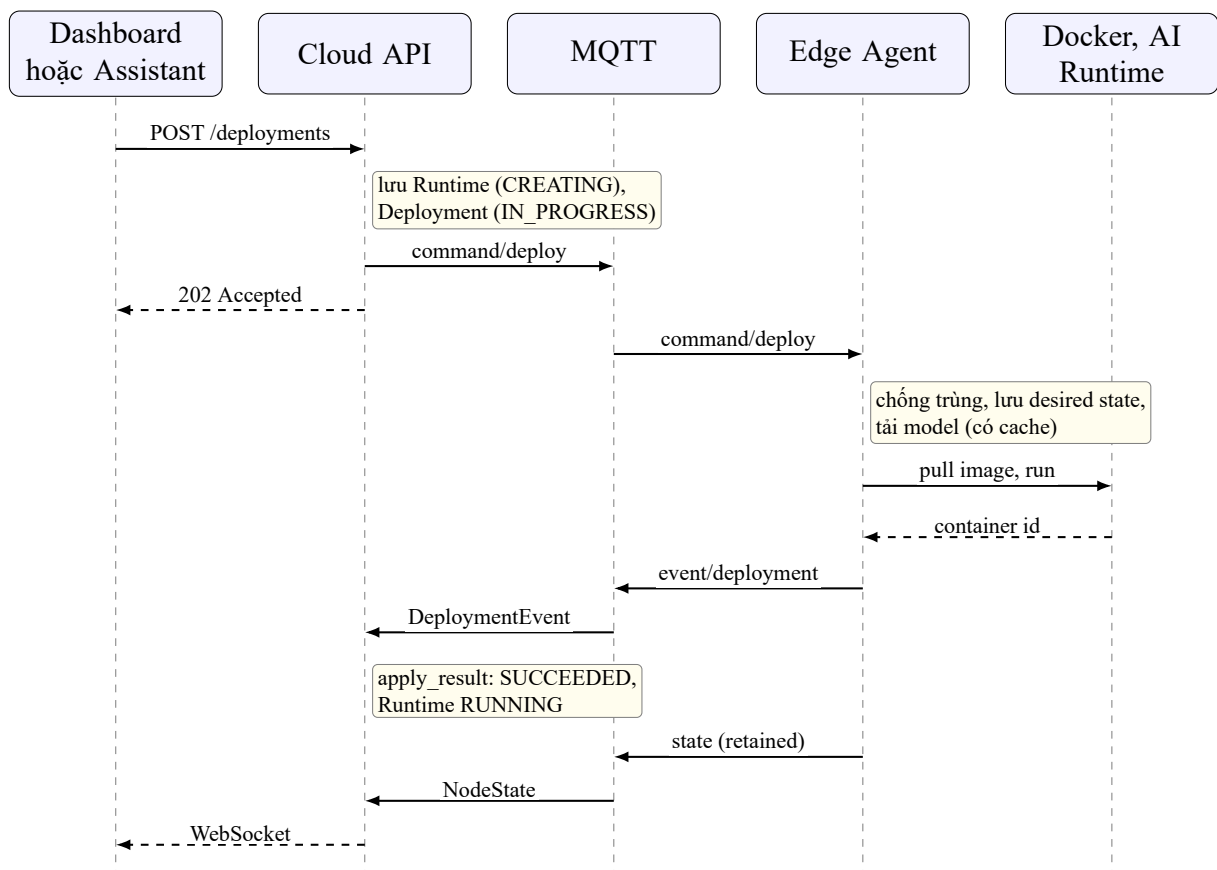
### 4.7.1 Lưu trữ và phân phối model

Model sau tối ưu (Chương 3) được lưu trên một kho model kiểu Hugging Face Hub, có thể là `huggingface.co`, mirror hoặc máy chủ tự dựng. Mỗi model được tham chiếu bằng `ModelRef(repo_id, filename, revision)`. Khi phân giải, `HuggingFaceModelRepository` chuyển `revision` (nhánh hoặc tag) thành `commit SHA` cụ thể và sinh URL tải đã ghim theo SHA đó (`ModelLocation`). Vì URL trỏ đến nội dung bất biến của một commit, hai node triển khai cùng một deployment chắc chắn nhận cùng một tệp model, kể cả khi nhánh main được cập nhật giữa chừng.

Phía Edge, `HttpArtifactFetcher` tải model qua HTTPS theo từng khối 1 MB vào tệp tạm `.part`, sau đó đổi tên nguyên tử thành tệp đích. Tệp được lưu cache theo hàm băm SHA-256 của URL (bỏ phần query), nên triển khai lại cùng model không tải lại. URI `file://` hoặc đường dẫn cục bộ được dùng trực tiếp, thuận tiện cho kiểm thử trên thiết bị.

### 4.7.2 Luồng triển khai

Luồng triển khai đầy đủ được mô tả trong Hình 4.7.



Hình 4.7: Trình tự triển khai model từ Dashboard đến AI Runtime

### 4.7.3 Cập nhật và khôi phục

Cập nhật model hoặc cấu hình là một lệnh update với spec mới trên cùng `runtime_id`. Agent chỉ tạo lại container khi spec thực sự thay đổi. Nếu khởi chạy với model mới thất bại, Agent báo **FAILED** kèm thông điệp lỗi (ví dụ không tải được model, image không tồn tại); người vận hành hoặc Assistant có thể gửi lại một update trở về model trước đó để khôi phục. Vì mỗi lệnh update mang `command_id` mới, kết quả của lần thử lỗi trước không ghi đè kết quả khôi phục. Việc tự động canary, chia đợt và rollback theo ngưỡng telemetry được xác định là hướng phát triển (Chương 7).

## 4.8 Thiết kế Monitoring và cảnh báo

Giám sát được thiết kế theo hai kênh bổ trợ nhau. Kênh **telemetry ứng dụng** đi qua MQTT, mang ngữ nghĩa của nền tảng (runtime, FPS, camera, chất lượng đầu ra model) và phục vụ Dashboard, Assistant. Kênh **metric hạ tầng** dùng hệ sinh thái Prometheus [23] để thu metric node, container và GPU một cách chuẩn hóa, lưu dài hạn và cảnh báo theo luật.

### 4.8.1 Đặc tả telemetry

Đặc tả telemetry chia chỉ số thành bảy nhóm và ba loại thông điệp (Bảng 4.5). Tất cả trường đều tùy chọn để thiết bị chỉ báo cáo những gì đo được; tên trường thống nhất giữa Edge Agent, kho lưu trữ, bộ mô phỏng dữ liệu và pipeline học máy.

Bảng 4.5: Nhóm đặc trưng telemetry của Edge Node

| Nhóm                   | Thông điệp  | Ví dụ trường                                                                                                   |
|------------------------|-------------|----------------------------------------------------------------------------------------------------------------|
| Tài nguyên node        | system      | cpu_usage_pct, cpu_load_1m/5m, memory_usage_pct, swap_usage_pct                                                |
| Nhiệt và điện          | system, gpu | temperature_c, thermal_throttling, power_w, gpu_temperature_c                                                  |
| Lưu trữ                | system      | disk_usage_pct, disk_free_gb, inode_usage_pct                                                                  |
| Mạng                   | system      | network_rx/tx_mb_s, packet_loss_pct, mqtt_publish_latency_ms                                                   |
| GPU                    | gpu         | gpu_usage_pct, gpu_memory_used_mb, gpu_frequency_mhz, gpu_power_w                                              |
| Runtime và suy luận    | runtime     | inference_fps, effective_fps_ratio, inference_p95_ms, inference_error_rate, runtime_queue_size                 |
| Camera và đầu ra model | runtime     | camera_fps, camera_frame_drop_pct, camera_freeze_duration_sec, detection_count, confidence_mean, class_entropy |

Telemetry system được thu bằng psutil; telemetry gpu được thu bằng nvidia-smi trên máy có GPU rồi (collector trả về rỗng khi công cụ không có, ví dụ trên Jetson); telemetry runtime do AI Runtime tự đo.

Nhóm *đầu ra model* có ý nghĩa đặc biệt: confidence\_mean, detection\_count và class\_entropy cho phép nhận biết suy giảm chất lượng suy luận (camera mờ, bị che, đổi góc, ánh sáng thay đổi, dữ liệu trôi) ngay cả khi CPU, GPU, FPS và latency vẫn bình thường — điều mà giám sát hạ tầng thuần túy không phát hiện được.

### 4.8.2 Metric hạ tầng với Prometheus

Trên mỗi Edge Node, node-exporter (metric hệ điều hành), cAdvisor (metric container) và dcgm-exporter (GPU NVIDIA, tùy chọn) chỉ lắng nghe trên 127.0.0.1. Một Prometheus chạy ở *agent mode* thu các exporter mỗi 15 s, gán nhãn node\_id, cluster\_id và đẩy về Prometheus trên Cloud bằng remote\_write có xác thực. Mô

hình đẩy (push) hoạt động được khi Edge nằm sau NAT; WAL của agent đệm mẫu khi mất mạng và gửi bù khi kết nối lại.

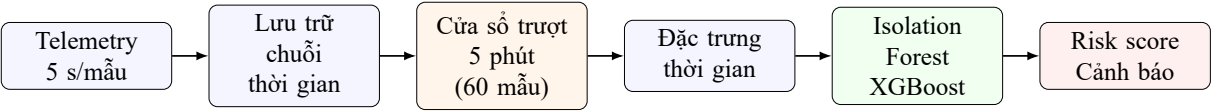
Trên Cloud, Prometheus lưu metric (mặc định 30 ngày) và đánh giá luật cảnh báo; Alertmanager định tuyến cảnh báo; Grafana hiển thị và cho phép nhúng panel vào Dashboard. Bộ luật cảnh báo Edge được liệt kê trong Bảng 4.6.

Bảng 4.6: Luật cảnh báo metric hạ tầng Edge

| Cảnh báo                | Điều kiện                                                          | Kéo dài | Mức      |
|-------------------------|--------------------------------------------------------------------|---------|----------|
| EdgeNodeDown            | Node từng báo cáo trong 1 ngày nhưng không đẩy metric trong 5 phút | 2 phút  | critical |
| EdgeDiskAlmostFull      | Phân vùng gốc sử dụng trên 90%                                     | 10 phút | warning  |
| EdgeHighMemory          | Bộ nhớ sử dụng trên 90%                                            | 10 phút | warning  |
| EdgeContainerRestarting | Container khởi động lại hơn 2 lần trong 15 phút                    | –       | warning  |
| EdgeGpuHot              | Nhiệt độ GPU trên 85 °C                                            | 5 phút  | warning  |

4.8.3 Thiết kế mô-đun cảnh báo sớm

Trên nền dữ liệu telemetry đã chuẩn hóa, đồ án thiết kế mô-đun cảnh báo sớm suy giảm (early warning) theo luồng ở Hình 4.8.



Hình 4.8: Luồng xử lý của mô-đun cảnh báo sớm suy giảm

**Đặc trưng thời gian.** Với chu kỳ lấy mẫu 5 s và cửa sổ 5 phút, mỗi chỉ số cốt lõi được biến đổi thành current, mean, min, max, std, slope (hệ số hồi quy tuyến tính) và delta (chênh lệch so với đầu cửa sổ). Bộ chỉ số khởi đầu gồm khoảng 20 trường: CPU, bộ nhớ và tốc độ tăng bộ nhớ, GPU, nhiệt độ và tốc độ tăng nhiệt, đĩa, RTT, mất gói, FPS và effective\_fps\_ratio, p95 latency, tỷ lệ lỗi, kích thước hàng đợi, số lần restart, FPS camera, tuổi khung, tỷ lệ drop, số detection và confidence trung bình. Một số đặc trưng dẫn xuất như latency\_ratio = p95 hiện tại / p95 baseline và tốc độ tăng hàng đợi giúp mô hình học quan hệ đa biến thay vì ngưỡng cứng.

**Nhân trạng thái.** Mỗi thời điểm được gán một trong bốn trạng thái vận hành: NORMAL, WARNING, DEGRADED, CRITICAL. Ví dụ DEGRADED khi FPS giảm rõ, latency và hàng đợi tăng; CRITICAL khi runtime FAILED, camera mất kết nối hoặc

node OFFLINE.

**Hai bài toán tách biệt.** Isolation Forest trả lời “hành vi hiện tại có bất thường không” (không cần nhãn). XGBoost trả lời “trong 10 phút tới có suy giảm không”: nhãn `future_degradation=1` nếu node chuyển sang DEGRADED hoặc CRITICAL trong 10 phút sau cửa sổ quan sát. Để tránh rò rỉ nhãn, các trường chỉ xuất hiện sau sự cố (ví dụ `runtime_status=FAILED`) không được dùng làm đầu vào dự báo. Dữ liệu thô luôn được giữ lại để có thể thay đổi cách trích đặc trưng mà không cần thu thập lại.

Ví dụ phân biệt: khi GPU tăng, nhiệt độ tăng, latency và hàng đợi tăng trong khi FPS và confidence giảm, mô hình cảnh báo nguy cơ suy giảm dù từng chỉ số chưa vượt ngưỡng. Ngược lại, khi GPU tăng do cảnh có nhiều đối tượng nhưng latency, FPS, hàng đợi và confidence ổn định, trạng thái vẫn là NORMAL, tránh cảnh báo sai. Trong phạm vi đồ án, hợp đồng telemetry, thu thập và lưu trữ đã được hiện thực; việc huấn luyện và tích hợp mô hình cảnh báo sớm vào luồng cảnh báo được trình bày ở mức thiết kế và là hướng phát triển tiếp theo.

## 4.9 Thiết kế AI Assistant

AI Assistant là một dịch vụ độc lập giúp người vận hành truy vấn và điều khiển đội thiết bị bằng ngôn ngữ tự nhiên. Nguyên tắc thiết kế cốt lõi là **Assistant không bao giờ truy cập trực tiếp cơ sở dữ liệu hay broker**: mọi dữ liệu và thao tác đều đi qua MCP server của Cloud, nơi cùng dùng các application service với REST API. Nhờ đó quy tắc nghiệp vụ, kiểm tra dữ liệu và xác thực chỉ được hiện thực một lần.

### 4.9.1 Công cụ qua Model Context Protocol

Cloud công bố 17 công cụ qua Model Context Protocol [20] (streamable HTTP tại `/mcp`, bảo vệ bằng API key), chia làm ba nhóm:

- **Fleet:** `list_clusters`, `get_cluster`, `create_cluster`, `list_nodes`, `get_node`, `register_node`, `list_runtimes`, `get_runtime`;
- **Deployment:** `list_deployments`, `get_deployment`, `create_deployment`, `update_deployment`, `remove_deployment`, `control_runtime`;
- **Monitoring:** `get_latest_telemetry`, `get_telemetry_history`, `get_node_events`.

Schema tham số của công cụ được sinh tự động từ chữ ký hàm Python, và phía Assistant chuyển mỗi công cụ MCP thành một StructuredTool của LangChain. Công cụ chỉ được định nghĩa một lần tại Cloud; thêm công cụ mới không cần sửa Assistant.

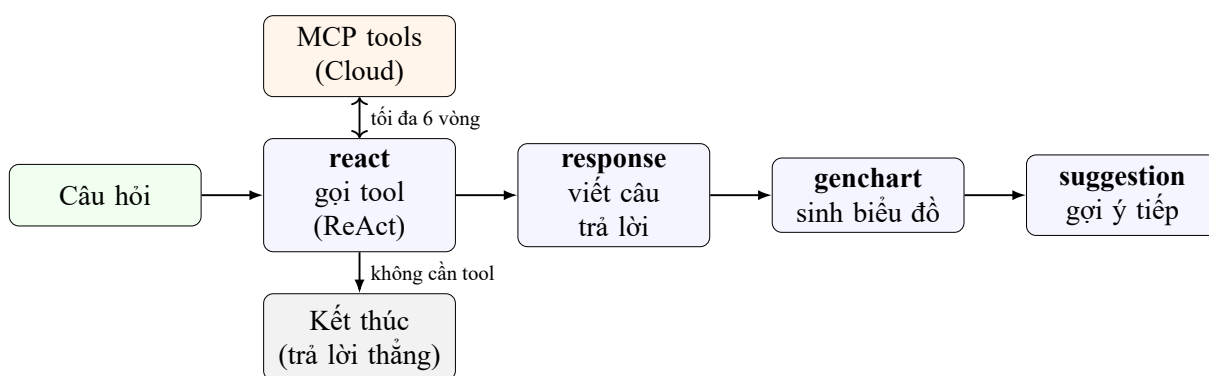


### 4.9.2 Kiểm soát thao tác thay đổi

Các công cụ có tiền tố `create_`, `update_`, `remove_`, `register_`, `control_` làm thay đổi trạng thái hệ thống. Chúng bị ẩn hoàn toàn khỏi mô hình trừ khi cấu hình cho phép ghi. Khi được phép, prompt hệ thống và mô tả của từng công cụ yêu cầu mô hình chỉ gọi sau khi đã mô tả chính xác hành động và người dùng xác nhận rõ ràng ở một tin nhắn trước đó. Như vậy có hai lớp bảo vệ: lớp cấu hình (quyết định công cụ có tồn tại với mô hình hay không) và lớp hội thoại (bắt buộc xác nhận).

### 4.9.3 Đồ thị tác tử

Một lượt hội thoại được xử lý bằng đồ thị LangGraph [21] bốn nút, trong đó nút đầu tiên theo mẫu ReAct [22] (Hình 4.9).



Hình 4.9: Đồ thị xử lý một lượt hội thoại của AI Assistant

- **react:** mô hình chính (mặc định Claude Sonnet) lặp tối đa 6 vòng *gọi tool* → *nhận kết quả*, chỉ thu thập dữ liệu mà không viết câu trả lời. Nếu ngay vòng đầu mô hình không cần tool (chào hỏi, hỏi lại khi thiếu đối tượng, hỏi xác nhận), câu trả lời đó được dùng luôn và đồ thị kết thúc.
- **response:** tổng hợp kết quả tool thành câu trả lời, phát dần từng token về giao diện.
- **genchart:** mô hình nhỏ (Claude Haiku) quyết định dữ liệu có đáng vẽ biểu đồ không và sinh đặc tả biểu đồ có cấu trúc; lỗi ở bước này chỉ làm mất biểu đồ, không làm hỏng lượt hội thoại.
- **suggestion:** sinh các câu hỏi gợi ý tiếp theo.

**Thực thi tool an toàn.** ToolRunner chạy các lời gọi tool của cùng một vòng song song (tối đa 8), gộp lời gọi trùng tên và tham số, áp thời gian chờ 60 s cho mỗi tool. Tool không tồn tại, lỗi hoặc quá thời gian được trả về mô hình dưới dạng dữ liệu lỗi để mô hình tự sửa tham số hoặc giải thích, thay vì làm dừng cả lượt. Lịch sử hội thoại được giữ theo `session_id` bằng checkpoint của LangGraph, cho phép câu hỏi nối tiếp như

“còn node 2 thì sao?”.

**Giao thức phát trực tuyến.** Client gửi POST /chat/stream để khởi tạo lượt và nhận stream\_id, sau đó mở GET /chat/stream/{id} dạng Server-Sent Events. Các sự kiện theo thứ tự: status (đang phân tích, đang gọi tool nào), tool\_end, token, done, chart, suggestions, hoặc một sự kiện error duy nhất.

#### 4.9.4 So sánh với hướng tiếp cận RAG

Trong bài toán vận hành, câu hỏi chủ yếu hướng tới *trạng thái hiện tại* của hệ thống (node nào offline, runtime nào lỗi, FPS vừa giảm bao nhiêu). Dữ liệu này thay đổi liên tục nên truy xuất qua tool có cấu trúc chính xác hơn so với tìm kiếm văn bản đã lập chỉ mục [14]. Vì vậy đề án ưu tiên tool calling qua MCP; RAG trên tài liệu, runbook được giữ ở vai trò mở rộng cho các câu hỏi về quy trình và kiến thức ổn định.

### 4.10 Kết chương

Chương 4 đã chuyển yêu cầu của đề tài thành kiến trúc Cloud Control Plane – Edge Data Plane. Cloud lưu desired state dưới dạng Deployment và phát lệnh qua MQTT; Edge Agent hội tụ trạng thái container về desired state bằng các thao tác idempotent, vòng reconcile và desired state lưu trên đĩa; AI Runtime chạy pipeline camera–suy luận với hàng đợi giới hạn và tự đo chỉ số. Hợp đồng dùng chung, cơ chế last\_command\_id, chống lệnh trùng và giới hạn số lần thử giữ cho hệ thống nhất quán qua kết nối không tin cậy. Telemetry được đặc tả thống nhất làm nền cho giám sát và cảnh báo sớm; AI Assistant truy cập hệ thống duy nhất qua công cụ MCP với cơ chế kiểm soát thao tác ghi. Chương 5 trình bày việc hiện thực và triển khai các thành phần này.

# CHƯƠNG 5. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

Chương này mô tả cách hiện thực kiến trúc ở Chương 4 thành các dịch vụ chạy được: Cloud Server, Edge Agent, AI Runtime, AI Assistant, hệ thống giám sát và Web Dashboard, cùng quy trình đóng gói container và CI/CD đưa hệ thống lên máy chủ thật.

## 5.1 Môi trường và công nghệ

### 5.1.1 Công nghệ sử dụng

Bảng 5.1 liệt kê các công nghệ chính. Phiên bản image hạ tầng được ghim trong tệp Docker Compose để môi trường có thể tái tạo.

Bảng 5.1: Các thành phần phần mềm của hệ thống

| Thành phần              | Công nghệ và phiên bản                                                                                        |
|-------------------------|---------------------------------------------------------------------------------------------------------------|
| Ngôn ngữ backend        | Python 3.12, lập trình bất đồng bộ asyncio                                                                    |
| Backend/API             | FastAPI 0.141, Uvicorn, Pydantic 2.13, pydantic-settings                                                      |
| Database                | PostgreSQL 16 (image postgres:16-alpine), driver asyncpg                                                      |
| Cache telemetry         | Redis 7 (redis:7-alpine)                                                                                      |
| Message broker          | Eclipse Mosquitto 2 (MQTT), client paho-mqtt qua gói hopquangdo-messaging                                     |
| Model store             | Hugging Face Hub (huggingface_hub)                                                                            |
| Container registry      | GitHub Container Registry (GHCR)                                                                              |
| Container engine (Edge) | Docker, NVIDIA Container Runtime trên Jetson                                                                  |
| AI Runtime              | Ultralytics YOLO (PyTorch/ONNX/TensorRT), OpenCV                                                              |
| AI Assistant            | LangGraph 1.x, LangChain 1.x, MCP Python SDK, Claude Sonnet 5 (mô hình chính), Claude Haiku 4.5 (mô hình phụ) |
| Giám sát                | Prometheus 2.54, Alertmanager 0.27, Grafana 11.2, node-exporter 1.8, cAdvisor 0.49, DCGM exporter 3.3         |
| Frontend                | React 19, TypeScript 6, Vite 8, Tailwind CSS 4, React Router 7                                                |
| Kiểm thử và CI/CD       | pytest, pytest-asyncio, Ruff, oxlint, GitHub Actions                                                          |

### 5.1.2 *Môi trường Cloud*

Cloud được triển khai trên một máy ảo AWS EC2 chạy Linux, tại thư mục `/opt/edgeops`. Các thành phần được đóng gói thành bốn *stack* Docker Compose độc lập, dùng chung mạng Docker `edgeops-cloud`:

- **cloud-infra**: PostgreSQL, Redis (có mật khẩu), Mosquitto. Các tệp migration SQL được gắn vào `/docker-entrypoint-initdb.d` nên schema được tạo tự động ở lần khởi động đầu tiên.
- **cloud**: image `cloud-api`, một tiến trình phục vụ REST, WebSocket, MCP và embedded worker trên cổng 8000.
- **assistant**: image `assistant` trên cổng 8100, gọi Cloud qua tên dịch vụ nội bộ `http://api:8000`.
- **web**: image `web-dashboard` (Nginx) trên cổng 80, phục vụ giao diện và reverse proxy tới API, WebSocket và Assistant.

Stack `cloud-monitoring` (Prometheus, Alertmanager, Grafana) được khởi chạy riêng khi cần giám sát metric hạ tầng. Tất cả cấu hình nhạy cảm (mật khẩu, API key, token Hugging Face, khóa LLM) được đọc từ tệp `.env`, không nằm trong mã nguồn.

### 5.1.3 *Môi trường Edge*

Mỗi Edge Node cần Docker (và NVIDIA Container Runtime nếu dùng GPU). Stack Edge chỉ chạy tĩnh một container `edge-agent`; container này được gắn Docker socket của máy chủ để tạo các container AI Runtime anh em (sibling container). Cấu hình node gồm `EDGE_CLUSTER_ID`, `EDGE_NODE_ID`, địa chỉ MQTT và API key. Image Agent được build đa kiến trúc `linux/amd64` và `linux/arm64`, chạy được trên máy x86 và Jetson; image runtime cho Jetson cần base image L4T tương ứng phiên bản JetPack. Hai thiết bị mục tiêu là Jetson Nano và Jetson Orin Nano (thông số ở Bảng 2.2); máy trạm x86 có GPU NVIDIA được dùng làm Edge Node trong quá trình phát triển.

### 5.1.4 *Mạng*

Edge chủ động kết nối ra Cloud qua MQTT (cổng công bố 11883) nên không cần mở cổng vào trên thiết bị; điều này phù hợp với thiết bị nằm sau NAT. Tải model đi qua HTTPS tới Hugging Face Hub; metric hạ tầng được đẩy qua HTTPS bằng Prometheus remote write. Script `tunnel-cloud-infra.sh` tạo đường hầm SSH tới PostgreSQL, Redis, MQTT trên Cloud để phát triển từ máy cá nhân mà không công khai các cổng quản trị.

5.1.5 CI/CD

Quy trình tích hợp và triển khai liên tục dùng GitHub Actions (Bảng 5.2). Workflow tái sử dụng `_build-deploy.yml` build image, đẩy lên GHCR với tag là commit SHA, sao chép tệp Compose và `.env` (lấy từ GitHub Secrets) lên EC2 qua SSH, chạy `docker compose pull` và `up -d`, rồi gọi health check sau khi cập nhật.

Bảng 5.2: Các workflow CI/CD

| Workflow             | Kích hoạt                       | Công việc                                                                       |
|----------------------|---------------------------------|---------------------------------------------------------------------------------|
| ci.yml               | Mọi pull request, push lên main | Ruff, pytest cho backend; lint và build Dashboard                               |
| ci-edge.yml          | Thay đổi mã Edge                | Build image edge-agent (amd64 + arm64) và edge-runtime; push lên GHCR khi merge |
| deploy-cloud.yml     | Thay đổi Cloud, migration       | Build cloud-api, triển khai, kiểm tra /health                                   |
| deploy-assistant.yml | Thay đổi Assistant              | Build và triển khai assistant                                                   |
| deploy-web.yml       | Thay đổi Dashboard              | Build và triển khai web-dashboard                                               |

5.2 Xây dựng Cloud Server

5.2.1 Điểm khởi chạy và vòng đời

Tệp `apps/cloud/main.py` tạo ứng dụng FastAPI từ hai container phụ thuộc: `CloudContainer` (repository, service, handler, kết nối MQTT) và `RealtimeContainer` (luồng WebSocket). Trong lifespan của ứng dụng, hệ thống lần lượt: khởi động container; chạy session manager của MCP trong một task riêng; nếu bật embedded worker thì đăng ký các consumer MQTT và chạy `NodeLivenessService`. Khi tắt, các task được hủy theo thứ tự ngược lại để không rò rỉ kết nối.

Router REST được gắn tiền tố `/api/v1` và phụ thuộc `ApiKeyAuth`; ứng dụng MCP được mount tại `/mcp` và bọc bởi `ApiKeyGuard` — một ASGI middleware kiểm tra header `x-api-key`, vì ứng dụng được mount không đi qua dependency của FastAPI. `/health` kiểm tra đồng thời database, Redis và broker.

### 5.2.2 REST API

Bảng 5.3 liệt kê các endpoint chính; đặc tả đầy đủ ở Phụ lục C.

Bảng 5.3: REST API của Cloud

| Method     | Endpoint                                | Chức năng                                |
|------------|-----------------------------------------|------------------------------------------|
| GET/POST   | /api/v1/clusters                        | Liệt kê, tạo cluster                     |
| GET/POST   | /api/v1/nodes                           | Liệt kê (lọc theo cluster), đăng ký node |
| GET        | /api/v1/agents/{node_id}                | Trạng thái Agent kèm runtime             |
| GET        | /api/v1/runtimes                        | Liệt kê runtime (lọc theo node)          |
| POST       | /api/v1/runtimes/{id}/{action}          | start/stop/restart                       |
| POST/GET   | /api/v1/deployments                     | Tạo (202), liệt kê                       |
| PUT/DELETE | /api/v1/deployments/{id}                | Cập nhật, gỡ bỏ (202)                    |
| GET        | /api/v1/monitoring/nodes/{id}/telemetry | Telemetry mới nhất                       |
| GET        | /api/v1/monitoring/nodes/{id}/events    | Event gần đây                            |
| WS         | /api/v1/ws?topics=...                   | Luồng trạng thái/telemetry/event         |

Lỗi nghiệp vụ được biểu diễn bằng `AppException` với mã chuẩn hóa (`NOT_FOUND`, `CONFLICT`, `INVALID_REQUEST`, `EXTERNAL_SERVICE...`) và được `AppExceptionHandler` chuyển thành mã HTTP tương ứng, giúp client và Assistant xử lý lỗi thống nhất.

### 5.2.3 Tầng lưu trữ

Repository PostgreSQL dùng `asyncpg` với truy vấn SQL viết tay và lớp `RowMapper` chuyển bản ghi thành entity. Schema gồm hai migration: V001 tạo các bảng người dùng, API key (chỉ lưu hàm băm và tiền tố hiển thị), cluster, node, khóa API của node, model, lớp của model, runtime, nhật ký cluster/node/runtime và phiên chat; V002 thêm bảng `deployments` và trạng thái `DEGRADED` cho runtime. Bảng `deployments` cố ý không có khóa ngoại tới `runtimes` vì bản ghi runtime bị xóa khi gỡ bỏ còn deployment được giữ làm lịch sử. Trigger `set_updated_at` tự cập nhật thời điểm sửa đổi. Mỗi repository có một bản hiện thực trong bộ nhớ tương ứng dùng để kiểm thử.

#### 5.2.4 *Tiêu thụ MQTT*

`CloudContainer.subscribe_consumers()` đăng ký các handler ở Mục 4.4 trên ba wildcard state, telemetry, event. Mỗi handler kiểm tra schema bằng Pydantic trước khi ghi; thông điệp lỗi chỉ bị ghi log cảnh báo, không làm dừng consumer. Chú ý kỹ thuật: thư viện paho-mqtt cần `add_reader` của vòng sự kiện, nên trên Windows ứng dụng chủ động dùng `SelectorEventLoop` thay cho `Proactor` mặc định.

#### 5.2.5 *Kho model*

Khi biến môi trường `CLOUD_MODEL_STORE` có giá trị `huggingface`, Cloud khởi tạo repository kho model (`HuggingFaceModelRepository`) với token và endpoint được cấu hình. Repository hỗ trợ `resolve` (phân giải revision thành commit SHA và URL tải), `upload` (đẩy tệp model, trả về commit mới) và `ping`. Lời gọi thư viện đồng bộ được đưa vào thread (`asyncio.to_thread`) để không chặn API. Deployment tham chiếu model bằng `model_uri` là URL đã ghim, Edge tải trực tiếp từ kho model nên băng thông tải model không đi qua Cloud Server.

### 5.3 Xây dựng Edge Agent

#### 5.3.1 *Lắp ráp và cấu hình*

`AgentContainer` lắp ráp Agent từ cấu hình `AgentSettings` (tiền tố biến môi trường `EDGE_`). Các lựa chọn quan trọng: `runtime_driver` (`docker` hoặc `fake` để chạy không cần Docker), `runtime_network`, `image_pull_policy`, `agent_url_for_runtimes` (địa chỉ để container runtime gọi ngược về Agent, mặc định `host.docker.internal:8081`), `state_dir` lưu desired state và cache model, cùng ba chu kỳ state (10 s), telemetry (5 s), reconcile (15 s). Ứng dụng FastAPI của Agent được tạo bởi `AgentAppFactory`: lifespan khởi động `AgentService` và dừng an toàn khi tắt; health check báo trạng thái kết nối broker.

#### 5.3.2 *Giao tiếp MQTT*

`EdgePublisher` đóng gói việc publish `NodeState` (retained), telemetry và event lên topic đúng định dạng, dùng `EdgeTopics` chung. Vì broker có thể giao lại thông điệp, tính đúng đắn không dựa vào QoS mà dựa vào idempotency ở tầng ứng dụng: chống trùng `command_id` và thao tác runtime idempotent. Khi Agent tạm mất kết nối, AI Runtime vẫn

chạy bình thường vì không phụ thuộc Cloud; vòng reconcile tiếp tục bảo đảm container sống; sau khi kết nối lại, bản tin NodeState kế tiếp đồng bộ lại trạng thái trên Cloud.

### 5.3.3 *Điều khiển container*

DockerRuntimeDriver hiện thực port RuntimeDriver bằng cách gọi Docker CLI qua `asyncio.create_subprocess_exec`, hoạt động được cả khi Agent chạy trong container có gắn socket. Container được đặt tên `edgeops-<tên>-<8 ký tự đầu runtime_id>` và gắn nhãn `edgeops.runtime_id` để tìm lại sau khi Agent khởi động lại. Hàm `inspect` đọc trạng thái chạy, mã thoát, số lần restart, thời điểm khởi động và lỗi từ `docker inspect`. FakeRuntimeDriver mô phỏng cùng giao diện trong bộ nhớ, cho phép kiểm thử kịch bản container bị crash.

### 5.3.4 *Tải model*

HttpArtifactFetcher tải model theo luồng (stream) với thời gian chờ 300 s, ghi vào tệp `.part` rồi `os.replace` sang tên đích để một lần tải dở dang không bao giờ bị dùng nhầm. Thư mục cache được phân tách theo 16 ký tự đầu của SHA-256 URL. Lược đồ URI không hỗ trợ (ví dụ `s3://`) bị từ chối với thông báo yêu cầu dùng URL HTTPS có chữ ký. Việc tải được thực hiện trong bước CREATING của runtime nên lỗi tải được báo về Cloud như một deployment thất bại.

### 5.3.5 *Thu thập telemetry*

PsutilSystemCollector thu CPU, tải trung bình 1/5 phút, iowait, bộ nhớ, swap, nhiệt độ (giá trị lớn nhất của các cảm biến hệ điều hành cung cấp), dung lượng đĩa và inode, uptime, số tiến trình. Các chỉ số dạng tốc độ — số lần chuyển ngữ cảnh mỗi giây, tốc độ tăng bộ nhớ (MB/phút), thông lượng đọc/ghi đĩa và mạng (MB/s), tốc độ tăng nhiệt (°C/phút) — được tính từ chênh lệch với lần đo trước. NvidiaSmiGpuCollector truy vấn `nvidia-smi` để lấy mức sử dụng, bộ nhớ, xung nhịp, công suất và nhiệt độ GPU; các giá trị [N/A] được chuyển thành rỗng. PsutilNodeInfoProvider cung cấp hostname, IP, số nhân, dung lượng bộ nhớ và tên GPU cho NodeState.



## 5.4 Xây dựng AI Runtime

### 5.4.1 Điểm khởi chạy

Tệp `apps/edge/edge-runtime/main.py` là entypoint của container runtime. `RuntimeSettings` đọc các biến môi trường do Agent truyền vào, sau đó chọn adapter: `source=synthetic` dùng `SyntheticFrameSource`, các giá trị khác (URL RTSP, chỉ số camera USB, đường dẫn video) dùng `OpenCvFrameSource`; có `MODEL_PATH` thì dùng `UltralyticsDetector` với ngưỡng `conf` (mặc định 0,25), ngược lại dùng `SyntheticDetector`. Kết quả được gửi qua `HttpTelemetrySink` tới API cục bộ của Agent.

### 5.4.2 Suy luận YOLO

`UltralyticsDetector` nạp model một lần khi khởi động bằng `YOLO(model_path)`. Cùng một mã nguồn chạy được checkpoint PyTorch `.pt`, mô hình ONNX `.onnx` hoặc TensorRT engine `.engine`; định dạng cụ thể được quyết định bởi tệp model trong deployment. Nhờ vậy việc chuyển từ model FP32 sang engine INT8 sau tối ưu (Chương 3) chỉ là một lệnh update đổi `model_uri`, không cần build lại image. Mỗi detection gồm chỉ số lớp, tên lớp, độ tin cậy và hộp bao xyxy.

### 5.4.3 Đo lường

`InferenceMetrics` (150 dòng) được đặt ở tầng domain để kiểm thử độc lập. Lớp này ghi thời gian nhận khung, độ sáng, số khung bị bỏ, latency từng lần suy luận, số lỗi và timeout, các detection; khi `snapshot()` được gọi, nó tính FPS theo thời gian thực của cửa sổ, phân vị latency bằng sắp xếp mẫu, thống kê confidence, entropy của phân bố lớp và tốc độ thay đổi đầu ra (khoảng cách biến phân toàn phần giữa phân bố lớp của hai cửa sổ liên tiếp), rồi đặt lại bộ đếm cho cửa sổ kế tiếp. Các quy tắc suy ra trạng thái được hiện thực trực tiếp:

- Camera DISCONNECTED khi mất nguồn; FROZEN khi khung không đổi quá 2 s; DEGRADED khi FPS camera thấp hơn 70% FPS mục tiêu; còn lại ONLINE.
- Runtime DEGRADED khi tỷ lệ lỗi suy luận vượt 10% hoặc `effective_fps_ratio` dưới 0,5.

Mỗi 5 s một mẫu `RuntimeTelemetry` được gửi tới Agent.

#### 5.4.4 Đóng gói

Image runtime (`infra/docker/edge/Dockerfile.worker`) dựng trên `python:3.12-slim`, cài thư viện hệ thống cho OpenCV và cài bản Ultralytics đã được mở rộng trong đồ án (gói `packages/ultralytics`, chứa các module pruned và QAT) thay vì bản phát hành trên PyPI, để runtime nạp được checkpoint đã cắt tỉa. Trên Jetson, base image được thay bằng `14t-pytorch` có sẵn CUDA, cuDNN và TensorRT đúng phiên bản JetPack để engine TensorRT tạo trên thiết bị chạy được.

### 5.5 Xây dựng AI Assistant

#### 5.5.1 Cấu trúc dịch vụ

AI Assistant (`apps/assistant`) là một ứng dụng FastAPI riêng. Lớp `AssistantContainer` khởi tạo mô hình ngôn ngữ qua `init_chat_model` với chuỗi cấu hình dạng "`<nhà cung cấp>:<mô hình>`" (mặc định `anthropic:claude-sonnet-5` cho nút chính và `anthropic:claude-haiku-4-5` cho nút phụ, nhiệt độ 0), tạo `CloudMcpClient`, lấy danh sách công cụ qua `CloudMcpToolkit`, dựng đồ thị `ChatGraph` và `AgentOrchestrator`. Việc tách chuỗi cấu hình mô hình giúp đổi nhà cung cấp LLM mà không sửa mã.

#### 5.5.2 Kết nối MCP

`CloudMcpClient` mở một phiên MCP ngắn cho mỗi lời gọi qua streamable HTTP tới `<cloud>/mcp/` với header `X-API-Key`. Thiết kế không giữ phiên dài giúp Assistant không bị treo khi Cloud khởi động lại. Kết quả có cấu trúc (`structured_content`) được ưu tiên; nếu công cụ trả lỗi hoặc Cloud không truy cập được, lỗi được trả về cho mô hình dưới dạng dữ liệu (`{"error": ...}`) để mô hình giải thích cho người dùng.

#### 5.5.3 Prompt

Mỗi nút có prompt riêng lưu dưới dạng Markdown (`react.md`, `response.md`, `chart.md`, `suggestion.md`). Prompt của nút react quy định mỗi lượt chọn đúng một trong ba hành động: gọi tool (mặc định), hỏi lại (khi đại từ không có đối tượng trong hội thoại), hoặc trả lời thẳng (chào hỏi, câu hỏi ngoài phạm vi); không bao giờ bịa id, trạng thái hay số liệu; gọi song song các tool độc lập; liệt kê trước khi người dùng dùng tên thay vì id; sửa tham số và thử lại tối đa một lần khi lỗi do tham số; và chỉ gọi tool thay

đổi hệ thống sau khi người dùng đã xác nhận ở tin nhắn trước. Thời điểm hiện tại được chèn vào prompt để mô hình diễn giải đúng các mốc thời gian trong telemetry.

#### 5.5.4 Biểu đồ và gợi ý

Nút genchart dùng *structured output* với schema `ChartDecision`: mô hình quyết định `has_chart` và trả danh sách đặc tả biểu đồ (loại, trục, chuỗi dữ liệu). Biểu đồ không có dữ liệu ý nghĩa bị lọc bỏ. Nút suggestion sinh một số câu hỏi tiếp theo phù hợp ngữ cảnh. Cả hai nút dùng mô hình nhỏ để giảm chi phí và độ trễ.

#### 5.5.5 API và phát trực tuyến

Assistant cung cấp `POST /chat` (trả lời đồng bộ) và cặp `POST /chat/stream + GET /chat/stream/{id}` (SSE). Lướt hội thoại chạy nền và ghi sự kiện vào hàng đợi trong bộ nhớ; client đăng ký nhận đúng một lần. Header `X-Accel-Buffering: no` tắt bộ đệm của Nginx để token hiển thị ngay khi được sinh.

### 5.6 Xây dựng hệ thống giám sát

#### 5.6.1 Telemetry ứng dụng

Luồng telemetry ứng dụng đã được hiện thực đầy đủ từ đầu đến cuối: AI Runtime đo và gửi `RuntimeTelemetry` tới Agent; Agent làm giàu và publish cùng `SystemTelemetry`, `GpuTelemetry`; Cloud kiểm tra schema và lưu vào Redis (mẫu mới nhất theo loại và lịch sử 720 mẫu mỗi loại); REST API, công cụ MCP và WebSocket đọc từ đó. Sự kiện runtime, lỗi và kết quả deployment được lưu vào bảng nhật ký node trong PostgreSQL. Nhờ chung đặc tả, cùng một dữ liệu được Dashboard vẽ biểu đồ và được Assistant dùng để trả lời.

#### 5.6.2 Metric hạ tầng

Cấu hình Prometheus agent tại Edge được sinh trực tiếp trong tệp `Compose` từ biến môi trường (`EDGE_NODE_ID`, `EDGE_CLUSTER_ID`, `URL` và mật khẩu đầy metric), vì Prometheus không tự đọc biến môi trường trong tệp cấu hình. Các exporter dùng `network_mode: host` và chỉ lắng nghe trên loopback; exporter GPU được bật bằng profile `nvidia` trên node có GPU rồi. Prometheus Cloud bật `--web.enable-remote-write-receiver` và yêu cầu basic auth cho đường nhận

metric. Luật cảnh báo được nạp từ `infra/config/monitoring/alerts`, Alertmanager định tuyến theo nhãn `severity`, và Grafana được cấp `datasource`, dashboard tự động qua thư mục `provisioning`, bật `allow_embedding` để nhúng vào Dashboard.

### 5.6.3 *Dữ liệu cho cảnh báo sớm*

Đặc tả telemetry trong `docs/design/telemetry-features.md` đồng thời là hợp đồng cho bộ mô phỏng dữ liệu tổng hợp. Mỗi bản ghi huấn luyện gồm các trường telemetry và nhãn `state`, `future_degradation`, `scenario` (ví dụ `thermal_degradation`). Mười một kịch bản lỗi được liệt kê cùng nhóm đặc trưng chính để sinh dữ liệu: quá tải CPU, rò rỉ bộ nhớ, quá tải GPU, giảm xung do nhiệt, áp lực điện, đầy đĩa, suy giảm mạng, suy giảm camera, lỗi worker suy luận, trôi dữ liệu/model và mất node.

## 5.7 **Giao diện quản lý**

Web Dashboard (`apps/dashboard`) là ứng dụng một trang viết bằng React 19 và TypeScript, build bằng Vite và định kiểu bằng Tailwind CSS. Mã nguồn được tổ chức theo module chức năng; mỗi module gồm `pages`, `components`, `hooks`, `services` và `types`. Lớp `core/realtime` quản lý kết nối WebSocket và đăng ký topic; `shared/streaming` xử lý luồng SSE của Assistant. Khi triển khai, ứng dụng được phục vụ bởi Nginx, đồng thời Nginx định tuyến `/api/v1/ws` (nâng cấp WebSocket, timeout 1 giờ) và `/api/` tới Cloud, `/api/v1/chat` tới Assistant (tắt bộ đệm để phát SSE), các đường dẫn còn lại trả về `index.html` theo cơ chế SPA. Trình duyệt chỉ làm việc với một origin nên không phát sinh vấn đề CORS.

Bảng 5.4: Các màn hình chính của Web Dashboard

| Module    | Màn hình                             | Chức năng                                                                                       |
|-----------|--------------------------------------|-------------------------------------------------------------------------------------------------|
| overview  | Overview, Live Monitor, Analytics    | Tổng quan số cluster, node online/offline, runtime; theo dõi trực tiếp FPS, latency, tài nguyên |
| cluster   | Clusters, Cluster Detail             | Danh sách và chi tiết cluster cùng các node thành viên                                          |
| agent     | Edge Devices, Node Detail, Node Logs | Thông tin phần cứng, trạng thái Agent, runtime đang chạy, nhật ký sự kiện                       |
| agent     | Cameras, Camera Groups, Snapshots    | Quản lý nguồn camera cấp cho runtime và ảnh chụp kết quả                                        |
| ai        | Models, Class Config                 | Danh sách model và cấu hình lớp đối tượng của model                                             |
| assistant | AI Assistant, Chat                   | Hội thoại với AI Assistant, hiển thị trạng thái gọi tool, biểu đồ và câu hỏi gợi ý              |
| auth      | Login                                | Đăng nhập                                                                                       |

Màn hình Node Detail kết hợp dữ liệu REST (thông tin node, runtime) với luồng WebSocket đăng ký `edge/<cluster>/<node>/telemetry/+` và `edge/<cluster>/<node>/event/+`, nên biểu đồ và nhật ký cập nhật ngay khi Agent báo cáo mà không cần tải lại trang. Màn hình AI Assistant hiển thị lần lượt các bước “đang phân tích”, “đang gọi công cụ”, văn bản trả lời dần theo token, biểu đồ do nút genchart sinh và các nút câu hỏi gợi ý.

Lớp `EdgeOpsContext` có chế độ mô phỏng (mock mode): khi bật, `MockRealtimeFeed` sinh các bản tin cùng định dạng envelope với WebSocket thật và đẩy vào cùng đường xử lý. Chế độ này cho phép phát triển và trình diễn giao diện khi chưa có thiết bị Edge kết nối, mà không phải sửa mã các màn hình.

## 5.8 Kết chương

Chương 5 đã trình bày việc hiện thực nền tảng thành bốn dịch vụ chính — Cloud Server, Edge Agent, AI Runtime, AI Assistant — cùng hệ thống giám sát và Web Dashboard. Toàn bộ backend dùng chung một gói hợp đồng Pydantic; các adapter PostgreSQL, Redis, Docker, OpenCV, Ultralytics, Hugging Face đều có bản thay thế trong bộ nhớ, cho phép kiểm thử luồng triển khai mà không cần hạ tầng. Hệ thống được đóng gói thành các stack Docker Compose và triển khai tự động lên AWS EC2 bằng GitHub Actions; image Edge được build đa kiến trúc cho x86 và Jetson. Chương 6 đánh

giá kết quả tối ưu mô hình và kiểm chứng hoạt động của nền tảng.

## CHƯƠNG 6. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này đánh giá hai phần chính của đề án: hiệu quả tối ưu YOLO và khả năng vận hành của nền tảng Edge AI. Phần mô hình gồm các thực nghiệm sơ bộ đã chạy để kiểm chứng từng giai đoạn của pipeline và các phép đo trên thiết bị Jetson. Phần nền tảng được kiểm chứng bằng bộ kiểm thử tự động bao phủ luồng triển khai đầu cuối và bằng kịch bản vận hành trên hệ thống đã triển khai.

### 6.1 Môi trường thực nghiệm

#### 6.1.1 Môi trường huấn luyện

Thực nghiệm mô hình được thực hiện trên hai môi trường. Các thực nghiệm sơ bộ (pilot) để kiểm chứng đúng đắn của từng script — thừa hóa, cắt tia, phục hồi, chung cất — chạy trên máy trạm Windows chỉ dùng CPU với tập COCO128 (128 ảnh, 80 lớp; dùng chung cho train và validation). Pipeline đầy đủ trên tập Construction-PPE chạy trên GPU của Google Colab qua notebook `yolo26_ppe_train_pipeline.ipynb`, vì QAT với `pytorch-quantization` bắt buộc có CUDA.

Bảng 6.1: Môi trường thực nghiệm

| Môi trường        | Cấu hình                                                                                                                              |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Pilot (CPU)       | Windows 11, Python 3.12, PyTorch CPU, Ultralytics (bản mở rộng của đề án); COCO128; ảnh 320 hoặc 640, batch 8, <code>workers=0</code> |
| Huấn luyện đầy đủ | Google Colab GPU, đầu ra lưu trên Google Drive; Construction-PPE; ảnh 640, batch 16                                                   |
| Jetson Nano       | Jetson Nano 4GB, JetPack 4.6, power mode MAXN (10 W), quạt tản nhiệt chủ động                                                         |
| Jetson Orin Nano  | Jetson Orin Nano 8GB, JetPack 6, power mode 15 W, quạt tản nhiệt chủ động                                                             |
| Cloud             | AWS EC2 Linux, Docker Compose (Mục 5.1)                                                                                               |

#### 6.1.2 Quy trình đo trên thiết bị

Mỗi engine được đo với ảnh  $640 \times 640$ , batch 1, sau 50 lần suy luận khởi động (warm-up), trên 500 lần suy luận liên tiếp. Latency báo cáo gồm trung vị và p95; FPS tính trên cùng khoảng thời gian. Mức sử dụng GPU, bộ nhớ, công suất và nhiệt độ được

thu bằng `tegrastats` trong bài chạy 30 phút để quan sát trạng thái ổn định, không chỉ vài giây đầu. Power mode và xung nhịp được cố định bằng `nvpmode1` và `jetson_clocks` trước mỗi bài đo.

## 6.2 Kết quả tối ưu mô hình

### 6.2.1 Thực nghiệm sơ bộ trên COCO128

Các thực nghiệm sơ bộ có mục đích kiểm chứng rằng từng giai đoạn của pipeline hoạt động đúng và tìm cấu hình hợp lý trước khi chạy tốn kém trên tập PPE. Do chạy trên CPU với tập rất nhỏ (COCO128 dùng chung cho train và validation), giá trị tuyệt đối của  $mAP$  không đại diện cho bài toán PPE; kết quả được dùng để so sánh tương đối giữa các phương án. Tất cả số liệu dưới đây được đọc từ tệp `results.csv` của các run, lấy epoch có  $mAP@50:95$  cao nhất.

**Thưa hóa BatchNorm.** Bảng 6.2 cho thấy với lịch  $s$  giảm dần, mô hình vẫn duy trì độ chính xác cao trong khi  $\gamma$  bị ép về 0; đây là điều kiện để cắt tỉa không phá hỏng mô hình.

Bảng 6.2: Thực nghiệm thưa hóa BatchNorm trên COCO128 (CPU)

| Cấu hình                               | Ảnh | Epoch                | P     | R     | mAP@50 | mAP@50:95 |
|----------------------------------------|-----|----------------------|-------|-------|--------|-----------|
| Sparsity $s = 10^{-2}$ , lịch giảm dần | 640 | 94/200 (tốt nhất 87) | 0,752 | 0,691 | 0,781  | 0,552     |
| Sparsity $s = 5 \cdot 10^{-2}$ , SGD   | 320 | 30                   | 0,714 | 0,414 | 0,531  | 0,359     |

**Phục hồi sau cắt tỉa 30%.** Bảng 6.3 so sánh các phương án phục hồi mô hình đã cắt tỉa (ảnh 320, AdamW,  $lr_0 = 10^{-3}$ ). Hai điểm đáng chú ý: (1) huấn luyện phục hồi ngắn (10 epoch) gần như không phục hồi được ( $mAP \approx 0$ ), cho thấy mô hình sau cắt cần ngân sách huấn luyện đủ dài; (2) với 100 epoch fine-tune, mô hình đã cắt đạt  $mAP@50:95 = 0,383$ . Các run chung cắt dài bị dừng sớm do giới hạn thời gian CPU (mỗi epoch chung cắt tốn gấp khoảng 2 lần fine-tune vì phải chạy thêm teacher), nên chưa thể kết luận chung cắt kém hơn; ở cùng số epoch (khoảng 20), chung cắt với teacher đạt 0,185.

Bảng 6.3: Phục hồi độ chính xác sau cắt tỉa 30% trên COCO128 (CPU, ảnh 320)

| Phương án                             | Epoch chạy       | P     | R     | mAP@50 | mAP@50:95 |
|---------------------------------------|------------------|-------|-------|--------|-----------|
| Fine-tune 10 epoch                    | 10               | 0,000 | 0,000 | 0,000  | 0,000     |
| Chung cắt 10 epoch                    | 10               | 0,000 | 0,001 | 0,000  | 0,000     |
| Fine-tune 100 epoch                   | 98 (tốt nhất 87) | 0,676 | 0,448 | 0,515  | 0,383     |
| Chung cắt 100 epoch (dừng sớm)        | 21               | 0,623 | 0,238 | 0,281  | 0,185     |
| Chung cắt, teacher YOLO26x (dừng sớm) | 7                | 0,605 | 0,117 | 0,158  | 0,099     |



**Độ nhạy của trọng số chung cất.** Để chọn  $\lambda_{cwd}$ , chung cất được chạy trên YOLO26n chưa cất tia trong 5 epoch với nhiều giá trị  $\lambda$  (Bảng 6.4). Độ chính xác giảm đơn điệu khi  $\lambda$  tăng; với  $\lambda = 10$   $mAP@50:95$  chỉ còn 0,075. Trong ngân sách huấn luyện ngắn, số hạng CWD lớn lấn át mất mát phát hiện, nên  $\lambda$  nhỏ phù hợp hơn cho giai đoạn đầu; giá trị  $\lambda = 50$  mặc định chỉ phù hợp khi huấn luyện đủ dài và nên được dò lại trên tập PPE.

Bảng 6.4: Ảnh hưởng của  $\lambda_{cwd}$  (YOLO26n, 5 epoch, COCO128, ảnh 320)

| $\lambda_{cwd}$       | mAP@50 | mAP@50:95 |
|-----------------------|--------|-----------|
| 0,5                   | 0,384  | 0,252     |
| 0,5 (teacher YOLO26x) | 0,386  | 0,255     |
| 1,0                   | 0,366  | 0,227     |
| 2,0                   | 0,362  | 0,209     |
| 5,0                   | 0,307  | 0,188     |
| 10,0                  | 0,141  | 0,075     |

**Độ biến thiên theo seed.** Chung cất ( $\lambda$  nhỏ) và fine-tune thường được lặp với 3 seed (0, 1, 2), 5 epoch (Bảng 6.5). Chênh lệch trung bình giữa hai phương án ( $\approx 0,01$   $mAP@50:95$ ) nhỏ hơn độ lệch giữa các seed của cùng một phương án (tới 0,04). Vì vậy so sánh giữa các cấu hình tối ưu phải dựa trên nhiều seed hoặc huấn luyện đủ dài; kết luận từ một run ngắn duy nhất là không tin cậy.

Bảng 6.5: Độ biến thiên theo seed (YOLO26n, 5 epoch, COCO128)

| Phương án | Seed 0 | Seed 1 | Seed 2 | Trung bình | Chỉ số    |
|-----------|--------|--------|--------|------------|-----------|
| Chung cất | 0,212  | 0,247  | 0,209  | 0,223      | mAP@50:95 |
| Fine-tune | 0,248  | 0,240  | 0,211  | 0,233      | mAP@50:95 |
| Chung cất | 0,345  | 0,405  | 0,357  | 0,369      | mAP@50    |
| Fine-tune | 0,370  | 0,367  | 0,355  | 0,364      | mAP@50    |

### 6.2.2 Kết quả trên tập Construction-PPE

Bảng 6.6 tổng hợp kết quả của các cấu hình ở Bảng 3.6 trên tập test Construction-PPE, đo bằng mô hình PyTorch (fake-quant đối với cấu hình QAT). Số tham số và FLOPs là giá trị đo được từ checkpoint.

Bảng 6.6: Kết quả các cấu hình tối ưu trên tập test Construction-PPE

| Cấu hình                  | Params | GFLOPs | Precision       | mAP@50          | mAP@50:95       |
|---------------------------|--------|--------|-----------------|-----------------|-----------------|
| Baseline FP32             | 2,57 M | 6,12   | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |
| Pruned-30 + fine-tune     | 2,17 M | 4,82   | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |
| Pruned-30 + distill       | 2,17 M | 4,82   | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |
| Pruned-30 + distill + QAT | 2,17 M | 4,82   | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |

### 6.3 Benchmark trên Jetson Nano

Jetson Nano được dùng để kiểm tra cấu hình trong điều kiện tài nguyên hạn chế nhất. Vì Jetson Nano không có đơn vị tăng tốc INT8 (Bảng 2.2), cấu hình chính trên thiết bị này là FP16; engine INT8 được đưa vào để kiểm chứng nhận định rằng INT8 không đem lại lợi ích tương xứng trên kiến trúc Maxwell.

Bảng 6.7: Suy luận YOLO26n trên Jetson Nano (ảnh 640, batch 1)

| Cấu hình                  | Precision | FPS             | Latency p50/p95 (ms) | GPU Memory (MB) | Power (W)       |
|---------------------------|-----------|-----------------|----------------------|-----------------|-----------------|
| Baseline                  | FP16      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ]      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |
| Pruned-30 + distill       | FP16      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ]      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |
| Pruned-30 + distill + QAT | INT8      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ]      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |

### 6.4 Benchmark trên Jetson Orin Nano

Quy trình đo trên Jetson Orin Nano giữ nguyên dữ liệu, kích thước ảnh, batch và số vòng lặp như Jetson Nano. Engine được build lại trên chính thiết bị, không sao chép engine từ Nano.

Bảng 6.8: Suy luận YOLO26n trên Jetson Orin Nano (ảnh 640, batch 1)

| Cấu hình                  | Precision | FPS             | Latency p50/p95 (ms) | GPU Memory (MB) | Power (W)       |
|---------------------------|-----------|-----------------|----------------------|-----------------|-----------------|
| Baseline                  | FP16      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ]      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |
| Pruned-30 + distill       | FP16      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ]      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |
| Pruned-30 + distill + QAT | INT8      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ]      | [CẦN BỔ SUNG: ] | [CẦN BỔ SUNG: ] |

### 6.5 So sánh Jetson Nano và Jetson Orin Nano

Speedup giữa hai thiết bị được tính bằng tỷ số FPS Orin/Nano trên cùng cấu hình model, đọc trực tiếp từ Bảng 6.7 và Bảng 6.8. Kỳ vọng dựa trên thông số phần cứng: tỷ số nhân CUDA là 8 lần (1024 so với 128), băng thông bộ nhớ gần 2,7 lần; ngoài ra Orin Nano có Tensor Core nên cấu hình INT8 được hưởng lợi thêm, còn trên Nano cấu hình INT8 không nhanh hơn FP16 đáng kể. Do đó khoảng cách hiệu năng giữa hai thiết bị

được dự kiến lớn nhất ở cấu hình INT8. Cùng một model version được build thành hai engine khác nhau; cả hai được phân phối qua kho model với tên tệp phân biệt theo loại thiết bị và triển khai bằng cùng một quy trình trên nền tảng.

## 6.6 Benchmark Edge Runtime

Benchmark Edge Runtime đo toàn pipeline trong container AI Runtime thay vì chỉ engine TensorRT. Các chỉ số được lấy trực tiếp từ telemetry runtime mà Runtime gửi về nền tảng (Mục 4.8), nên phép đo này đồng thời kiểm chứng luồng telemetry. Nguồn vào là tệp video công trường phát ở 15 FPS (`target_fps=15`).

Bảng 6.9: Chỉ số pipeline AI Runtime lấy từ telemetry (cấu hình Pruned-30 + distill)

| Chỉ số telemetry                             | Jetson Nano (FP16) | Orin Nano (INT8) |
|----------------------------------------------|--------------------|------------------|
| <code>inference_fps</code>                   | [CẦN BỔ SUNG: ]    | [CẦN BỔ SUNG: ]  |
| <code>effective_fps_ratio</code>             | [CẦN BỔ SUNG: ]    | [CẦN BỔ SUNG: ]  |
| <code>inference_p50_ms / p95_ms</code>       | [CẦN BỔ SUNG: ]    | [CẦN BỔ SUNG: ]  |
| <code>camera_frame_drop_pct</code>           | [CẦN BỔ SUNG: ]    | [CẦN BỔ SUNG: ]  |
| <code>runtime_queue_size</code> (trung bình) | [CẦN BỔ SUNG: ]    | [CẦN BỔ SUNG: ]  |

Vì hàng đợi của Runtime giới hạn 8 khung và loại khung cũ khi đầy, latency không tăng tích lũy khi detector chậm hơn camera; thay vào đó tỷ lệ frame drop tăng. Do đó `effective_fps_ratio` và `camera_frame_drop_pct` là hai chỉ số trực tiếp cho biết cấu hình có đáp ứng thời gian thực hay không.

## 6.7 Kiểm chứng nền tảng

### 6.7.1 Kiểm thử tự động

Nền tảng được kiểm chứng bằng bộ kiểm thử `pytest` chạy trong CI. Các kiểm thử tích hợp khởi tạo Cloud và Edge Agent thật trong cùng tiến trình, nối với nhau qua bus thông điệp trong bộ nhớ, dùng `FakeRuntimeDriver` thay Docker và repository trong bộ nhớ thay PostgreSQL/Redis. Như vậy toàn bộ luồng Cloud → MQTT → Agent → container → event → Cloud được thực thi bằng mã sản phẩm, chỉ thay phần hạ tầng. Kết quả chạy bộ kiểm thử được trình bày ở Bảng 6.10.

Bảng 6.10: Kết quả kiểm thử tự động của nền tảng

| Kiểm thử                                                            | Nội dung kiểm chứng                                                               | Kết quả   |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------|
| <i>test_deployment_flow.py — luồng triển khai đầu cuối</i>          |                                                                                   |           |
| node_reports_online                                                 | Agent phát NodeState, Cloud cập nhật node ONLINE và thông tin phần cứng           | Đạt       |
| deploy_lifecycle                                                    | deploy → SUCCEEDED/RUNNING; stop, start, restart; remove → REMOVED và xóa runtime | Đạt       |
| conflicting_deploy_rejected                                         | Triển khai lại cùng tên với spec khác bị từ chối (409)                            | Đạt       |
| failed_pull_reported                                                | Lỗi pull image được báo về, deployment FAILED kèm thông điệp                      | Đạt       |
| reconcile_recreates_crashed_container                               | Container bị crash được vòng reconcile tạo lại                                    | Đạt       |
| telemetry_flows_to_cloud                                            | Telemetry từ Agent được Cloud lưu và truy vấn được                                | Đạt       |
| api_key_required                                                    | Yêu cầu thiếu API key bị từ chối                                                  | Đạt       |
| <i>test_agent_api.py — API cục bộ của Agent</i>                     |                                                                                   |           |
| local_api_and_runtime_reporting                                     | Quản lý runtime qua REST cục bộ; Runtime gửi telemetry/event qua Agent            | Đạt       |
| <i>test_mcp_tools.py, test_assistant_mcp_tools.py — công cụ MCP</i> |                                                                                   |           |
| tools_registered                                                    | Các công cụ đọc và ghi chính được đăng ký trên MCP server                         | Đạt       |
| tools_share_the_app_container                                       | Công cụ MCP dùng chung service với REST API                                       | Đạt       |
| read_only_by_default                                                | Assistant chỉ thấy công cụ đọc khi chưa cho phép ghi                              | Đạt       |
| errors_are_returned_as_data                                         | Lỗi của Cloud được trả về mô hình dưới dạng dữ liệu                               | Đạt       |
| write_tools_when_allowed_and_calls_reach_cloud                      | Công cụ ghi xuất hiện khi được phép và lời gọi tới được Cloud                     | Không đạt |
| <i>test_ws_router.py, test_shared.py — realtime và hợp đồng</i>     |                                                                                   |           |

| Kiểm thử                             | Nội dung kiểm chứng                                  | Kết quả      |
|--------------------------------------|------------------------------------------------------|--------------|
| ws_streams_matching_topics           | WebSocket chỉ đẩy thông điệp khớp bộ lọc topic       | Đạt          |
| ws_rejects_bad_api_key               | WebSocket từ chối API key sai                        | Đạt          |
| topic_matcher, edge_topics_roundtrip | So khớp wildcard MQTT; sinh và phân tích topic       | Đạt          |
| inference_metrics_snapshot           | Tính FPS, phân vị latency, trạng thái camera/runtime | Đạt          |
| <b>Tổng</b>                          | <b>18 kiểm thử</b>                                   | <b>17/18</b> |

Kiểm thử không đạt nằm ở lớp chuyển đổi kết quả công cụ phía Assistant: khi gọi `create_cluster`, kết quả nhận được là chuỗi văn bản thay vì đối tượng có cấu trúc, nên phép kiểm tra truy cập trường `name` báo lỗi `TypeError`. Hàm `result_data` trả về văn bản khi `structured_content` rỗng; cần giải mã JSON trong trường hợp này hoặc bảo đảm công cụ trả về kết quả có cấu trúc. Lỗi đã được khoanh vùng và ghi nhận để sửa; nó không ảnh hưởng tới các công cụ đọc (kiểm thử `read_only_by_default` và `errors_are_returned_as_data` vẫn đạt).

### 6.7.2 Hiệu năng vận hành

Các chỉ số thời gian của luồng quản lý được đo trên hệ thống đã triển khai (Cloud trên EC2, Edge Node qua Internet), mỗi chỉ số lấy trung vị của 20 lần thực hiện.

Bảng 6.11: Hiệu năng vận hành của nền tảng

| Chỉ số                                                                | Kết quả                                                 |
|-----------------------------------------------------------------------|---------------------------------------------------------|
| Độ trễ lệnh (Cloud publish → DeploymentEvent về Cloud, lệnh restart)  | [CẦN BỔ SUNG: ]                                         |
| Thời gian triển khai (POST deployment → SUCCEEDED, model đã có cache) | [CẦN BỔ SUNG: ]                                         |
| Thời gian tải model lần đầu (YOLO26n engine)                          | [CẦN BỔ SUNG: ]                                         |
| Thời gian phát hiện node mất kết nối                                  | 30–40 s theo thiết kế (timeout 30 s + chu kỳ quét 10 s) |
| Chu kỳ telemetry                                                      | 5 s (system, gpu, runtime)                              |
| CPU / RAM của Edge Agent khi nhàn rỗi                                 | [CẦN BỔ SUNG: ]                                         |

## 6.8 Kịch bản vận hành đầu cuối

Kịch bản đánh giá xuyên suốt liên kết kết quả tối ưu mô hình với nền tảng:

1. Đẩy engine của mô hình tối ưu lên kho model; phân giải thành URL đã ghim commit.
2. Tạo cluster, đăng ký node; khởi động Edge Agent trên thiết bị và xác nhận node chuyển ONLINE trên Dashboard.
3. Tạo deployment AI Runtime với model\_uri vừa tạo, nguồn là camera hoặc video công trường.
4. Theo dõi FPS, latency, số detection, confidence trên Dashboard qua WebSocket.
5. Dừng container runtime thủ công trên thiết bị và quan sát vòng reconcile tạo lại container.
6. Hỏi AI Assistant về trạng thái node và telemetry gần nhất; yêu cầu Assistant khởi động lại runtime và xác nhận.
7. Cập nhật deployment sang model khác, sau đó cập nhật ngược lại model cũ để khôi phục.
8. Tắt Edge Agent và xác nhận Cloud đánh dấu node OFFLINE.

Bảng 6.12: Kết quả kịch bản đầu cuối

| Tiêu chí                                    | Kiểm thử tự động   | Trên thiết bị   |
|---------------------------------------------|--------------------|-----------------|
| Node đăng ký và báo ONLINE                  | Đạt                | [CẦN BỔ SUNG: ] |
| Triển khai runtime và báo SUCCEEDED         | Đạt                | [CẦN BỔ SUNG: ] |
| Tự tạo lại container bị dừng                | Đạt                | [CẦN BỔ SUNG: ] |
| Telemetry hiển thị realtime                 | Đạt<br>(WebSocket) | [CẦN BỔ SUNG: ] |
| Assistant tra cứu và điều khiển có xác nhận | Đạt (đọc)          | [CẦN BỔ SUNG: ] |
| Cập nhật và khôi phục model                 | Đạt (update)       | [CẦN BỔ SUNG: ] |
| Phát hiện node OFFLINE                      | Theo thiết kế      | [CẦN BỔ SUNG: ] |

Một lần chạy được xem là thành công khi trạng thái trên Cloud khớp trạng thái thực tế trên thiết bị ở mọi bước, không cần thao tác SSH vào thiết bị ngoài bước chủ động gây lỗi.

## 6.9 Kết chương

Chương 6 đã trình bày kết quả thực nghiệm của hai phần. Về mô hình, cắt tỉa 30% giảm 15,7% tham số và 21,2% FLOPs của YOLO26n; các thực nghiệm sơ bộ xác nhận pipeline chạy đúng, cho thấy thừa hóa với hệ số giảm dần giữ được độ chính xác, mô hình sau cắt cần ngân sách phục hồi đủ dài, trọng số chung cắt lớn gây hại khi huấn luyện ngắn, và độ biến thiên theo seed đủ lớn để đòi hỏi so sánh nhiều lần chạy. Về nền tảng, 17/18 kiểm thử tự động đạt, bao phủ luồng triển khai đầu cuối, tự phục hồi container, telemetry, realtime và công cụ MCP; kiểm thử còn lại là lỗi chuyển đổi kết quả đã xác định nguyên nhân. Các kết quả này là cơ sở cho kết luận ở Chương 7.

## CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương cuối tổng hợp kết quả của hai trục nghiên cứu: tối ưu YOLO cho thiết bị Jetson và xây dựng nền tảng vận hành AI phân tán; đồng thời chỉ ra hạn chế và hướng phát triển.

### 7.1 Kết quả đạt được

#### 7.1.1 Tối ưu mô hình YOLO

Đồ án đã xây dựng pipeline tối ưu YOLO26n sáu giai đoạn, mỗi giai đoạn là một script độc lập và được đóng gói thành notebook chạy lại được trên Google Colab:

- Hiện thực thuật toán Sea-Horse Optimizer độc lập với framework và dùng nó để tìm đồng thời 18 siêu tham số tối ưu hóa và tăng cường dữ liệu.
- Mở rộng Ultralytics với các module cắt tỉa cho kiến trúc YOLO26 (C3k2Pruned, C2PSAPruned, SPPFPruned, DetectPruned, DetectionModelPruned), huấn luyện thừa hóa BatchNorm với hệ số giảm dần và cắt tỉa theo ngưỡng toàn cục có giới hạn an toàn. Cắt tỉa 30% giảm số tham số từ 2,57 triệu xuống 2,17 triệu ( $-15,7\%$ ) và FLOPs từ 6,12 xuống 4,82 GFLOPs ( $-21,2\%$ ).
- Hiện thực chung cất tri thức Channel-Wise Distillation kết hợp chung cất điểm phân lớp cho mô hình đã cắt tỉa.
- Hiện thực QAT cho YOLO26 đã cắt tỉa bằng pytorch-quantization, gồm các lớp lượng tử tùy chỉnh cho phép cộng dư/ghép kênh và hiệu chỉnh entropy, cho phép xuất ONNX Q/DQ sang TensorRT INT8.
- Các thực nghiệm sơ bộ rút ra ba nhận xét thực tiễn: lịch giảm dần của hệ số thừa hóa là cần thiết; mô hình sau cắt cần ngân sách phục hồi đủ dài; trọng số chung cất lớn gây hại khi huấn luyện ngắn và độ biến thiên theo seed đủ lớn để đòi hỏi so sánh nhiều lần chạy.

#### 7.1.2 Nền tảng Edge AI

Đồ án đã xây dựng và triển khai một nền tảng Cloud-Edge hoàn chỉnh:

- **Cloud Control Plane** một tiến trình phục vụ REST, WebSocket, MCP và bộ tiêu thụ MQTT; quản lý cluster, node, runtime, deployment; lưu PostgreSQL và Redis; theo dõi liveness của node.



- **Edge Agent** chạy đa kiến trúc (x86, ARM64), điều khiển container qua Docker theo mô hình desired state: thao tác idempotent, chống lệnh trùng, lưu desired state xuống đĩa, tiếp quản container sau khi khởi động lại, vòng reconcile tự tạo lại container lỗi với giới hạn số lần thử.
- **AI Runtime** chạy YOLO (PyTorch, ONNX hoặc TensorRT) trên luồng camera với hàng đợi giới hạn, tự đo hơn 25 chỉ số runtime, camera và đầu ra model, tự suy ra trạng thái DEGRADED/FROZEN.
- **Phân phối model** qua kho Hugging Face Hub ghim theo commit, tải về Edge có cache và ghi nguyên tử.
- **Giám sát** hai kênh: telemetry ứng dụng theo đặc tả thống nhất và metric hạ tầng Prometheus (agent mode, remote write) với luật cảnh báo; thiết kế mô-đun cảnh báo sớm dựa trên đặc trưng của số trượt.
- **Web Dashboard** React theo dõi realtime qua WebSocket.
- **Đóng gói và CI/CD**: các stack Docker Compose, năm workflow GitHub Actions build image lên GHCR và triển khai tự động lên AWS EC2.

Bộ 18 kiểm thử tự động chạy mã sản phẩm của Cloud và Agent qua bus trong bộ nhớ đạt 17/18, bao phủ luồng triển khai đầu cuối, tự phục hồi, telemetry, realtime, xác thực và công cụ MCP.

### 7.1.3 *AI Assistant*

AI Assistant được xây dựng trên LangGraph với đồ thị bốn nút (react–response–genchart–suggestion), truy cập hệ thống duy nhất qua 17 công cụ MCP do Cloud công bố. Công cụ chỉ định nghĩa một lần ở Cloud và dùng chung service với REST API; công cụ ghi bị ẩn theo mặc định và chỉ được gọi sau khi người dùng xác nhận; lỗi công cụ được trả về mô hình như dữ liệu. Câu trả lời được phát trực tuyến qua SSE kèm trạng thái gọi công cụ, biểu đồ và gợi ý câu hỏi.

## 7.2 Hạn chế

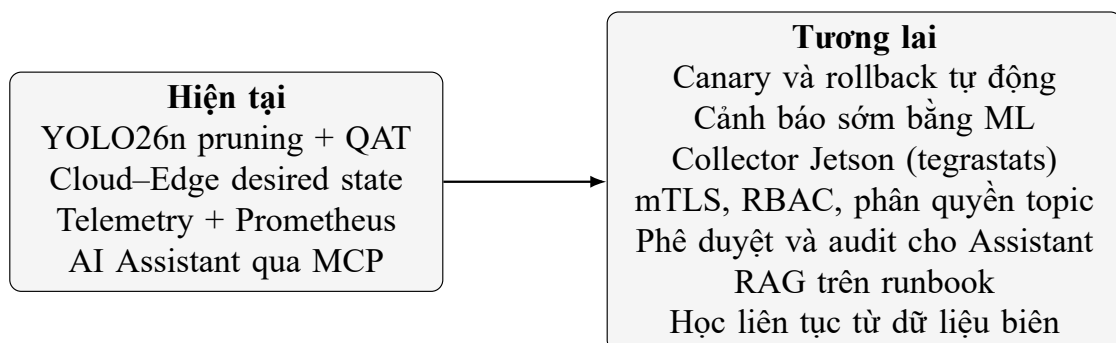
**Về mô hình.** Phần lớn thực nghiệm so sánh các phương án phục hồi và chung cất mới được chạy ở quy mô sơ bộ trên CPU với tập COCO128, nhiều run dài bị dừng sớm do giới hạn thời gian tính toán; kết luận định lượng cuối cùng phụ thuộc vào pipeline đầy đủ trên tập PPE và phép đo trên thiết bị Jetson. Mới khảo sát một tỷ lệ cắt tĩa chính (30%); không gian tìm kiếm của SHO với ngân sách 6 cá thể  $\times$  3 thế hệ và proxy 3 epoch còn nhỏ nên chưa bảo đảm tối ưu toàn cục. Tập Construction-PPE chỉ có khoảng 1.400 ảnh, chưa phản ánh đủ điều kiện ánh sáng, góc camera và khoảng cách ngoài thực tế.

**Về nền tảng.** Mỗi deployment nhắm tới một node; chưa có triển khai theo nhóm, canary hay rollback tự động theo ngưỡng telemetry — khôi phục hiện thực bằng một lệnh update về model cũ. Chưa có kiểm tra checksum của model sau khi tải ngoài việc ghim URL theo commit. Kết nối MQTT chưa bắt buộc TLS và chưa phân quyền topic theo node trên broker. Xác thực dùng API key tĩnh; bảng người dùng, API key bấm và RBAC đã có trong schema nhưng chưa được dùng ở tầng ứng dụng. Telemetry ngắn hạn trong Redis chỉ giữ 1 giờ. Mô-đun cảnh báo sớm mới dùng ở mức thiết kế và đặc tả dữ liệu. Thu telemetry GPU dùng nvidia-smi nên chưa có trên Jetson, cần bổ sung collector dựa trên tegrastats.

**Về AI Assistant.** Chất lượng phụ thuộc vào LLM thương mại và cần kết nối Internet; cơ chế xác nhận thao tác ghi dựa trên prompt, chưa có luồng phê duyệt tường minh và nhật ký kiểm toán riêng cho hành động của Assistant. Một kiểm thử chuyển đổi kết quả công cụ ghi chưa đạt.

### 7.3 Hướng phát triển

Hướng phát triển được xây dựng từ nền tảng hiện tại như Hình 7.1.



Hình 7.1: Các hướng phát triển từ kết quả hiện tại

**Hoàn thiện đánh giá mô hình.** Chạy đầy đủ pipeline trên tập PPE với nhiều seed, khảo sát thêm các tỷ lệ cắt tia 20% và 40%, mở rộng ngân sách SHO, và đo trên Jetson Nano, Jetson Orin Nano theo quy trình ở Mục 6.1.

**Triển khai theo nhóm và tự động khôi phục.** Mở rộng Deployment từ một node sang một nhóm node chọn theo nhãn, với chiến lược canary: triển khai trước cho một phần nhỏ, so sánh effective\_fps\_ratio, p95 latency, tỷ lệ lỗi với baseline rồi mới mở rộng, và tự động update về model trước khi vượt ngưỡng. Bổ sung kiểm tra SHA-256 của model sau khi tải.

**Cảnh báo sớm bằng học máy.** Sinh dữ liệu tổng hợp theo 11 kịch bản lỗi đã đặc tả,

huấn luyện Isolation Forest và XGBoost trên đặc trưng cửa sổ trượt, đánh giá thời gian cảnh báo trước và tỷ lệ báo động sai, rồi tích hợp risk score vào Dashboard và Assistant.

**Bảo mật.** Bắt buộc TLS cho MQTT, cấp credential riêng cho từng node với ACL topic, chuyển từ API key tĩnh sang tài khoản người dùng và RBAC đã có trong schema, và ký image runtime.

**AI Assistant.** Thay xác nhận dựa trên prompt bằng luồng phê duyệt tường minh: Assistant tạo kế hoạch hành động, người dùng phê duyệt trên giao diện, hệ thống thực thi và ghi nhật ký kiểm toán. Bổ sung RAG trên runbook và tài liệu vận hành cho câu hỏi về quy trình, và cho Assistant đọc risk score để chủ động đề xuất xử lý trước khi sự cố xảy ra.

# TÀI LIỆU THAM KHẢO

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.
- [2] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 7263–7271.
- [3] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [4] B. Jacob *et al.*, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [5] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *International Conference on Learning Representations*, 2016.
- [6] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, “Pruning Convolutional Neural Networks for Resource Efficient Inference,” in *International Conference on Learning Representations*, 2017.
- [7] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [8] ONNX Community. “Open Neural Network Exchange Documentation,” visited on Sept. 23, 2026. address: <https://onnx.ai/onnx/>
- [9] NVIDIA. “NVIDIA TensorRT Documentation,” visited on Sept. 23, 2026. address: <https://docs.nvidia.com/deeplearning/tensorrt/>
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. DOI: 10.1109/JIOT.2016.2579198
- [11] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017. DOI: 10.1109/MC.2017.9

- [12] OASIS Open, “MQTT Version 5.0,” Organization for the Advancement of Structured Information Standards, tech. rep., 2019. visited on Sept. 23, 2026. address: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [13] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” National Institute of Standards and Technology, tech. rep. NIST Special Publication 800-207, 2020. DOI: 10.6028/NIST.SP.800-207
- [14] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [15] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [16] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, “Learning Efficient Convolutional Networks through Network Slimming,” in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2736–2744.
- [17] C. Shu, Y. Liu, J. Gao, Z. Yan, and C. Shen, “Channel-wise Knowledge Distillation for Dense Prediction,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 5311–5320.
- [18] S. Zhao, T. Zhang, S. Ma, and M. Wang, “Sea-horse optimizer: a novel nature-inspired meta-heuristic for global optimization problems,” *Applied Intelligence*, vol. 53, pp. 11833–11860, 2023.
- [19] Ultralytics. “Ultralytics YOLO Documentation,” visited on Sept. 28, 2026. address: <https://docs.ultralytics.com/>
- [20] Anthropic. “Model Context Protocol Specification,” visited on Sept. 28, 2026. address: <https://modelcontextprotocol.io/>
- [21] LangChain. “LangGraph Documentation,” visited on Sept. 28, 2026. address: <https://langchain-ai.github.io/langgraph/>
- [22] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *International Conference on Learning Representations*, 2023.
- [23] Prometheus Authors. “Prometheus Documentation,” visited on Sept. 28, 2026. address: <https://prometheus.io/docs/>
- [24] NVIDIA. “Jetson Modules Technical Specifications,” visited on Sept. 28, 2026. address: <https://developer.nvidia.com/embedded/jetson-modules>

## PHỤ LỤC A. CẤU TRÚC SOURCE CODE

Mã nguồn gồm hai repository: repository tối ưu mô hình (bản mở rộng của Ultralytics) và repository nền tảng (gắn vào repository thứ nhất dưới dạng submodule app/).

Listing A.1: Cấu trúc repository tối ưu mô hình

```
yolo-ppe-optimizer/  
|-- optimization/          sho.py (Sea-Horse Optimizer), objective.py  
    (1 - mAP50-95)  
|-- scripts/              sho_search.py, train_sparsity.py, prune.py  
    ,  
|                          finetune.py, distill.py, qat.py,  
    val_pruned.py  
|-- ultralytics/  
|   |-- nn/modules/       block_pruned.py, head_pruned.py  
|   |-- nn/tasks_pruned.py DetectionModelPruned  
|   |-- cfg/models/26/    yolo26-pruned.yaml  
|   |-- models/yolo/detect/distill_trainer.py  CWD + cls/reg KD  
|   `-- qat/nvidia_tensorrt/  QATTrainerYolo26,  
    quant ops  
|-- notebooks/           yolo26_ppe_train_pipeline.ipynb  
|-- dataset/             ppe-custom.yaml, du lieu thu thap  
`-- app/                 (submodule) nen tang Edge AI
```

Listing A.2: Cấu trúc repository nền tảng Edge AI

```
app/  
|-- apps/                diem khoi chay  
|   |-- cloud/           main.py (REST + WS + MCP + worker)  
|   |   |-- https/router/v1/  cluster, node, agent, runtime,  
|   |   deployment, monitoring  
|   |   |-- mcp_server/tools/  fleet_tools, deployment_tools,  
|   |   monitoring_tools  
|   |   |-- ws/              realtime_router  
|   |   `-- worker/          MQTT consumer tach rieng  
|   |-- edge/edge-agent/     Edge Agent API :8081  
|   |-- edge/edge-runtime/   entrypoint container AI Runtime  
|   |-- assistant/          AI Assistant API :8100 (chat, SSE)  
|   `-- dashboard/          React + Vite + Tailwind, nginx.conf  
|-- src/                  nghiep vu
```

```
|  |-- shared/contracts/          commands, state, telemetry, events,  
|    topics, enums  
|  |-- cloud/{domain,application,infrastructure}/  
|  |-- edge/{domain,application,infrastructure}/  
|  `-- assistant/agents/         chat_graph, nodes/, prompts/,  
|    tool_runner  
|-- packages/                    messaging (MQTT), http,  
|    observability, ultralytics  
|-- db/migrations/              V001__init.sql, V002__deployments.  
|    sql  
|-- infra/  
|  |-- compose/{cloud,edge,assistant,web}/  
|  |-- docker/{cloud,edge,assistant}/  
|  `-- config/                  mosquitto.conf, monitoring/ (  
|    prometheus, alerts, grafana)  
|-- docs/                       architecture, api, design/telemetry  
|    -features.md  
|-- tests/                      18 kiem thu pytest  
`-- .github/workflows/          ci, ci-edge, deploy-cloud, deploy-  
|    assistant, deploy-web
```

## PHỤ LỤC B. DATABASE SCHEMA

Schema PostgreSQL được tạo bởi hai migration trong db/migrations. Khóa chính là UUID sinh bởi `gen_random_uuid()` (extension `pgcrypto`); mọi bảng có `created_at`, phần lớn có `updated_at` cập nhật bằng trigger; thuộc tính mở rộng lưu kiểu JSONB.

Bảng B.1: Các bảng nghiệp vụ chính

| Bảng                                        | Khóa/quan hệ chính                                        | Vai trò                                                                                        |
|---------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------|
| users                                       | id; username, email duy nhất                              | Người dùng hệ thống                                                                            |
| api_keys                                    | id, user_id → users                                       | Khóa API: chỉ lưu key_hash và key_prefix; loại USER, NODE_AGENT, SERVICE; trạng thái, hạn dùng |
| clusters                                    | id; name duy nhất                                         | Edge Cluster, trạng thái, vị trí                                                               |
| nodes                                       | id, cluster_id → clusters;<br>(cluster_id, name) duy nhất | Edge Node: hostname, IP (INET), phiên bản Agent, CPU, RAM, GPU, heartbeat cuối                 |
| node_api_keys                               | id, node_id → nodes                                       | Khóa riêng cho Agent, tách khỏi khóa người dùng                                                |
| models                                      | id                                                        | Thông tin model, trạng thái ACTIVE, INACTIVE hoặc DEPRECATED                                   |
| model_classes                               | model_id → models                                         | Danh sách lớp đối tượng của model                                                              |
| runtimes                                    | id, node_id → nodes                                       | AI Runtime: loại, trạng thái, image, model, config, env, giới hạn tài nguyên, container_id     |
| deployments                                 | id, cluster_id, node_id, runtime_id (không FK)            | Desired state (spec JSONB), trạng thái, last_command_id, thông điệp                            |
| cluster_logs,<br>node_logs,<br>runtime_logs | Khóa ngoại tới tài nguyên tương ứng                       | Nhật ký sự kiện theo mức DEBUG–CRITICAL                                                        |



| Bảng                            | Khóa/quan hệ chính | Vai trò                                                           |
|---------------------------------|--------------------|-------------------------------------------------------------------|
| chat_sessions,<br>chat_messages | session_id         | Lịch sử hội thoại với vai trò<br>USER, ASSISTANT, SYSTEM,<br>TOOL |

Các kiểu enum (node\_status, runtime\_status, deployment\_status...) có giá trị trùng khớp với enum Python trong src/shared/contracts/enums.py. Chỉ mục được tạo trên các cột lọc thường dùng: nodes(cluster\_id), nodes(status), nodes(last\_heartbeat\_at), deployments(node\_id), deployments(runtime\_id), deployments(status).

Listing B.1: Bảng deployments (V002)

```
CREATE TABLE deployments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  cluster_id UUID NOT NULL REFERENCES clusters(id) ON DELETE
  CASCADE,
  node_id    UUID NOT NULL REFERENCES nodes(id)    ON DELETE
  CASCADE,
  runtime_id UUID NOT NULL,                        -- không FK: giu lịch su sau
  khi runtime bi xoa
  spec JSONB NOT NULL,
  status deployment_status NOT NULL DEFAULT 'PENDING',
  last_command_id VARCHAR(64),
  message TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

## PHỤ LỤC C. API SPECIFICATION

Mọi endpoint (trừ /health) yêu cầu header X-API-Key. Tài liệu OpenAPI tương tác được FastAPI sinh tự động tại /docs của từng dịch vụ.

Bảng C.1: Cloud REST API

| Method | Endpoint                                              | Mục đích                                                               |
|--------|-------------------------------------------------------|------------------------------------------------------------------------|
| GET    | /health                                               | Kiểm tra database, Redis, MQTT                                         |
| GET    | /api/v1/clusters                                      | Liệt kê cluster                                                        |
| POST   | /api/v1/clusters                                      | Tạo cluster (409 nếu trùng tên)                                        |
| GET    | /api/v1/clusters/{id}                                 | Chi tiết cluster                                                       |
| GET    | /api/v1/nodes?cluster_id=                             | Liệt kê node                                                           |
| POST   | /api/v1/nodes                                         | Đăng ký node vào cluster                                               |
| GET    | /api/v1/nodes/{id}                                    | Chi tiết node                                                          |
| GET    | /api/v1/agents/{node_id}                              | Trạng thái Agent và các runtime                                        |
| GET    | /api/v1/runtimes?node_id=                             | Liệt kê runtime                                                        |
| GET    | /api/v1/runtimes/{id}                                 | Chi tiết runtime                                                       |
| POST   | /api/v1/runtimes/{id}/{start stop restart}            | Điều khiển runtime                                                     |
| POST   | /api/v1/deployments                                   | Tạo deployment (202)                                                   |
| GET    | /api/v1/deployments?node_id=                          | Liệt kê deployment                                                     |
| GET    | /api/v1/deployments/{id}                              | Chi tiết deployment                                                    |
| PUT    | /api/v1/deployments/{id}                              | Cập nhật từng trường (202)                                             |
| DELETE | /api/v1/deployments/{id}                              | Gỡ bỏ (202)                                                            |
| GET    | /api/v1/monitoring/nodes/{id}/telemetry               | Telemetry mới nhất theo loại                                           |
| GET    | /api/v1/monitoring/nodes/{id}/telemetry/{kind}?limit= | Lịch sử telemetry (1–1000)                                             |
| GET    | /api/v1/monitoring/nodes/{id}/events?limit=           | Event gần đây                                                          |
| WS     | /api/v1/ws?topics=...                                 | Luồng realtime theo bộ lọc topic; khóa qua tham số api_key hoặc header |

Bảng C.2: Edge Agent API (cổng 8081) và AI Assistant API (cổng 8100)

| Method | Endpoint                            | Mục đích                                                      |
|--------|-------------------------------------|---------------------------------------------------------------|
| GET    | /status                             | NodeState hiện tại                                            |
| GET    | /runtimes, /runtimes/{id}           | Trạng thái runtime                                            |
| POST   | /runtimes                           | Tạo runtime từ RuntimeSpec                                    |
| PUT    | /runtimes/{id}                      | Cập nhật runtime                                              |
| DELETE | /runtimes/{id}                      | Gỡ runtime                                                    |
| POST   | /runtimes/{id}/{start stop restart} | Điều khiển runtime                                            |
| POST   | /runtimes/{id}/telemetry            | Runtime gửi RuntimeTelemetry (202)                            |
| POST   | /runtimes/{id}/events               | Runtime gửi event (202)                                       |
| POST   | /api/v1/chat                        | Hỏi đáp đồng bộ                                               |
| POST   | /api/v1/chat/stream                 | Khởi tạo lượt, trả stream_id                                  |
| GET    | /api/v1/chat/stream/{stream_id}     | SSE: status, tool_end, token, done, chart, suggestions, error |

Listing C.1: Ví dụ yêu cầu tạo deployment

```
POST /api/v1/deployments
X-API-Key: <key>
{
  "node_id": "5b0c...e1",
  "name": "ppe-detector",
  "type": "SAFETY",
  "image_uri": "ghcr.io/hopquangdo/edgeops/edge-runtime",
  "image_tag": "latest",
  "model_uri": "https://huggingface.co/<repo>/resolve/<commit>/yolo26n_pruned_int8.engine",
  "config": {"source": "rtsp://camera-01/stream", "target_fps": 15, "conf": 0.3},
  "resource_limits": {"runtime": "nvidia", "memory": "2g"}
}
```

## PHỤ LỤC D. MCP TOOL SPECIFICATION

MCP server của Cloud được mount tại /mcp (streamable HTTP), yêu cầu header x-api-key. Schema tham số của mỗi công cụ được sinh từ chữ ký hàm Python; kết quả là entity tương ứng dưới dạng có cấu trúc.

Bảng D.1: Danh sách công cụ MCP

| Tool                  | Loại      | Tham số và chức năng                                                                                      |
|-----------------------|-----------|-----------------------------------------------------------------------------------------------------------|
| list_clusters         | Đọc       | Liệt kê cluster                                                                                           |
| get_cluster           | Đọc       | cluster_id                                                                                                |
| create_cluster        | Ghi       | name, description?, location?, metadata?                                                                  |
| list_nodes            | Đọc       | cluster_id?; trạng thái và heartbeat cuối                                                                 |
| get_node              | Đọc       | node_id                                                                                                   |
| register_node         | Ghi       | cluster_id, name, metadata?                                                                               |
| list_runtimes         | Đọc       | node_id?                                                                                                  |
| get_runtime           | Đọc       | runtime_id                                                                                                |
| list_deployments      | Đọc       | node_id?                                                                                                  |
| get_deployment        | Đọc       | deployment_id                                                                                             |
| create_deployment     | Ghi       | node_id, name, image_uri, type, image_tag, model_id?, model_uri?, config?, environment?, resource_limits? |
| update_deployment     | Ghi       | deployment_id và các trường cần đổi                                                                       |
| remove_deployment     | Ghi (hủy) | deployment_id; dừng và xóa runtime                                                                        |
| control_runtime       | Ghi       | runtime_id, action ∈ {start, stop, restart}                                                               |
| get_latest_telemetry  | Đọc       | node_id; mẫu mới nhất theo loại                                                                           |
| get_telemetry_history | Đọc       | node_id, kind, limit=100                                                                                  |
| get_node_events       | Đọc       | node_id, limit=50                                                                                         |

Mô tả của mọi công cụ ghi chứa câu “Only call after the user explicitly confirmed”. Phía Assistant, công cụ có tiền tố create\_, update\_, remove\_, register\_, control\_ chỉ được đưa cho mô hình khi cấu hình allow\_writes bật.

## PHỤ LỤC E. MODEL TRAINING CONFIGURATION

Listing E.1: Cấu hình pipeline tối ưu trong notebook

```
# --- du lieu va mo hinh ---
MODEL      = weights/yolo26n.pt          # student / diem xuat phat
TEACHER     = weights/yolo26l.pt         # teacher chung cat
DATA        = construction-ppe.yaml      # 1132/143/141 anh, 11 lop
IMG_SIZE    = 640
BATCH       = 16

# --- SHO + baseline ---
BASELINE_EPOCHS = 100
SHO_POP, SHO_GMAX, SHO_PROXY_EPOCHS = 6, 3, 3

# --- sparsity (Network Slimming) ---
SPARSITY_EPOCHS = 200
SPARSITY_SR      = 1e-2                  # giam dan: sr * (1 - 0.9 *
    epoch / epochs)

# --- pruning ---
PRUNE_RATIO = 0.30

# --- phuc hoi ---
RECOVERY_MODE    = 'distill'             # hoac 'finetune'
RECOVERY_EPOCHS  = 100
LAMBDA_CWD       = 50.0
KD_TEMPERATURE   = 4.0

# --- QAT (TensorRT INT8) ---
QAT_EPOCHS       = 20
QAT_BATCH        = 8
QAT_LRO          = 1e-3                  # chia 100 ben trong
    QATTrainerYolo26
QAT_CALIB_BATCHES = 256                  # hieu chinh entropy
```

Listing E.2: Trình tự lệnh của pipeline

```
python scripts/sho_search.py --model yolo26n.pt --data construction
    -ppe.yaml \
    --pop 6 --gmax 3 --proxy-epochs 3 --full-epochs 100 --imgsz
```

```
640 --batch 16
python scripts/train_sparsity.py --weights <baseline best.pt> --
    epochs 200 --sr 1e-2
python scripts/prune.py          --weights sparsity.pt --model-size n
    --prune-ratio 0.30
python scripts/distill.py        --teacher yolo26l.pt --student-
    checkpoint yolo26_pruned.pt \
    --epochs 100 --lambda-cwd 50 --temperature 4.0
python scripts/qat.py            --checkpoint recovered.pt --epochs 20
    --batch 8 \
    --lr0 1e-3 --calib-batches 256
```

Mỗi run lưu `args.yaml` (toàn bộ tham số, seed, deterministic) và `results.csv` (loss và chỉ số theo epoch) cùng thư mục `weights/` chứa `best.pt` và `last.pt`.

## PHỤ LỤC F. BENCHMARK CONFIGURATION

Bảng F.1: Cấu hình tối thiểu của một lần benchmark

| Tham số         | Giá trị cần lưu                                        |
|-----------------|--------------------------------------------------------|
| Device identity | Model thiết bị, serial ẩ danh, RAM, storage            |
| Software        | OS, JetPack, CUDA, cuDNN, TensorRT, driver             |
| Power/thermal   | Power mode, clock, fan, nhiệt độ môi trường            |
| Model           | Version, checksum, precision, input shape              |
| Input           | Dataset/video/camera, độ phân giải, FPS nguồn          |
| Runtime         | Batch, queue size, threshold, NMS                      |
| Protocol        | Warm-up, số vòng, thời gian chạy, số lần lặp           |
| Metrics         | Median, p90, p95, FPS, memory, GPU, power, temperature |

Script benchmark phải ghi kết quả thô ở định dạng có cấu trúc và tách bước thu thập khỏi bước tạo bảng/biểu đồ. Mọi lần loại bỏ outlier cần nêu quy tắc trước, không xóa mẫu chỉ vì kết quả không thuận lợi.

## PHỤ LỤC G. DEPLOYMENT CONFIGURATION

Listing G.1: Triển khai Cloud trên máy chủ

```
# 1. Ha tang: PostgreSQL, Redis, Mosquitto (migration tu chay lan dau
)
docker compose -f infra/compose/cloud/docker-compose.infra.yml \
    --env-file infra/compose/cloud/.env up -d
# 2. Cloud API (REST + WS + MCP + embedded worker), cong 18000 ->
8000
docker compose -f infra/compose/cloud/docker-compose.yml \
    --env-file infra/compose/cloud/.env up -d
# 3. AI Assistant va Web Dashboard
docker compose -f infra/compose/assistant/docker-compose.yml \
    --env-file infra/compose/assistant/.env up -d
docker compose -f infra/compose/web/docker-compose.yml up -d
# 4. (tuy chon) Prometheus, Alertmanager, Grafana
docker compose -f infra/compose/cloud/docker-compose.monitoring.yml \
    --env-file infra/compose/cloud/.env up -d
```

Listing G.2: Cấu hình Edge Node (infra/compose/edge/.env)

```
EDGE_CLUSTER_ID=<uuid cluster>
EDGE_NODE_ID=<uuid node da dang ky tren Cloud>
EDGE_MQTT_HOST=<dia chi Cloud>
EDGE_MQTT_PORT=11883
EDGE_API_KEYS=<khoa cho API cuc bo>
EDGE_RUNTIME_DRIVER=docker
EDGE_IMAGE_PULL_POLICY=missing
EDGE_STATE_DIR=/var/lib/edge-agent
# giam sat ha tang (tuy chon)
MONITORING_METRICS_URL=<URL remote write cua Prometheus tren Cloud>
MONITORING_PUSH_PASSWORD=<mat khau>
```

Listing G.3: Khởi chạy Edge Agent và giám sát trên thiết bị

```
docker compose -f infra/compose/edge/docker-compose.yml \
    --env-file infra/compose/edge/.env up -d
# node co GPU roi: them --profile nvidia de bat dcgm-exporter
docker compose -f infra/compose/edge/docker-compose.monitoring.yml \
    --env-file infra/compose/edge/.env up -d
```



Tệp `.env` chứa mật khẩu, API key, token Hugging Face và khóa LLM; trong CI/CD, nội dung tệp được lưu trong GitHub Secrets và chỉ ghi ra máy chủ khi triển khai, không đưa vào repository hay phụ lục.